HTML Interview Questions and Answers – Beginner to Advanced

Part 1: HTML

Beginner to Advanced

1. HTML Fundamentals – Basic Questions

Q1. What is HTML?

Answer (Simple English):

HTML stands for HyperText Markup Language. It is the standard markup language used to create the structure of web pages. It defines elements such as headings, paragraphs, images, links, tables, and forms.

Marathi Explanation:

HTML ही एक Markup Language आहे. Website चा basic structure तयार करण्यासाठी HTML वापरतात. Website वर कोणता text, image, button, link दाखवायचा हे HTML ठरवते.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Website</title>
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This is my first webpage.</p>
</body>
</html>
```

Industry Use: Website चे structure तयार करण्यासाठी HTML वापरतात. React मध्ये JSX लिहितानाही HTML सारखी syntax वापरली जाते.

Tricky Follow-up:

Is HTML a programming language?

Answer: No. HTML is a markup language, not a programming language. It defines the structure of content but does not perform programming logic like loops, conditions, or calculations.

Q2. What is the difference between HTML and HTML5?

Answer:

HTML5 is the latest major version of HTML. It introduced new semantic elements, multimedia support, form input types, and browser APIs.

HTML (Older versions)	HTML5
Limited semantic elements	Supports semantic elements
External plugins often needed for media	Native audio and video support
Limited input types	New input types like email, date
Less browser API support	Supports APIs like Canvas and Geolocation

Marathi Explanation:

HTML5 मध्ये नवीन features आले आहेत. उदाहरणार्थ, आधी video दाखवण्यासाठी external plugins लागायचे; HTML5 मध्ये <video> tag वापरून video दाखवता येतो.

Example:

```
<!-- HTML5 Video -->
<video controls width="400">
  <source src="video.mp4" type="video/mp4">
</video>

<!-- HTML5 Input -->
<input type="email">
<input type="date">
```

Industry Use: Modern websites, web applications, media players आणि forms तयार करताना HTML5 features वापरतात.

Q3. What is the basic structure of an HTML document?

Answer:

An HTML document contains a standard structure with a document declaration, HTML root element, head section, and body section.

Example:

```
<!DOCTYPE html>

<html lang="en">


<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>My Website</title>

</head>


<body>

  <h1>Hello World</h1>

  <p>Welcome to HTML.</p>

</body>


</html>
```

Marathi Explanation:

- <!DOCTYPE html> – Browser ला HTML5 document असल्याचे सांगते.
- <html> – संपूर्ण HTML document चा root element.
- <head> – Page ची information ठेवतो.
- <title> – Browser tab मध्ये title दाखवतो.
- <meta> – Character encoding आणि viewport सारखी माहिती देतो.
- <body> – Website वर दिसणारा content ठेवतो.

Industry Use: प्रत्येक HTML page ची सुरुवात या basic structure ने केली जाते.

Tricky Follow-up:

What happens if we don't write <!DOCTYPE html>?

Answer: The browser may use Quirks Mode, where some CSS and layout behavior can differ from modern standards.

Q4. What is the difference between HTML elements and HTML tags?

Answer:

An HTML tag is the markup written inside angle brackets. An HTML element generally includes the opening tag, content, and closing tag.

Example:

Hello World

- – Opening tag
- Hello World – Content
- – Closing tag
- Complete Hello World – HTML element

Marathi Explanation:

Tag म्हणजे किंवा आणि या tags सोबत content मिळून Element तयार होतो.

Tricky Follow-up: Is
 an element?

Answer:
 is a void element. It does not have a closing tag or contain content.

Q5. What are HTML attributes?

Answer:

HTML attributes provide additional information about HTML elements. They are usually written inside the opening tag as name-value pairs.

Example:

Visit Google

<img src="profile.jpg" alt="User Profile" width="200">

Marathi Explanation:

Attributes मुळे HTML elements ला extra information देता येते.

- href – Link चा address.
- target – Link कुठे open करायची.
- src – Image किंवा media file चा path.
- alt – Image load न झाल्यास alternative text.
- width – Image ची रुंदी.

Industry Use: Links, images, forms, buttons आणि accessibility features मध्ये attributes वापरतात.

Q6. What is the difference between Block-level and Inline elements?

Answer:

Block-level elements start on a new line and usually take the full available width.

Inline elements do not start on a new line and take only the space required by their content.

Example:

<!-- Block Elements -->

First Div

First Paragraph

<h1>Heading</h1>

<!-- Inline Elements -->

Hello

Important

Click Here

Marathi Explanation:

- Block: नवीन line पासून सुरू होतात आणि available width घेतात.
- Inline: त्यांना लागेल तेवढीच जागा घेतात; नवीन line सुरू करत नाहीत.

Block Elements Inline Elements

<p>	<a>
<h1>	
<section>	

Tricky Follow-up: Is <img> block or inline?

Answer: By default, <img> is an inline-level replaced element.

Q7. What is the difference between and ?

Answer:

 is a generic block-level container, while is a generic inline container. Both are used to group content for styling or scripting.

Example:

```
<div class="card">
  <h2>Product Name</h2>
  <p>Product Description</p>
</div>

<p>
  This product is
  <span style="color: green;">Available</span>.
</p>
```

Marathi Explanation:

- – मोठा section किंवा container तयार करण्यासाठी.
- – Text मधील एखादा छोटा भाग style करण्यासाठी.

Industry Use: React components, cards, layouts आणि text styling मध्ये वापरतात.

Q8. What is the difference between <id> and <class>?

Answer:

The id attribute identifies a unique element, while the class attribute can be shared by multiple elements.

Example:

```
<h1 id="main-title">Welcome</h1>


<p class="description">First paragraph</p>

<p class="description">Second paragraph</p>
```

CSS:

```
#main-title {
  color: blue;
}


.description {
  color: gray;
}
```

Marathi Explanation:

- id – साधारणपणे एका unique element साठी वापरतात.
- class – अनेक elements ला एकच CSS style देण्यासाठी वापरतात.

Important Interview Point: HTML मध्ये एका page वर ID unique असावा. एका element ला multiple classes देता येतात.

```
<div id="product" class="card featured active"></div>
```

2. HTML Text Formatting and Lists

Q9. What are heading tags in HTML?

Answer:

HTML provides six heading tags, from <h1> to <h6>. <h1> represents the highest-level heading, while <h6> represents the lowest-level heading.

Example:

```
<h1>Main Heading</h1>

<h2>Section Heading</h2>

<h3>Subsection Heading</h3>

<h4>Heading 4</h4>
```

<h5>Heading 5</h5>

<h6>Heading 6</h6>

Marathi Explanation:

Website मधील headings दाखवण्यासाठी हे tags वापरतात. <h1> हा मुख्य heading असतो आणि <h6> हा सर्वात कमी level चा heading असतो.

Industry Use: Content structure, SEO आणि accessibility साठी headings महत्त्वाचे आहेत.

Tricky Follow-up: Can we use multiple <h1> tags?

Answer: HTML5 allows multiple <h1> elements, but a clear and logical heading hierarchy is recommended for accessibility and content structure.

Q10. What is the difference between and ?

Answer:

Both usually display text in bold, but they have different meanings.

- makes text visually bold without adding importance.
- indicates that the text has strong importance.

Example:

Product Name

Warning: Payment Failed!

Marathi Explanation:

 फक्त text bold दाखवतो, तर त्या text ला महत्त्व आहे हे सांगतो.

Q11. What is the difference between and ?

Answer:

- represents text in an alternate voice or mood, often displayed in italic.
- represents emphasized text and carries semantic importance.

Example:

Technical Term

Please read this carefully.

Marathi Explanation:

दोन्ही text italic दाखवू शकतात; पण वापरल्याने त्या text वर विशेष जोर दिला जातो.

Q12. What are HTML Lists?

Answer:

HTML provides three main types of lists:

1. Ordered List –
2. Unordered List –
3. Description List – <dl>

Example:

```
<!-- Ordered List -->
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ol>

<!-- Unordered List -->
<ul>
  <li>React</li>
  <li>Node.js</li>
  <li>MongoDB</li>
</ul>

<!-- Description List -->
<dl>
  <dt>HTML</dt>
  <dd>Markup language for web pages.</dd>
</dl>
```

Marathi Explanation:

- – Numbering असलेली list.
- – Bullet points असलेली list.
- <dl> – Term आणि त्याचे description दाखवण्यासाठी.

Industry Use: Navigation menus, product features, steps, FAQs आणि documentation मध्ये lists वापरतात.

3. HTML Links and Images

Q13. What is an Anchor Tag?

Answer:

The <a> tag is used to create hyperlinks that connect one page or resource to another.

Example:

```
<a href="https://www.google.com">
  Visit Google
</a>
```

Important Attributes:

Attribute	Purpose
href	Destination URL
target	Where to open the link
rel	Relationship/security information
download	Suggests downloading a resource

Marathi Explanation:

एका page वरून दुसऱ्या page वर जाण्यासाठी Anchor tag वापरतात.

Open link in new tab:

```
<a
  href="https://example.com"
  target="_blank"
  rel="noopener noreferrer"
```

>

Open Website

Tricky Follow-up: What is the difference between _self and _blank?

- _self – Same tab मध्ये open होते.
- _blank – नवीन tab किंवा browsing context मध्ये open होते.

Q14. What is the Image Tag in HTML?

Answer:

The <img> tag is used to display images on a webpage. It is a void element and does not require a closing tag.

Example:

<img

src="profile.jpg"

alt="Profile Photo"

width="300"

height="200"

>

Important Attributes:

- src – Image location.
- alt – Alternative text.
- width – Image width.
- height – Image height.
- loading="lazy" – Allows deferred loading of off-screen images.

Marathi Explanation:

Website वर image दाखवण्यासाठी <img> tag वापरतात. alt attribute accessibility आणि image load न झाल्यास पर्यायी माहिती देण्यासाठी महत्त्वाचा आहे.

Industry Example:

<img

src="product.jpg"

alt="Black Wireless Headphones"

loading="lazy"

>

Q15. What is the difference between Absolute URL and Relative URL?

Answer:

An absolute URL contains the complete address of a resource, while a relative URL points to a resource based on the current page's location.

Example:

<!-- Absolute URL -->

About

<!-- Relative URL -->

About

<!-- Relative image path -->

<img src="./images/profile.jpg" alt="Profile">

Marathi Explanation:

- Absolute URL – पूर्ण website address असतो.
- Relative URL – Current project किंवा page च्या location नुसार path दिला जातो.

Industry Use: React projects, website navigation आणि assets जोडण्यासाठी relative paths वापरतात.

4. HTML Tables

Q16. How do you create a table in HTML?

Answer:

HTML tables are used to display data in rows and columns.

Important Tags:

- <table> – Creates a table.

- <tr> – Creates a row.
- <th> – Creates a header cell.
- <td> – Creates a data cell.
- <thead> – Groups header rows.
- <tbody> – Groups body rows.
- <tfoot> – Groups footer rows.

Example:

```
<table border="1">
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Role</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>1</td>
      <td>Manoj</td>
      <td>Developer</td>
    </tr>

    <tr>
      <td>2</td>
      <td>Rahul</td>
      <td>Designer</td>
    </tr>
```

```
  </tbody>
</table>
```

Marathi Explanation:

Table मध्ये data rows आणि columns मध्ये दाखवतात. <tr> म्हणजे row, <th> म्हणजे heading cell आणि <td> म्हणजे normal data cell.

Industry Use: Admin dashboards, reports, employee lists आणि transaction history.

Q17. What is the difference between colspan and rowspan?

Answer:

- colspan allows a table cell to span multiple columns.
- rowspan allows a table cell to span multiple rows.

Example:

```
<table border="1">
  <tr>
    <th colspan="2">Student Details</th>
  </tr>

  <tr>
    <td rowspan="2">Manoj</td>
    <td>HTML</td>
  </tr>

  <tr>
    <td>CSS</td>
  </tr>
</table>
```

Marathi Explanation:

- colspan="2" – एक cell दोन columns मध्ये पसरतो.
- rowspan="2" – एक cell दोन rows मध्ये पसरतो.

5. Semantic HTML

Q18. What is Semantic HTML?

Answer:

Semantic HTML means using HTML elements that clearly describe the meaning and purpose of their content.

Example:

```
<header>
  <h1>My Website</h1>
</header>

<nav>
  <a href="/">Home</a>
  <a href="/about">About</a>
</nav>

<main>
  <article>
    <h2>Latest News</h2>
    <p>News content goes here.</p>
  </article>
</main>

<footer>
  <p>Copyright 2026</p>
</footer>
```

Marathi Explanation:

Semantic tags वापरल्यामुळे browser, search engines आणि screen readers यांना त्या section चा उपयोग समजतो.

Important Semantic Tags:

Tag	Purpose
<header>	Introductory content
<nav>	Navigation links
<main>	Main page content
<section>	Related content section
<article>	Independent content
<aside>	Related side content
<footer>	Footer information
<figure>	Self-contained media/content
<figcaption>	Figure caption

Industry Use: SEO-friendly websites, blogs, news websites आणि accessible web applications.

Q19. What is the difference between <section>, <article>, and ?

Answer:

- <section> groups related content under a common theme.
- <article> represents independent, self-contained content.
- is a generic container with no semantic meaning.

Example:

```
<section>
  <h2>Latest Articles</h2>

  <article>
    <h3>Introduction to React</h3>
    <p>React is a JavaScript library.</p>
  </article>
```

```
  <article>
    <h3>Introduction to Node.js</h3>
    <p>Node.js runs JavaScript outside the browser.</p>
  </article>
</section>

<div class="container">
  Generic container
</div>
```

Marathi Explanation:

- Section – एकाच विषयाशी संबंधित content.
- Article – स्वतंत्रपणे समजू शकणारा content.
- Div – Generic container; स्वतःचा semantic अर्थ नसतो.

Q20. What is the difference between <header> and <head>?

Answer:

<head> contains metadata and resources for the HTML document. <header> represents introductory content visible within the webpage.

Example:

```
<head>
  <title>My Website</title>
  <meta charset="UTF-8">
</head>

<body>
  <header>
    <h1>Welcome to My Website</h1>
  </header>
</body>
```

Marathi Explanation:

<head> मधील माहिती साधारणपणे webpage वर थेट दिसत नाही. <header> हा webpage चा visible भाग असतो.

6. HTML Forms – Very Important for Interviews

Q21. What is an HTML Form?

Answer:

An HTML form is used to collect user input and submit it to a server for processing.

Example:

```
<form action="/register" method="POST">

  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>

  <button type="submit">Register</button>

</form>
```

Marathi Explanation:

User कडून information घेण्यासाठी form वापरतात. उदाहरणार्थ, Login, Registration, Contact आणि Checkout forms.

Important Attributes:

Attribute	Meaning
action	Form data कुठे submit करायचा
method	HTTP method – GET किंवा POST
name	Submitted data ची key

required	Field भरणे आवश्यक
placeholder	Input मधील hint
value	Input ची value
autocomplete	Browser autofill behavior

Industry Use: Login, registration, payment, contact आणि admin forms.

Q22. What is the difference between GET and POST methods?

Answer:

GET is generally used to retrieve data from a server, while POST is generally used to submit data or request the creation of a resource.

GET	**POST**
Data usually goes in URL query parameters	Data usually goes in request body
Can be cached	Not normally cached by default
Can be bookmarked with its URL	Request body is not stored in the URL
Used for fetching data	Used for submitting data
Has URL length limitations	Supports larger request bodies, subject to server limits

Example:

```
<!-- GET -->
<form action="/search" method="GET">
  <input type="text" name="query">
  <button type="submit">Search</button>
</form>

<!-- POST -->
<form action="/register" method="POST">
  <input type="text" name="username">
```

```
  <button type="submit">Register</button>
</form>
```

Marathi Explanation:

- GET – Data मागवण्यासाठी वापरतात. Search करण्यासाठी उपयोगी.
- POST – Server कडे data submit करण्यासाठी वापरतात. Registration साठी उपयोगी.

Important Interview Point: POST वापरल्याने data automatically secure होत नाही. Sensitive data साठी HTTPS आणि server-side validation आवश्यक आहे.

Q23. What are different HTML input types?

Answer:

HTML provides different input types to collect different kinds of user information.

Important Input Types:

Input Type	Purpose
text	Text input
password	Password input
email	Email input
number	Numeric input
tel	Telephone number
date	Date selection
time	Time selection
file	File upload
checkbox	Multiple selections
radio	Single selection from a group
range	Slider input
color	Color selection
submit	Form submission

hidden Hidden form value

Example:

```
<form>
  <input type="text" placeholder="Enter Name">
  <input type="email" placeholder="Enter Email">
  <input type="password" placeholder="Enter Password">
  <input type="date">
  <input type="file">

  <button type="submit">Submit</button>
</form>
```

Industry Use: Registration forms, profile updates, product filters आणि document uploads.

Q24. What is the difference between Radio Button and Checkbox?

Answer:

A radio button allows the user to select one option from a group. A checkbox allows the user to select multiple independent options.

Example:

```
<!-- Radio Buttons -->
<p>Select Gender:</p>

<input type="radio" name="gender" value="male">
<label>Male</label>

<input type="radio" name="gender" value="female">
<label>Female</label>
```

```
<!-- Checkboxes -->
<p>Select Skills:</p>

<input type="checkbox" name="skills" value="html">
<label>HTML</label>

<input type="checkbox" name="skills" value="css">
<label>CSS</label>

<input type="checkbox" name="skills" value="javascript">
<label>JavaScript</label>
```

Marathi Explanation:

- Radio Button – एका group मधून एकच option निवडता येतो. Radio buttons साठी समान name असणे महत्त्वाचे.
- Checkbox – एकापेक्षा जास्त options निवडता येतात.

Q25. What is the difference between readonly and disabled?

Answer:

Both prevent users from editing an input, but they behave differently.

readonly	**disabled**
User cannot edit the value	User cannot edit the value
Usually focusable	Not focusable
Value is submitted with the form	Value is not submitted
Can be selected/copied	Generally cannot be interacted with

Example:

```
<input type="text" value="Manoj" readonly>

<input type="text" value="Developer" disabled>
```

Marathi Explanation:

readonly मध्ये user value बदलू शकत नाही, पण form submit झाल्यावर value पाठवली जाते.

disabled मध्ये field पूर्णपणे inactive असतो आणि form submit करताना त्याची value पाठवली जात नाही.

Tricky Follow-up: Can we use readonly on a checkbox?

Answer: No. The readonly attribute does not apply to checkbox and radio inputs.

Q26. What is the difference between required and pattern attributes?

Answer:

- required makes a field mandatory.
- pattern validates the input against a regular expression.

Example:

```
<form>
  <input
    type="text"
    name="username"
    required
    minlength="3"
  >
  <input
    type="text"
    name="code"
    pattern="[A-Z]{3}[0-9]{3}"
    title="Enter 3 capital letters followed by 3 digits"
  >

  <button type="submit">Submit</button>
</form>
```

Marathi Explanation:

- required – Field रिकामा ठेवता येत नाही.
- pattern – ठरावीक format प्रमाणे value असावी.

Important: HTML validation user experience साठी उपयोगी आहे; पण security साठी backend वरही validation करणे आवश्यक आहे.

7. Advanced HTML Questions

Q27. What is the difference between localStorage, sessionStorage, and cookies?

Answer:

These are browser storage mechanisms, but they differ in lifetime, access, and how data is sent to the server.

Feature	localStorage	sessionStorage	Cookies
Lifetime	Until cleared	Until tab session ends	Depends on expiry
Storage	Browser	Browser tab session	Browser
Automatically sent with HTTP requests	No	No	Yes, when matching rules apply
Typical use	Preferences	Temporary tab data	Sessions and authentication

Example:

```
// localStorage
localStorage.setItem("username", "Manoj");

// sessionStorage
sessionStorage.setItem("token", "abc123");

// Read data
console.log(localStorage.getItem("username"));

// Remove data
```

localStorage.removeItem("username");

Marathi Explanation:

- localStorage – Browser बंद करून पुन्हा उघडला तरी data राहतो.
- sessionStorage – Tab session संपल्यावर data clear होतो.
- Cookies – Server सोबत session information share करण्यासाठी उपयोगी.

MERN Industry Use: Login sessions, user preferences आणि temporary data management.

Important Security Point: Sensitive authentication tokens localStorage मध्ये ठेवण्याऐवजी योग्य परिस्थितीत HttpOnly, Secure, SameSite cookie वापरणे अधिक सुरक्षित ठरू शकते.

Q28. What is the difference between async and defer attributes in script?

Answer:

Both async and defer allow external JavaScript files to download without blocking HTML parsing during the download.

- async: Executes as soon as the script is downloaded. Execution order is not guaranteed.
- defer: Executes after HTML parsing is complete, preserving the order of deferred scripts.

Example:

<!-- Async -->

<script src="analytics.js" async></script>

<!-- Defer -->

<script src="app.js" defer></script>

Marathi Explanation:

async script download झाल्यावर लगेच execute होते. त्यामुळे script चा order बदलू शकतो.

defer script HTML parsing पूर्ण झाल्यानंतर execute होते आणि defer scripts चा order maintain होतो.

Industry Use: Analytics scripts साठी async आणि DOM वर अवलंबून असलेल्या scripts साठी defer उपयोगी आहे.

Q29. What is the difference between localStorage and HttpOnly Cookies?

Answer:

JavaScript can access localStorage, but it cannot directly read an HttpOnly cookie. HttpOnly cookies are automatically included in matching HTTP requests by the browser.

Example:

```
// JavaScript can access localStorage

localStorage.setItem("theme", "dark");


// JavaScript cannot read an HttpOnly cookie

console.log(document.cookie);
```

Marathi Explanation:

MERN application मध्ये authentication करताना HttpOnly cookie उपयोगी असते. JavaScript मधून ती वाचता येत नाही, त्यामुळे XSS attack मधून token चोरी होण्याचा धोका कमी होतो. मात्र HttpOnly cookie वापरल्याने XSS किंवा CSRF चे सर्व धोके संपत नाहीत.

Q30. What is SEO in HTML?

Answer:

SEO stands for Search Engine Optimization. HTML helps search engines understand webpage content through proper headings, metadata, semantic elements, and descriptive links.

Example:

```
<head>
  <title>Best MERN Stack Course</title>

  <meta
    name="description"
    content="Learn MongoDB, Express, React and Node.js."
  >

  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
</head>


<body>
  <main>
    <h1>MERN Stack Development</h1>
    <p>Learn full-stack web development.</p>
  </main>
</body>
```

Marathi Explanation:

Search engines ला आपल्या website चा content समजण्यासाठी योग्य title, description, headings आणि semantic HTML वापरतात.

Important SEO Elements:

- <title>
- Meta description
- Heading hierarchy
- Semantic HTML
- Image alt
- Descriptive URLs
- Internal links

Tricky Follow-up: Does adding meta keywords improve Google ranking?

Answer: Google does not use the meta keywords tag as a ranking signal.

Q31. What is the difference between <script>, <style>, and <link>?

Answer:

- <script> – Includes or writes JavaScript.
- <style> – Writes internal CSS.
- <link> – Connects external resources, commonly CSS files.

Example:

```
<head>
```

```
  <!-- External CSS -->
  <link rel="stylesheet" href="style.css">

  <!-- Internal CSS -->
  <style>
    h1 {
      color: blue;
    }
  </style>
</head>

<body>
  <h1>Hello</h1>

  <!-- JavaScript -->
  <script src="app.js"></script>
</body>
```

Marathi Explanation:

HTML मध्ये CSS आणि JavaScript जोडण्यासाठी हे tags वापरतात.

Q32. What is an iframe in HTML?

Answer:

An iframe (inline frame) is used to embed another webpage or external resource inside the current webpage.

Example:

```
<iframe
  src="https://example.com"
  width="600"
  height="400"
```

```
  title="Example Website"
>
</iframe>
```

Marathi Explanation:

एका webpage च्या आत दुसरी webpage किंवा supported external content दाखवण्यासाठी iframe वापरतात.

Industry Use: Google Maps embeds, video embeds आणि external tools.

Security Point: External iframe वापरताना sandbox, appropriate permissions आणि trusted sources यांचा विचार करावा.

8. HTML5 Important Features

Q33. What is the Canvas element?

Answer:

The HTML <canvas> element provides a drawing area that can be controlled using JavaScript. It is commonly used for graphics, animations, and games.

Example:

```
<canvas id="myCanvas" width="300" height="150"></canvas>


<script>
  const canvas = document.getElementById("myCanvas");
  const ctx = canvas.getContext("2d");


  ctx.fillStyle = "blue";
  ctx.fillRect(20, 20, 150, 80);
</script>
```

Marathi Explanation:

Canvas मध्ये JavaScript वापरून shapes, graphics आणि animations तयार करू शकतो.

Q34. What is the difference between Audio and Video tags?

Answer:

The <audio> tag embeds audio content, while the <video> tag embeds video content.

Example:

```
<!-- Audio -->
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
</audio>

<!-- Video -->
<video controls width="400">
  <source src="movie.mp4" type="video/mp4">
</video>
```

Important Attributes:

- controls – Player controls दाखवतो.
- autoplay – Media automatically play करण्याचा प्रयत्न करतो.
- muted – Audio mute करतो.
- loop – Media पुन्हा play करतो.
- poster – Video सुरू होण्यापूर्वी preview image दाखवतो.

Q35. What is the viewport meta tag?

Answer:

The viewport meta tag controls how a webpage is displayed on different screen sizes, especially mobile devices.

Example:

```
<meta
  name="viewport"
  content="width=device-width, initial-scale=1.0"
>
```

Marathi Explanation:

Mobile, tablet आणि desktop वर website योग्य प्रकारे दिसण्यासाठी viewport meta tag वापरतात.

Industry Use: Responsive website development साठी हा tag अत्यंत महत्त्वाचा आहे.

9. Practical HTML Coding Interview Questions

Q36. Create a simple Registration Form.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Form</title>
</head>
<body>

  <h1>User Registration</h1>

  <form action="/register" method="POST">

    <label for="name">Full Name:</label>
    <input type="text" id="name" name="name" required>

    <br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <br><br>

    <label for="password">Password:</label>
```

```
    <input type="password" id="password" name="password" required>

    <br><br>

    <label for="dob">Date of Birth:</label>
    <input type="date" id="dob" name="dob">

    <br><br>

    <button type="submit">Register</button>

  </form>

</body>
</html>
```

Interview Explanation:

This form collects user information such as name, email, password, and date of birth. The required attribute ensures that mandatory fields are filled before browser submission.

Q37. Create a simple Navigation Bar.

```
<!DOCTYPE html>
<html>
<head>
  <title>Navigation Bar</title>
</head>
<body>

  <header>
    <h2>My Website</h2>
```

```
    <nav>
      <a href="/">Home</a>
      <a href="/about">About</a>
      <a href="/services">Services</a>
      <a href="/contact">Contact</a>
    </nav>
  </header>


</body>
</html>
```

Interview Explanation:

The <nav> element represents a section containing navigation links. It helps users navigate between different pages of a website.

10. HTML Tricky Interview Questions

Question	**Correct Answer**
Is HTML case-sensitive?	HTML tag and attribute names are generally case-insensitive.
Is HTML whitespace important?	Multiple normal spaces in HTML text are generally collapsed into one space.
Can an element have multiple classes?	Yes.
Can an element have multiple IDs?	No. An element should have only one ID value.
Is a self-closing tag?	It is a void element; no closing tag is required.
What is the use of alt?	Provides alternative text for images.
What is the use of target="_blank"?	Opens a link in a new browsing context.

What is the use of data-*?	Stores custom data on HTML elements.
What is the use of autocomplete?	Controls browser autofill behavior.
What is the use of required?	Makes a form field mandatory.
What is the use of placeholder?	Displays a hint inside an input.
What is the use of contenteditable?	Makes an element's content editable by the user.

Additional practical example – data-*:

```
<button data-product-id="101">
  Add to Cart
</button>
const button = document.querySelector("button");


console.log(button.dataset.productId); // 101
```

CSS Interview Questions and Answers – Beginner to Advanced

Part 2: CSS

Beginner to Advanced

1. CSS Fundamentals – Basic Questions

Q1. What is CSS?

Answer (Simple English):

CSS stands for Cascading Style Sheets. It is used to style and design HTML elements. CSS controls colors, fonts, spacing, layouts, animations, and responsive designs.

Marathi Explanation:

CSS चा full form Cascading Style Sheets आहे. HTML website चा structure तयार करते, तर CSS त्या website ला सुंदर आणि attractive बनवते.

Example:

```
<h1>Hello World</h1>

<p>Welcome to my website.</p>
```

```
h1 {
  color: blue;
  font-size: 32px;
}


p {
  color: gray;
  font-size: 18px;
}
```

Industry Use: Website design, responsive layouts, dashboards, forms, animations आणि UI development.

Q2. What are the different types of CSS?

Answer:

There are three main ways to apply CSS to HTML.

1. Inline CSS
2. Internal CSS
3. External CSS

1. Inline CSS

CSS is written directly inside the HTML element using the style attribute.

```
<h1 style="color: red;">Hello World</h1>
```

2. Internal CSS

CSS is written inside the <style> tag in the HTML document.

```
<head>
  <style>
    h1 {
      color: blue;
    }
  </style>
</head>
```

3. External CSS

CSS is written in a separate .css file and connected using the <link> tag.

```
<link rel="stylesheet" href="style.css">
```

style.css

```
h1 {
  color: green;
}
```

Marathi Explanation:

- Inline – HTML tag मध्येच CSS लिहिणे.
- Internal – HTML file च्या <style> tag मध्ये CSS लिहिणे.
- External – वेगळी CSS file तयार करून HTML ला जोडणे.

Industry Use: Professional projects मध्ये maintainability आणि code reusability साठी External CSS वापरतात.

Interview Follow-up: Which CSS method is best?

Answer: External CSS is generally preferred for larger applications because it improves code organization, reuse, and maintenance.

Q3. What is the CSS Syntax?

Answer:

CSS syntax consists of a selector and a declaration block. Each declaration contains a property and a value.

Example:

```
h1 {
  color: red;
  font-size: 30px;
}
```

Explanation:

- h1 – Selector
- color – Property
- red – Value
- font-size – Property
- 30px – Value
- { } – Declaration block

Marathi Explanation:

CSS मध्ये कोणत्या HTML element ला style द्यायची आहे ते selector ठरवतो. Property कोणती style बदलायची ते सांगते आणि Value त्या property ची setting ठरवते.

Q4. What are CSS Selectors?

Answer:

CSS selectors are used to select HTML elements that we want to style.

Important CSS Selectors:

Selector	Meaning	Example
Universal	Selects all elements	*
Element	Selects by tag name	p
Class	Selects by class	.box
ID	Selects by ID	#header
Group	Selects multiple elements	h1, p
Descendant	Selects nested elements	.box p
Child	Selects direct children	.box > p
Adjacent sibling	Selects immediate next sibling	h1 + p
General sibling	Selects following siblings	h1 ~ p

Example:

```
<div class="container">
  <h1 id="title">Welcome</h1>
  <p>First Paragraph</p>
  <p>Second Paragraph</p>
</div>
```

```
* {
  box-sizing: border-box;
}

h1 {
  color: blue;
}

.container {
  padding: 20px;
}
```

```
#title {
  font-size: 32px;
}

.container p {
  color: gray;
}

.container > p {
  margin-bottom: 10px;
}
```

Marathi Explanation:

Selectors वापरून आपण specific HTML elements निवडतो आणि त्यांना CSS apply करतो.

Industry Use: React components, navigation, cards, forms आणि responsive UI styling.

Q5. What is the difference between ID Selector and Class Selector?

Answer:

An ID selector targets an element with a specific ID, while a class selector targets all elements with a specific class.

Example:

```
<h1 id="heading">Welcome</h1>

<p class="text">Hello</p>
<p class="text">World</p>

#heading {
  color: red;
}
```

```
.text {
  color: blue;
}
```

ID	Class
Uses #	Uses .
Should be unique per page	Can be reused
Higher specificity	Lower specificity
One ID value per element	Multiple classes per element

Important: एका element ला multiple classes देता येतात.

```
<div class="card active featured"></div>
```

Q6. What is CSS Specificity?

Answer:

CSS Specificity is a scoring system used by browsers to determine which CSS rule applies when multiple rules target the same element.

Specificity Priority:

1. Inline styles
2. ID selectors
3. Class, attribute and pseudo-class selectors
4. Element and pseudo-element selectors

Example:

```
<p id="text" class="message">Hello World</p>
```

```
p {
  color: green;
}


.message {
  color: blue;
```

```
}


#text {
   color: red;
}
```

Output: Red

Marathi Explanation:

एकाच element ला वेगवेगळ्या selectors मधून CSS मिळत असेल, तर browser specificity च्या आधारावर कोणता rule लागू करायचा ते ठरवतो.

Specificity Score:

Selector	Score
Inline style	1000 (conceptual)
ID	100
Class / Attribute / Pseudo-class	10
Element / Pseudo-element	1
Universal selector	0

Example:

```
#box .title p {
   color: red;
}
```

Specificity = 100 + 10 + 1 = 111.

Tricky Follow-up: Does specificity always decide which style wins?

Answer: No. The CSS cascade first considers origin, importance, and cascade layers. Specificity is compared within the applicable cascade rules. If specificity is equal, source order can decide the winner.

Q7. What is the difference between !important and normal CSS?

Answer:

The !important declaration gives a CSS declaration priority over normal declarations within the applicable cascade context.

Example:

```
p {
  color: blue !important;
}


p {
  color: red;
}
```

Output: Blue.

Marathi Explanation:

!important वापरल्यावर normal CSS declaration पेक्षा त्या declaration ला जास्त priority मिळते. मात्र शक्यतो याचा जास्त वापर करू नये.

Industry Use: Existing third-party styles override करण्यासाठी कधीकधी वापरतात. पण maintainability साठी टाळणे चांगले.

2. CSS Box Model

Q8. What is the CSS Box Model?

Answer:

The CSS Box Model describes how every HTML element is represented as a rectangular box. It consists of four parts:

1. Content
2. Padding
3. Border
4. Margin

Visual Structure:

Margin

Border

Padding

Content

CSS Box Model – outside to inside: Margin → Border → Padding → Content.

Example:

```
.box {
  width: 200px;
  height: 100px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
```

Marathi Explanation:

- Content – Actual text किंवा image.
- Padding – Content आणि border मधील space.
- Border – Element ची boundary.
- Margin – Element च्या बाहेरील space.

Important Interview Point:

Default content-box मध्ये:

Total width = Content width + Left/Right padding + Left/Right border.

वरील example मध्ये:

200 + 20 + 20 + 5 + 5 = 250px total width.

Q9. What is the difference between content-box and border-box?

Answer:

content-box includes only content in the declared width and height. Padding and border are added outside it.

border-box includes content, padding, and border within the declared width and height.

Example:

```
.box1 {
```

```
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: content-box;
}

.box2 {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: border-box;
}
```

Property	Total Width
content-box	250px
border-box	200px

Marathi Explanation:

border-box वापरल्यावर padding आणि border मुळे element ची declared width वाढत नाही.

Industry Best Practice:

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

यामुळे layout calculations सोपे होतात.

Q10. What is the difference between Margin and Padding?

Answer:

Margin creates space outside an element's border, while padding creates space between the content and its border.

Example:

```
.card {
  margin: 20px;
  padding: 15px;
}
```

Marathi Explanation:

- Margin – दोन elements मधील बाहेरील अंतर.
- Padding – element च्या आत content भोवती असलेली जागा.

Tricky Follow-up: Does margin have a background color?

Answer: No. An element's background does not extend into its margin. It can extend behind its padding and border depending on background settings.

Q11. What is Margin Collapse?

Answer:

Margin collapse happens when vertical margins of certain block-level elements combine into a single margin instead of adding together.

Example:

```
.box1 {
  margin-bottom: 30px;
}

.box2 {
  margin-top: 20px;
}
```

Output: In the usual adjoining vertical margin case, the gap is 30px, not 50px.

Marathi Explanation:

दोन block elements चे vertical margins एकमेकांना भेटले, तर ते नेहमी add होत नाहीत. सामान्य परिस्थितीत मोठा margin लागू होतो.

Important: Horizontal margins do not collapse. Flexbox आणि Grid items मध्येही margins collapse होत नाहीत.

3. CSS Display Property

Q12. What is the CSS Display Property?

Answer:

The display property specifies how an HTML element is displayed and how it participates in layout.

Important Values:

Value	Description
block	Starts on a new line and takes available width
inline	Stays within the text flow
inline-block	Inline flow with block-like sizing
none	Removes element from layout
flex	Creates a flex container
grid	Creates a grid container

Example:

```
.block {
  display: block;
}

.inline {
  display: inline;
}

.inline-block {
  display: inline-block;
}
```

```
.hidden {
  display: none;
}
```

Marathi Explanation:

display property ठरवते की element page वर कसा दिसेल आणि layout मध्ये कसा behave करेल.

Q13. What is the difference between display: none and visibility: hidden?

Answer:

display: none removes the element from the layout, while visibility: hidden hides the element but preserves its layout space.

Example:

```
.box1 {
  display: none;
}

.box2 {
  visibility: hidden;
}
```

display: none	**visibility: hidden**
Element is hidden	Element is hidden
Layout space is removed	Layout space is preserved
Element does not participate in layout	Element still occupies layout space

Marathi Explanation:

display: none केल्यावर element ची जागाही निघून जाते.

visibility: hidden केल्यावर element दिसत नाही, पण त्याची जागा कायम राहते.

Industry Use: Conditional UI visibility, dropdowns आणि hidden components.

Q14. What is the difference between inline, block, and inline-block?

Answer:

Feature	Block	Inline	Inline-block
New line	Yes	No	No
Width / Height	Supported	Generally not applied	Supported
Vertical padding	Supported	Applies visually, but doesn't increase line-box height in the same way	Supported
Example			Styled

Example:

```
.box1 {
  display: block;
  width: 200px;
}

.box2 {
  display: inline;
}

.box3 {
  display: inline-block;
  width: 200px;
}
```

Marathi Explanation:

inline-block मध्ये inline सारखे elements एकाच line मध्ये राहू शकतात आणि block सारखी width-height देखील देता येते.

4. CSS Position Property

Q15. What is the CSS Position Property?

Answer:

The CSS position property determines how an element is positioned in a document.

Five Main Values:

1. static
2. relative
3. absolute
4. fixed
5. sticky

Example:

```
.box {
  position: relative;
  top: 20px;
  left: 30px;
}
```

Marathi Explanation:

Element page वर कुठे ठेवायचा आणि तो normal document flow मध्ये कसा behave करेल हे position ठरवते.

Q16. Explain Static, Relative, Absolute, Fixed, and Sticky.

1. Static

Default positioning. top, left, right, and bottom do not reposition a static element.

```
.box {
  position: static;
}
```

2. Relative

Element remains in normal flow but can be visually moved relative to its original position.

```
.box {
  position: relative;
```

```
  top: 20px;
  left: 20px;
}
```

3. Absolute

Element is removed from normal flow and positioned relative to its nearest positioned ancestor, or the initial containing block if none exists.

```
.parent {
  position: relative;
}

.child {
  position: absolute;
  top: 0;
  right: 0;
}
```

4. Fixed

Element is removed from normal flow and generally positioned relative to the viewport.

```
.button {
  position: fixed;
  bottom: 20px;
  right: 20px;
}
```

5. Sticky

Element behaves like a relatively positioned element until it reaches a specified scroll threshold, then sticks within its scrolling container.

```
.header {
  position: sticky;
  top: 0;
}
```

Marathi Explanation:

Position	**सोपा अर्थ**
Static	Default position
Relative	स्वतःच्या original position पासून move होते
Absolute	Nearest positioned parent च्या संदर्भात position होते
Fixed	Viewport च्या संदर्भात fixed राहते
Sticky	Scroll केल्यावर ठरावीक position ला चिकटते

Industry Use: Sticky navbar, floating buttons, product badges, modal overlays आणि dropdown menus.

Q17. What is the difference between Relative and Absolute positioning?

Answer:

Relative positioning keeps the element in normal document flow. Absolute positioning removes the element from normal document flow.

Example:

```
<div class="parent">
  Parent
  <div class="child">Child</div>
</div>
.parent {
  position: relative;
  width: 300px;
  height: 200px;
  background: lightgray;
}

.child {
  position: absolute;
```

```
  top: 10px;
  right: 10px;
}
```

Marathi Explanation:

Parent ला position: relative दिल्यामुळे child चा absolute position त्या parent च्या संदर्भात ठरतो.

Tricky Follow-up: Does an absolute element always position relative to its parent?

Answer: No. It positions relative to its nearest ancestor that establishes an appropriate containing block, commonly a positioned ancestor.

Q18. What is Z-index?

Answer:

The z-index property controls the stacking order of overlapping elements. An element with a higher stacking level is generally displayed in front of one with a lower stacking level within the relevant stacking context.

Example:

```
.box1 {
  position: absolute;
  z-index: 1;
}

.box2 {
  position: absolute;
  z-index: 10;
}
```

Output: Box 2 appears above Box 1, assuming they are in the same stacking context.

Marathi Explanation:

दोन elements एकमेकांवर overlap होत असतील, तर कोणता element पुढे दिसेल हे z-index ठरवते.

Important Interview Point: A child with a very high z-index cannot escape its parent's stacking context.

5. CSS Flexbox

Q19. What is Flexbox in CSS?

Answer:

Flexbox is a one-dimensional CSS layout system used to arrange elements in a row or column. It provides easy alignment, spacing, and distribution of items.

Example:

```
<div class="container">
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
</div>
```

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 20px;
}
```

Marathi Explanation:

Flexbox वापरून एका row किंवा column मध्ये elements व्यवस्थित arrange करता येतात. त्यांची alignment आणि spacing सहज control करता येते.

Industry Use: Navbar, cards, buttons, headers, login forms आणि dashboard layouts.

Q20. What are the important Flexbox properties?

Answer:

Flexbox properties are divided into two categories:

A. Parent Properties (Flex Container)

Property	**Purpose**
display	Creates flex container

flex-direction	Sets row or column direction
justify-content	Aligns items along main axis
align-items	Aligns items along cross axis
flex-wrap	Controls wrapping
align-content	Aligns multiple flex lines
gap	Adds spacing between items

B. Child Properties (Flex Items)

Property	**Purpose**
flex-grow	Controls how much item can grow
flex-shrink	Controls how much item can shrink
flex-basis	Sets initial main size
flex	Shorthand for grow, shrink, basis
align-self	Overrides individual item alignment
order	Changes visual order

Q21. What is the difference between justify-content and align-items?

Answer:

justify-content aligns flex items along the main axis, while align-items aligns items along the cross axis.

Example:

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 300px;
}
```

Marathi Explanation:

Flexbox मध्ये:

- justify-content – Main axis वर alignment.
- align-items – Cross axis वर alignment.

Important: flex-direction: row असेल तर main axis horizontal असतो. flex-direction: column असेल तर main axis vertical होतो.

Flexbox Containers | Litho

fblitho.com

Q22. Explain the difference between flex-grow, flex-shrink, and flex-basis.

Answer:

- flex-grow controls how much an item can grow when extra space is available.
- flex-shrink controls how much an item can shrink when space is insufficient.
- flex-basis defines the initial main size of a flex item.

Example:

```
.item1 {
  flex-grow: 1;
}

.item2 {
  flex-shrink: 0;
}

.item3 {
  flex-basis: 200px;
}
```

Marathi Explanation:

- Grow – Extra जागा असेल तर item किती वाढेल.
- Shrink – जागा कमी असेल तर item किती लहान होईल.

- Basis – सुरुवातीला item चा main size किती असेल.

Tricky Follow-up: What does flex: 1 mean?

Answer: It is a shorthand commonly equivalent to flex: 1 1 0% in browsers, allowing the item to grow and shrink from a zero-percent flex basis.

Q23. How do you center a div horizontally and vertically?

Answer:

The easiest modern method is Flexbox or CSS Grid.

Using Flexbox:

```
.parent {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

Using Grid:

```
.parent {
  display: grid;
  place-items: center;
  min-height: 100vh;
}
```

Marathi Explanation:

justify-content: center horizontal center करते आणि align-items: center vertical center करते, जेव्हा flex direction row असतो.

Industry Use: Login page, modal, loading screen आणि hero section.

6. CSS Grid

Q24. What is CSS Grid?

Answer:

CSS Grid is a two-dimensional layout system used to create layouts with rows and columns.

Example:

```
<div class="grid-container">
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
  <div>Box 4</div>
</div>
```

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 20px;
}
```

Marathi Explanation:

CSS Grid वापरून rows आणि columns दोन्हीमध्ये layout तयार करता येतो.

Industry Use: Admin dashboard, product listing, gallery, page layout आणि analytics dashboard.

Q25. What is the difference between Flexbox and Grid?

Answer:

Flexbox is mainly a one-dimensional layout system, while Grid is a two-dimensional layout system.

Flexbox	**Grid**
One-dimensional	Two-dimensional
Row or column layout	Rows and columns together
Content-based alignment	Layout-based alignment
Best for navbar and component alignment	Best for complete page layouts

Marathi Explanation:

Flexbox एका direction मध्ये layout करण्यासाठी चांगले आहे. Grid मध्ये एकाच वेळी rows आणि columns control करता येतात.

Interview Example:

- Navbar – Flexbox.
- Product listing – Grid.
- Complete dashboard layout – Grid + Flexbox.

Q26. What is the fr unit in CSS Grid?

Answer:

The fr unit represents a fraction of the available space in a grid container.

Example:

```
.container {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
}
```

Marathi Explanation:

वरील example मध्ये तीन columns आहेत. उपलब्ध जागेचे 1:2:1 प्रमाणात वाटप होते, subject to grid sizing constraints.

Q27. What is the difference between auto-fit and auto-fill?

Answer:

Both are used with CSS Grid's repeat function to create responsive layouts.

- auto-fill creates as many tracks as can fit, preserving empty tracks.
- auto-fit collapses empty tracks so occupied tracks can expand.

Example:

```
.container {
  display: grid;
  grid-template-columns: repeat(
    auto-fit,
    minmax(200px, 1fr)
```

```
  );
  gap: 20px;
}
```

Marathi Explanation:

दोन्ही responsive grid तयार करण्यासाठी उपयोगी आहेत. auto-fit रिकाम्या columns collapse करू शकतो, तर auto-fill रिकाम्या tracks साठी जागा ठेवतो.

7. CSS Units and Colors

Q28. What are CSS Units?

Answer:

CSS units define the size of properties such as width, height, margin, padding, and font size.

Two Main Types:

Absolute Units

Unit	Meaning
px	CSS pixels
cm	Centimeter
mm	Millimeter
in	Inch
pt	Point

Relative Units

Unit	Meaning
%	Relative to a reference dimension
em	Relative to the element's font size, with property-specific inheritance behavior
rem	Relative to the root element's font size
vw	1% of viewport width
vh	1% of viewport height

vmin 1% of smaller viewport dimension

vmax 1% of larger viewport dimension

Example:

```
.container {
  width: 80%;
  font-size: 1rem;
  padding: 2em;
  height: 100vh;
}
```

Marathi Explanation:

CSS मध्ये size देण्यासाठी वेगवेगळी units वापरतात.

- px – Fixed CSS pixel unit.
- % – Reference dimension च्या तुलनेत.
- em – संबंधित element च्या font size वर आधारित.
- rem – Root (html) font size वर आधारित.
- vh – Viewport height च्या 1% प्रमाणे.
- vw – Viewport width च्या 1% प्रमाणे.

Q29. What is the difference between em and rem?

Answer:

em is relative to the relevant element's font size, while rem is relative to the root element's font size.

Example:

```
html {
  font-size: 16px;
}


.parent {
```

```
  font-size: 20px;
}

.child {
  font-size: 2em;
  padding: 2rem;
}
```

Output:

- 2em = 40px because the parent font size is 20px.
- 2rem = 32px because the root font size is 16px.

Marathi Explanation:

em मुळे nested elements मध्ये size बदलू शकतो. rem नेहमी root font size वर आधारित असल्यामुळे मोठ्या project मध्ये consistent sizing साठी उपयोगी आहे.

Industry Best Practice: Typography आणि spacing system साठी rem वापरणे अनेक projects मध्ये सोयीचे ठरते.

Q30. What are different ways to define colors in CSS?

Answer:

CSS supports multiple color formats.

```
.box1 {
  color: red;
}

.box2 {
  color: #ff0000;
}

.box3 {
  color: rgb(255, 0, 0);
```

```
}

.box4 {
  color: rgba(255, 0, 0, 0.5);
}

.box5 {
  color: hsl(0, 100%, 50%);
}
```

Marathi Explanation:

CSS मध्ये color नावाने, HEX, RGB, RGBA आणि HSL formats ने define करता येतो.

- HEX – Hexadecimal color.
- RGB – Red, Green, Blue.
- RGBA – RGB सोबत alpha transparency.
- HSL – Hue, Saturation, Lightness.

8. CSS Responsive Design

Q31. What is Responsive Web Design?

Answer:

Responsive Web Design is an approach where a website automatically adapts its layout and content to different screen sizes and devices.

Marathi Explanation:

Website mobile, tablet आणि desktop या सर्व devices वर व्यवस्थित दिसली पाहिजे. यालाच Responsive Web Design म्हणतात.

Example:

```
.container {
  width: 100%;
  padding: 20px;
}
```

```
@media (min-width: 768px) {
  .container {
    width: 80%;
    margin: auto;
  }
}

@media (min-width: 1024px) {
  .container {
    width: 70%;
  }
}
```

Industry Use: E-commerce websites, portfolios, admin panels आणि web applications.

Q32. What is a Media Query?

Answer:

A media query is a CSS feature used to apply styles based on device or viewport conditions such as width, height, orientation, and user preferences.

Example:

```
/* Desktop */
.container {
  display: flex;
}

/* Mobile */
@media (max-width: 600px) {
  .container {
    flex-direction: column;
```

```
  }
}
```

Marathi Explanation:

Specific screen size किंवा device condition नुसार CSS बदलण्यासाठी media queries वापरतात.

Important Types:

```
@media (max-width: 600px) {
  /* Small screens */
}

@media (min-width: 768px) {
  /* Medium and larger screens */
}

@media (orientation: landscape) {
  /* Landscape orientation */
}
```

Q33. What is the difference between min-width and max-width?

Answer:

- min-width applies styles when the viewport width is at least the specified value.
- max-width applies styles when the viewport width is no greater than the specified value.

Example:

```
@media (min-width: 768px) {
  body {
    background: lightblue;
  }
}
```

```
@media (max-width: 600px) {
  body {
    background: lightgreen;
  }
}
```

Marathi Explanation:

- min-width: 768px – 768px किंवा त्यापेक्षा मोठ्या viewport साठी.
- max-width: 600px – 600px किंवा त्यापेक्षा लहान viewport साठी.

Q34. What is Mobile First Design?

Answer:

Mobile First Design is a responsive design approach where we first create the layout for small screens and then add styles for larger screens using min-width media queries.

Example:

```
/* Mobile First */
.container {
  display: flex;
  flex-direction: column;
}

/* Tablet and Desktop */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
  }
}
```

Marathi Explanation:

सुरुवातीला mobile साठी design तयार करायचे आणि नंतर tablet व desktop साठी extra styles add करायच्या.

Industry Use: Modern responsive frontend development.

9. CSS Pseudo-Classes and Pseudo-Elements

Q35. What is a Pseudo-class in CSS?

Answer:

A pseudo-class is used to select an element based on its state or position in the document.

Common Pseudo-classes:

Pseudo-class	Purpose
:hover	Mouse pointer over element
:focus	Element has focus
:active	Element is being activated
:first-child	First child
:last-child	Last child
:nth-child()	Selects by child position
:checked	Checked form control
:disabled	Disabled form control
:not()	Excludes matching elements

Example:

```
button:hover {
  background: blue;
  color: white;
}

input:focus {
  border-color: green;
}
```

```
li:first-child {
  color: red;
}

li:nth-child(2) {
  color: blue;
}
```

Marathi Explanation:

Element च्या state किंवा position नुसार style apply करण्यासाठी pseudo-classes वापरतात.

Q36. What is a Pseudo-element in CSS?

Answer:

A pseudo-element is used to style a specific part of an element or insert generated content.

Common Pseudo-elements:

- ::before
- ::after
- ::first-letter
- ::first-line
- ::selection
- ::placeholder

Example:

```
.heading::before {
  content: "★ ";
  color: gold;
}

.heading::after {
  content: " ✓";
```

```
  color: green;
}

p::first-letter {
  font-size: 30px;
}
```

Marathi Explanation:

Element च्या specific भागाला style देण्यासाठी किंवा CSS मधून content insert करण्यासाठी pseudo-elements वापरतात.

Q37. What is the difference between Pseudo-class and Pseudo-element?

Pseudo-class	**Pseudo-element**
Selects an element based on state or position	Styles a specific part of an element or generated content
Uses : syntax	Uses :: syntax
:hover	::before
:focus	::after
:first-child	::first-letter

Example:

```
button:hover {
  background: black;
}

p::first-letter {
  color: red;
}
```

10. CSS Advanced Questions

Q38. What is CSS Inheritance?

Answer:

CSS inheritance is a behavior where some CSS properties are passed from a parent element to its child elements.

Example:

```
.parent {
  color: blue;
  font-family: Arial;
}
<div class="parent">
  <p>Hello</p>
  <span>Welcome</span>
</div>
```

Both the paragraph and span inherit the text color and font family.

Marathi Explanation:

Parent element ला दिलेल्या काही CSS properties child elements ला automatically मिळतात.

Important: color, font-family आणि line-height सामान्यतः inherit होतात. margin, padding आणि border सामान्यतः inherit होत नाहीत.

Q39. What is CSS Overflow?

Answer:

The overflow property controls what happens when content is larger than an element's available space.

Values:

Value	Meaning
visible	Content can extend outside
hidden	Extra content is clipped
scroll	Scrollbars are provided
auto	Scrollbars appear when needed

clip Clips content without creating a scroll container

Example:

```
.box {
  width: 200px;
  height: 100px;
  overflow: auto;
}
```

Marathi Explanation:

Container मध्ये content जास्त झाले तर ते कसे दाखवायचे हे overflow ठरवते.

Industry Use: Scrollable cards, chat windows, tables आणि modal content.

Q40. What is CSS Transition?

Answer:

CSS transition allows property changes to happen gradually over a specified duration.

Example:

```
.button {
  background: blue;
  color: white;
  transition: background 0.3s ease;
}

.button:hover {
  background: red;
}
```

Marathi Explanation:

एखाद्या element ची style अचानक बदलण्याऐवजी हळूहळू बदलण्यासाठी transition वापरतात.

Important Properties:

- transition-property

- transition-duration
- transition-timing-function
- transition-delay

Q41. What is CSS Animation?

Answer:

CSS animations allow elements to change styles over time using keyframes.

Example:

```
.box {
  width: 100px;
  height: 100px;
  background: blue;

  animation: move 2s infinite alternate;
}

@keyframes move {
  from {
    transform: translateX(0);
  }

  to {
    transform: translateX(200px);
  }
}
```

Marathi Explanation:

CSS Animation वापरून JavaScript शिवाय elements ला movement, rotation, fade आणि इतर effects देता येतात.

Difference between Transition and Animation:

Transition	Animation
Usually triggered by a state change	Can start automatically
Requires start and end states	Supports multiple keyframes
Commonly used for hover effects	Commonly used for loaders and complex effects

Q42. What is the difference between visibility: hidden, opacity: 0, and display: none?

Answer:

Property	Visible?	Layout Space?	Can Receive Pointer Events?
display: none	No	No	No
visibility: hidden	No	Yes	No
opacity: 0	No	Yes	Yes, unless pointer events are disabled

Example:

```
.box1 {
  display: none;
}

.box2 {
  visibility: hidden;
}

.box3 {
  opacity: 0;
}
```

Marathi Explanation:

तीनही methods element hide करतात; पण layout space आणि interaction वर त्यांचा परिणाम वेगळा असतो.

Q43. What is CSS Variable?

Answer:

CSS variables, also called custom properties, allow us to store reusable values and use them throughout CSS.

Example:

```
:root {
  --primary-color: #2563eb;
  --text-color: #222;
  --spacing: 20px;
}

.button {
  background: var(--primary-color);
  color: white;
  padding: var(--spacing);
}

.heading {
  color: var(--text-color);
}
```

Marathi Explanation:

एकच color किंवा spacing अनेक ठिकाणी वापरायची असेल तर CSS variables वापरतात. त्यामुळे project मध्ये design changes करणे सोपे होते.

Industry Use: Design systems, themes, dark mode आणि reusable components.

Q44. What is the difference between min-width, max-width, and width?

Answer:

- width sets the preferred width.
- min-width defines the minimum allowed width.
- max-width defines the maximum allowed width.

Example:

```
.container {
  width: 100%;
  min-width: 300px;
  max-width: 1200px;
  margin: auto;
}
```

Marathi Explanation:

Element ची width कशी control करायची हे या properties ठरवतात. Responsive container तयार करताना max-width खूप उपयोगी आहे.

Q45. What is the difference between absolute, fixed, and sticky positioning in real projects?

Answer:

- absolute: Used for badges, icons, and overlays inside components.
- fixed: Used for floating buttons and fixed-position elements.
- sticky: Used for sticky navigation bars and table headers.

Practical Example:

```
.product-card {
  position: relative;
}

.discount-badge {
  position: absolute;
  top: 10px;
  right: 10px;
}

.chat-button {
```

```
  position: fixed;
  bottom: 20px;
  right: 20px;
}

.navbar {
  position: sticky;
  top: 0;
}
```

Marathi Explanation:

Real projects मध्ये product card वर discount badge, screen वर floating chat button आणि scroll करताना वर राहणारा navbar तयार करण्यासाठी या properties वापरतात.

11. CSS Practical Coding Interview Questions

Q46. Create a responsive card using CSS.

HTML:

```
<div class="card">
  <img src="product.jpg" alt="Product">

  <div class="card-content">
    <h2>Product Name</h2>
    <p>Product description goes here.</p>
    <button>Buy Now</button>
  </div>
</div>
```

CSS:

```
* {
  box-sizing: border-box;
}
```

```
.card {
  width: 100%;
  max-width: 350px;
  border: 1px solid #ddd;
  border-radius: 12px;
  overflow: hidden;
}

.card img {
  width: 100%;
  height: 200px;
  object-fit: cover;
}

.card-content {
  padding: 20px;
}

.card button {
  width: 100%;
  padding: 12px;
  background: blue;
  color: white;
  border: none;
  cursor: pointer;
}
```

Interview Explanation:

This card uses max-width for responsive sizing, box-sizing for predictable dimensions, object-fit for image cropping, and border-radius for rounded corners.

Q47. Create a responsive two-column layout.

```
<div class="container">
  <div class="sidebar">Sidebar</div>
  <div class="content">Main Content</div>
</div>
.container {
  display: grid;
  grid-template-columns: 250px 1fr;
  gap: 20px;
}

.sidebar {
  background: lightgray;
  padding: 20px;
}

.content {
  background: lightblue;
  padding: 20px;
}

@media (max-width: 768px) {
  .container {
    grid-template-columns: 1fr;
  }
}
```

Interview Explanation:

CSS Grid creates two columns on larger screens. On screens up to 768px, the layout changes to a single column.

Q48. Create a button with hover animation.

```
<button class="btn">Get Started</button>

.btn {
  padding: 12px 25px;
  background: #2563eb;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;

  transition: all 0.3s ease;
}

.btn:hover {
  background: #1d4ed8;
  transform: translateY(-3px);
  box-shadow: 0 5px 15px rgba(0, 0, 0, 0.2);
}
```

Interview Explanation:

The transition property makes the hover effect smooth, while transform moves the button slightly upward.

12. CSS Output-Based Tricky Questions

हे questions interviews मध्ये तुझी actual CSS understanding तपासण्यासाठी विचारले जाऊ शकतात.

Q49. What will be the output?

```
.box {
```

```
  width: 200px;
  padding: 20px;
  border: 5px solid black;
}
```

Question: What is the total width?

Answer: 250px.

Calculation:

200 + 20 + 20 + 5 + 5 = 250px.

Q50. What will be the output?

```
p {
  color: red;
}

.text {
  color: blue;
}
<p class="text">Hello World</p>
```

Output: Blue

Reason: Class selector has higher specificity than the element selector.

Q51. What happens here?

```
.box {
  display: none;
}
```

Answer: The element is not displayed and does not occupy layout space.

Q52. What happens here?

```
.parent {
  display: flex;
  justify-content: center;
```

```
  align-items: center;
}
```

Answer: The child items are centered along the main and cross axes respectively.

Important: For the default flex-direction: row, this means horizontal and vertical centering.

Q53. What is the output?

```
.container {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
}
```

Answer:

- justify-content: center – Centers items vertically.
- align-items: center – Centers items horizontally.

Reason: The main axis changes when flex-direction changes to column.

13. CSS Important Interview Questions – Quick Revision

No.	Interview Question	Short Answer
1	What is CSS?	Styling language for HTML
2	What is CSS specificity?	Rule priority system
3	What is Box Model?	Content, padding, border, margin
4	What is Flexbox?	One-dimensional layout
5	What is Grid?	Two-dimensional layout
6	What is z-index?	Controls stacking order
7	What is Media Query?	Applies conditional CSS
8	What is Pseudo-class?	Selects element state or position
9	What is Pseudo-element?	Styles a part of an element

10	What is Inheritance?	Passing some properties to children
11	What is Overflow?	Controls extra content
12	What is Transition?	Smooth property change
13	What is Animation?	Keyframe-based animation
14	What is !important?	Important cascade declaration
15	What is box-sizing?	Controls box size calculation
16	What is rem?	Relative to root font size
17	What is em?	Relative to relevant font size
18	What is position: sticky?	Scroll-based sticking behavior
19	What is object-fit?	Controls replaced content fitting
20	What is CSS Variable?	Reusable custom property

Tailwind CSS Interview Questions and Answers – Beginner to Advanced

Part 3: Tailwind CSS

Beginner to Advanced

1. Tailwind CSS Fundamentals

Q1. What is Tailwind CSS?

Answer (Simple English):

Tailwind CSS is a utility-first CSS framework used to build modern and responsive user interfaces. It provides predefined utility classes for styling HTML elements directly without writing much custom CSS.

Marathi Explanation:

Tailwind CSS हे CSS framework आहे. यामध्ये प्रत्येक CSS property साठी predefined class असते. त्यामुळे वेगळी CSS लिहिण्याऐवजी HTML किंवा React JSX मध्ये classes वापरून design तयार करता येते.

Example:

Normal CSS:

```
<h1 class="heading">Hello World</h1>

.heading {
  color: blue;
  font-size: 30px;
  font-weight: bold;
  text-align: center;
}
```

Tailwind CSS:

```
<h1 class="text-3xl font-bold text-blue-600 text-center">
  Hello World
</h1>
```

Industry Use: React applications, admin dashboards, e-commerce websites, portfolios आणि SaaS applications.

Q2. What is Utility-First CSS?

Answer:

Utility-first CSS is an approach where small, single-purpose classes are used to style elements directly.

Example:

```
<div class="bg-blue-500 p-5 rounded-lg shadow-md">
  Welcome to Tailwind CSS
</div>
```

Explanation:

- bg-blue-500 – Background color.
- p-5 – Padding.
- rounded-lg – Border radius.
- shadow-md – Medium shadow.

Marathi Explanation:

Utility-first म्हणजे प्रत्येक CSS property साठी छोटी class वापरणे. अनेक utility classes एकत्र वापरून पूर्ण design तयार करता येते.

Q3. What are the advantages of Tailwind CSS?

Answer:

1. Faster UI development.
2. Responsive design support.
3. Less need for custom CSS.
4. Consistent design system.
5. Easy customization.
6. Reusable styling patterns.
7. Small production CSS through automatic detection and optimization of used classes.

Marathi Explanation:

Tailwind मुळे development जलद होते. प्रत्येक component साठी वेगळी CSS file लिहिण्याची गरज कमी होते. तसेच colors, spacing आणि responsive design मध्ये consistency राखता येते.

Industry Use: मोठ्या React projects मध्ये reusable UI components तयार करण्यासाठी उपयोगी.

Q4. What is the difference between Tailwind CSS and Bootstrap?

Answer:

Tailwind CSS is a utility-first framework, while Bootstrap provides predefined components and a grid system.

Tailwind CSS	Bootstrap
Utility-first approach	Component-based approach
More design flexibility	Predefined component styles
Styles written using utility classes	Often uses component classes
Custom UI design is flexible	Default look can be recognizable
Commonly customized through theme configuration	Customized through Sass variables and other methods

Example:

Bootstrap:

```
<button class="btn btn-primary">
  Submit
</button>
```

Tailwind:

```
<button class="bg-blue-600 text-white px-5 py-2 rounded-lg hover:bg-blue-700">
  Submit
</button>
```

Marathi Explanation:

Bootstrap मध्ये ready-made components मिळतात. Tailwind मध्ये छोट्या utility classes वापरून स्वतःच्या design प्रमाणे UI तयार करता येतो.

Tricky Follow-up: Which one is better?

Answer: It depends on the project. Tailwind is useful for highly customized interfaces, while Bootstrap can be convenient when predefined components and faster standard layouts are preferred.

Q5. How do you install Tailwind CSS?

Answer:

Tailwind CSS installation depends on the project setup and Tailwind version.

For a modern Vite project, Tailwind CSS v4 can be installed using:

npm install tailwindcss @tailwindcss/vite

Configure the Vite plugin in vite.config.js:

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [
    react(),
    tailwindcss()
  ]
});
```

In your main CSS file:

```
@import "tailwindcss";
```

Marathi Explanation:

Tailwind install केल्यानंतर Vite configuration मध्ये plugin जोडायचा आणि CSS file मध्ये Tailwind import करायचे.

Important Interview Point: Tailwind CSS v3 आणि v4 ची installation व configuration पद्धत वेगळी आहे. त्यामुळे project ची version तपासणे आवश्यक आहे.

2. Tailwind CSS Utility Classes

Q6. What are Utility Classes in Tailwind CSS?

Answer:

Utility classes are predefined classes that perform specific styling tasks such as setting colors, spacing, fonts, borders, and layouts.

Example:

```
<div class="bg-gray-100 p-6 rounded-xl shadow-lg">
  <h2 class="text-2xl font-bold text-gray-800">
    Product Name
  </h2>

  <p class="mt-2 text-gray-600">
    Product Description
  </p>
</div>
```

Marathi Explanation:

प्रत्येक class एक specific काम करते. अनेक classes एकत्र वापरून complete design तयार करता येते.

Important Utility Classes:

Category	**Tailwind Classes**
Text Color	text-red-500
Background	bg-blue-500
Font Size	text-xl
Font Weight	font-bold
Padding	p-4, px-4, py-2
Margin	m-4, mt-5, mx-auto
Width	w-full, w-1/2
Height	h-screen, h-20
Border	border, border-gray-300

Rounded	rounded-lg, rounded-full
Shadow	shadow-md, shadow-xl

Q7. Explain Padding and Margin in Tailwind CSS.

Answer:

Padding creates space inside an element, while margin creates space outside an element.

Tailwind Padding Classes:

```
<div class="p-4">All sides padding</div>

<div class="px-4">Horizontal padding</div>

<div class="py-4">Vertical padding</div>

<div class="pt-4">Top padding</div>

<div class="pb-4">Bottom padding</div>
```

Tailwind Margin Classes:

```
<div class="m-4">All sides margin</div>

<div class="mx-4">Horizontal margin</div>

<div class="my-4">Vertical margin</div>

<div class="mt-4">Top margin</div>

<div class="mb-4">Bottom margin</div>
```

Marathi Explanation:

- p – सर्व बाजूंनी padding.

- px – Left आणि right padding.
- py – Top आणि bottom padding.
- m – सर्व बाजूंनी margin.
- mx – Horizontal margin.
- my – Vertical margin.

Example:

```
<div class="mx-auto max-w-4xl p-6">
  Content
</div>
```

येथे mx-auto element ला horizontal center करण्यास मदत करते, जर width constraint असेल.

Q8. How do you set Width and Height in Tailwind CSS?

Answer:

Tailwind provides utility classes for setting width and height.

Example:

```
<div class="w-full h-40 bg-blue-500">
  Box
</div>
```

Important Classes:

Class	Meaning
w-full	Width 100%
w-screen	Width 100vw
w-fit	Fit-content width
w-auto	Automatic width
h-full	Height 100%
h-screen	Height 100vh
h-auto	Automatic height

min-h-screen	Minimum height 100vh
max-w-xl	Maximum width preset

Marathi Explanation:

Element ची width आणि height control करण्यासाठी या classes वापरतात.

Industry Example:

```
<div className="min-h-screen w-full">
  <main className="mx-auto max-w-7xl px-4">
    Dashboard
  </main>
</div>
```

Q9. How do you apply Text Styling in Tailwind CSS?

Answer:

Tailwind provides utility classes for font size, weight, color, alignment, line height, and letter spacing.

Example:

```
<h1 class="text-4xl font-bold text-gray-900 text-center">
  Welcome
</h1>

<p class="text-base leading-relaxed text-gray-600">
  Learn Tailwind CSS.
</p>
```

Important Classes:

Purpose	Classes
Font Size	text-sm, text-lg, text-2xl
Font Weight	font-normal, font-medium, font-bold
Text Color	text-white, text-gray-700

Alignment	text-left, text-center, text-right
Line Height	leading-none, leading-relaxed
Letter Spacing	tracking-wide, tracking-tight
Text Transform	uppercase, lowercase, capitalize
Decoration	underline, line-through, no-underline

Q10. What are Border and Border Radius utilities?

Answer:

Tailwind provides classes for border width, border color, border style, and border radius.

Example:

```
<div class="border border-gray-300 rounded-xl p-5">
  Product Card
</div>
```

Important Classes:

Class	Purpose
border	1px border
border-2	2px border
border-red-500	Border color
border-dashed	Dashed border
rounded-sm	Small radius
rounded-lg	Large radius
rounded-xl	Extra-large radius
rounded-full	Fully rounded corners

3. Tailwind CSS Flexbox and Grid

Q11. How do you use Flexbox in Tailwind CSS?

Answer:

Tailwind provides utility classes to create and control flex layouts.

Example:

```
<div className="flex items-center justify-between gap-4">
  <h2>Dashboard</h2>
  <button>Logout</button>
</div>
```

Important Classes:

CSS Property	Tailwind Class
display: flex	flex
flex-direction: row	flex-row
flex-direction: column	flex-col
justify-content: center	justify-center
justify-content: between	justify-between
align-items: center	items-center
flex-wrap: wrap	flex-wrap
gap: 16px	gap-4

Marathi Explanation:

Tailwind मध्ये flex, items-center, justify-between वापरून CSS न लिहिता Flexbox layout तयार करता येतो.

Industry Use: Navbar, dashboard header, product card आणि buttons.

Q12. How do you create a Grid layout in Tailwind CSS?

Answer:

Tailwind provides grid utility classes for creating responsive row and column layouts.

Example:

```
<div className="grid grid-cols-3 gap-4">
  <div className="bg-blue-200 p-5">Box 1</div>
  <div className="bg-blue-200 p-5">Box 2</div>
  <div className="bg-blue-200 p-5">Box 3</div>
```

Important Classes:

Class	Meaning
grid	Enables CSS Grid
grid-cols-2	Two columns
grid-cols-3	Three columns
grid-cols-4	Four columns
grid-rows-2	Two rows
col-span-2	Spans two columns
row-span-2	Spans two rows
gap-4	Gap between items

Responsive Grid Example:

```
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6">
  {/* Product Cards */}
</div>
```

Marathi Explanation:

Mobile वर एक column, small screens वर दोन आणि large screens वर चार columns दिसतील.

4. Responsive Design in Tailwind CSS

Q13. How does Responsive Design work in Tailwind CSS?

Answer:

Tailwind uses responsive prefixes to apply styles at specific minimum viewport widths.

Default Breakpoints in Tailwind CSS v4:

Prefix	Minimum Width
sm:	640px
md:	768px

lg: 1024px

xl: 1280px

2xl: 1536px

Example:

```
<div className="
  text-sm
  sm:text-base
  md:text-lg
  lg:text-2xl
">
  Responsive Text
</div>
```

Marathi Explanation:

Tailwind मध्ये sm:, md:, lg: यांसारखे prefixes वापरून वेगवेगळ्या screen sizes साठी style लागू करता येते.

Important Interview Point: Tailwind responsive design mobile-first आहे. Prefix नसलेली class सर्व sizes साठी default असते. md: class 768px आणि त्यापेक्षा मोठ्या viewport साठी लागू होते.

Q14. What is Mobile-First Design in Tailwind CSS?

Answer:

Mobile-first design means writing the default styles for smaller screens first and adding responsive styles for larger screens.

Example:

```
<div className="
  flex flex-col
  md:flex-row
  gap-4
">
```

Sidebar

Main Content

Marathi Explanation:

- Mobile – flex-col म्हणजे vertical layout.
- Tablet/Desktop – md:flex-row म्हणजे horizontal layout.

Industry Use: Responsive dashboards, portfolios आणि e-commerce websites.

Q15. What is the difference between sm:, md:, and lg:?

Answer:

These are responsive prefixes used to apply styles at different minimum viewport widths.

<div className="

bg-red-500

sm:bg-blue-500

md:bg-green-500

lg:bg-yellow-500

">

Responsive Box

Output:

- Below 640px – Red.
- 640px and above – Blue.
- 768px and above – Green.
- 1024px and above – Yellow.

Marathi Explanation:

मोबाइलवर red, small breakpoint नंतर blue, medium breakpoint नंतर green आणि large breakpoint नंतर yellow background लागू होईल.

5. Tailwind CSS States and Variants

Q16. What are Hover, Focus, and Active variants?

Answer:

Tailwind provides state variants to apply styles when an element is hovered, focused, or active.

Example:

```
<button className="
  bg-blue-600
  hover:bg-blue-800
  focus:ring-2
  focus:ring-blue-300
  active:scale-95
  text-white
  px-5 py-2
  rounded-lg
">
  Submit
</button>
```

Important Variants:

Variant	Purpose
hover:	Mouse hover
focus:	Element focused
active:	Element activated
disabled:	Disabled element
checked:	Checked form control
first:	First child
last:	Last child
odd:	Odd child

even: Even child

group-hover: Parent group hovered

peer-checked: Peer element checked

Marathi Explanation:

User च्या interaction नुसार style बदलण्यासाठी variants वापरतात.

Q17. What is Group Hover in Tailwind CSS?

Answer:

The group and group-hover: utilities allow child elements to change their styles when the parent element is hovered.

Example:

```
<div className="group p-5 rounded-xl bg-gray-100">
  <h2 className="text-gray-800 group-hover:text-blue-600">
    Product Name
  </h2>

  <p className="opacity-60 group-hover:opacity-100">
    View Product Details
  </p>
</div>
```

Marathi Explanation:

Parent div वर hover केल्यावर त्याच्या child elements ची style बदलते.

Industry Use: Product cards, navigation menus आणि interactive cards.

Q18. What is Peer in Tailwind CSS?

Answer:

The peer utility allows an element to style a following sibling based on the state of another sibling.

Example:

```
<label>
```

```
  <input type="checkbox" className="peer" />

  <span className="peer-checked:text-green-600">
    Accept Terms
  </span>
</label>
```

Marathi Explanation:

एका sibling element च्या state वर दुसऱ्या following sibling ची style बदलण्यासाठी peer वापरतात.

Important: Peer-based styling सामान्य sibling relationship वर आधारित असते; पुढील sibling ला style करता येते, आधीच्या sibling ला नाही.

Q19. What is Dark Mode in Tailwind CSS?

Answer:

Dark mode allows an application to use a dark color scheme based on the user's system preference or an application-controlled theme.

Example:

```
<div className="bg-white text-gray-900 dark:bg-gray-900 dark:text-white">
  Dark Mode Content
</div>
```

Marathi Explanation:

dark: prefix वापरून dark theme साठी वेगळे styles देता येतात.

Industry Use: Admin dashboards, developer tools आणि modern SaaS applications.

Important Version Point: Tailwind v4 मध्ये default dark variant system preference (prefers-color-scheme) वापरतो. Manual toggle साठी custom dark variant configure करावा लागू शकतो.

6. Tailwind CSS Customization

Q20. How do you add custom colors in Tailwind CSS?

Answer:

Tailwind CSS v4 supports theme customization using CSS theme variables.

Example:

```
@import "tailwindcss";


@theme {
  --color-brand: #2563eb;
  --color-primary: #7c3aed;
}
```

Now use:

```
<div className="bg-brand text-white">
  Brand Section
</div>


<button className="bg-primary text-white">
  Submit
</button>
```

Marathi Explanation:

Project मध्ये स्वतःचे brand colors वापरायचे असतील तर theme variables define करता येतात.

Industry Use: Company design systems आणि consistent branding.

Q21. What is the difference between Tailwind CSS v3 and v4?

Answer:

Tailwind CSS v4 introduced a CSS-first configuration approach, a new installation workflow, and improvements to its build engine.

Tailwind v3	**Tailwind v4**
Commonly uses tailwind.config.js	CSS-first theme configuration
Uses @tailwind directives	Uses @import "tailwindcss"
Content paths commonly configured manually	Automatic source detection

Common Vite setup uses PostCSS plugin	Dedicated Vite plugin available
Theme often configured in JS	Theme can be configured with @theme

Tailwind v3 CSS:

@tailwind base;

@tailwind components;

@tailwind utilities;

Tailwind v4 CSS:

@import "tailwindcss";

Marathi Explanation:

v3 मध्ये configuration साठी JavaScript config file महत्त्वाची होती. v4 मध्ये CSS file मधून theme आणि configuration करणे सोपे झाले आहे.

Important Interview Point: दोन्ही versions चे syntax आणि configuration एकसारखे समजू नये.

Q22. What is the Arbitrary Value in Tailwind CSS?

Answer:

Arbitrary values allow developers to use custom values directly inside Tailwind utility classes.

Example:

```
<div className="w-[350px] h-[200px] bg-[#123456]">
  Custom Box
</div>
```

Other Examples:

```
<div className="top-[120px]"></div>

<p className="text-[18px]"></p>

<div className="grid-cols-[200px_1fr_100px]"></div>
```

Marathi Explanation:

Tailwind मध्ये predefined value उपलब्ध नसेल तर square brackets मध्ये custom value देता येते.

Industry Use: Unique layouts, custom spacing आणि exact design requirements.

Q23. What is the @apply directive in Tailwind CSS?

Answer:

The @apply directive allows developers to combine existing Tailwind utility classes inside a CSS rule.

Example:

```
@import "tailwindcss";


.btn-primary {
  @apply bg-blue-600 text-white px-5 py-2 rounded-lg;
}
```

Use it:

```
<button class="btn-primary">
  Submit
</button>
```

Marathi Explanation:

एकाच utility classes चा group पुन्हा पुन्हा वापरायचा असेल तर custom CSS class मध्ये @apply वापरू शकतो.

Industry Note: प्रत्येक छोट्या component साठी @apply वापरण्याऐवजी React मध्ये reusable components तयार करणे अनेकदा चांगले असते.

Q24. What is the purpose of container class in Tailwind CSS?

Answer:

The container utility sets an element's width to the current responsive breakpoint's maximum width.

Example:

```
<div className="container mx-auto px-4">
```

<h1>Welcome</h1>

Marathi Explanation:

Container वापरल्यामुळे content ला responsive maximum width मिळते. mx-auto मुळे तो horizontal center होतो.

Important: container class स्वतःहून प्रत्येक breakpoint वर centered किंवा padded असेलच असे नाही. त्यासाठी गरजेनुसार mx-auto आणि px-* वापरतात.

7. Tailwind CSS Advanced Questions

Q25. How does Tailwind CSS generate CSS classes?

Answer:

Tailwind scans source files for class names and generates CSS for the utilities it detects.

Marathi Explanation:

Project मधील HTML, JSX आणि इतर supported source files मध्ये कोणत्या Tailwind classes वापरल्या आहेत ते Tailwind शोधते आणि त्यासाठी CSS generate करते.

Example:

Hello

या classes साठी आवश्यक CSS production build मध्ये तयार होते.

Important: Dynamic class strings Tailwind ला नेहमी detect होतीलच असे नाही.

Q26. Why should we avoid dynamically constructing Tailwind class names?

Answer:

Tailwind detects complete class names in source files. Dynamically building class names may prevent the scanner from detecting the required utilities.

Incorrect Example:

const color = "red";

```
<div className={`bg-${color}-500`}>
  Hello
</div>
```

Correct Example:

```
const colors = {
  red: "bg-red-500",
  blue: "bg-blue-500",
  green: "bg-green-500"
};

<div className={colors.red}>
  Hello
</div>
```

Marathi Explanation:

Tailwind ला पूर्ण class name source code मध्ये दिसणे महत्त्वाचे आहे. म्हणून पूर्ण class strings आधी define करून वापरणे सुरक्षित आहे.

Industry Use: Dynamic buttons, status badges, alerts आणि dashboard themes.

Q27. How do you conditionally apply Tailwind classes in React?

Answer:

We can use JavaScript conditional expressions to apply Tailwind classes based on component state or props.

Example:

```
function Button({ isActive }) {
  return (
    <button
      className={
        isActive
          ? "bg-blue-600 text-white"
```

```
            : "bg-gray-200 text-gray-800"
        }
      >
        Click Me
      </button>
    );
}
```

Marathi Explanation:

React मधील condition नुसार वेगवेगळ्या Tailwind classes apply करता येतात.

Another Example:

```
<div className={`p-4 rounded-lg ${isError ? "bg-red-100 text-red-700" : "bg-green-100 text-green-700"}`}>
  Status Message
</div>
```

Q28. What is the difference between hidden and invisible in Tailwind CSS?

Answer:

- hidden applies display: none.
- invisible applies visibility: hidden.

Example:

```
<div className="hidden">
  Hidden Element
</div>

<div className="invisible">
  Invisible Element
</div>
```

Marathi Explanation:

- hidden – Element दिसत नाही आणि त्याची layout space निघून जाते.

- invisible – Element दिसत नाही, पण त्याची जागा कायम राहते.

Q29. How do you add custom CSS in a Tailwind project?

Answer:

Custom CSS can be added in a CSS file alongside Tailwind utilities when a design requires styles that are not conveniently expressed through utilities.

Example:

```
@import "tailwindcss";


.custom-heading {
  text-shadow: 2px 2px 5px gray;
}
```

React:

```
<h1 className="custom-heading text-3xl font-bold">
  Welcome
</h1>
```

Marathi Explanation:

Tailwind असूनही काही special effects किंवा custom CSS आवश्यक असल्यास वेगळी class लिहू शकतो.

Q30. What is the difference between space-x-* and gap-*?

Answer:

gap-* sets spacing between rows and columns in layout containers such as Flexbox and Grid. space-x-* adds horizontal spacing between direct children using margins.

Example:

```
<div className="flex gap-4">
  <button>One</button>
  <button>Two</button>
  <button>Three</button>
</div>
```

Using space-x:

```
<div className="flex space-x-4">
  <button>One</button>
  <button>Two</button>
  <button>Three</button>
</div>
```

Marathi Explanation:

- gap-4 – Flex/Grid items मधील spacing.
- space-x-4 – Direct children मध्ये horizontal spacing.

Industry Best Practice: Flexbox आणि Grid layouts साठी gap बहुतेक वेळा अधिक सोयीचा असतो.

8. Tailwind CSS Practical React Interview Questions

Q31. Create a Responsive Navbar using Tailwind CSS.

Answer:

```
function Navbar() {
  return (
    <nav className="bg-white shadow-md">
      <div className="max-w-7xl mx-auto px-4 py-4 flex items-center justify-between">
        <h1 className="text-2xl font-bold text-blue-600">
          MyBrand
        </h1>
        <div className="hidden md:flex items-center gap-6">
          <a href="/" className="hover:text-blue-600">Home</a>
          <a href="/about" className="hover:text-blue-600">About</a>
          <a href="/services" className="hover:text-blue-600">Services</a>
          <a href="/contact" className="hover:text-blue-600">Contact</a>
        </div>
```

```
          <button className="bg-blue-600 text-white px-4 py-2 rounded-lg hover:bg-blue-700">
            Login
          </button>

        </div>
      </nav>
    );
}

export default Navbar;
```

Interview Explanation:

- max-w-7xl – Maximum container width.
- mx-auto – Centers the container.
- flex – Creates flex layout.
- justify-between – Places items at opposite ends.
- hidden md:flex – Navigation links hidden on small screens and displayed from the medium breakpoint.

Note: हा basic responsive layout आहे. Mobile menu button प्रत्यक्ष काम करण्यासाठी React state आणि click handler जोडावा लागेल.

Q32. Create a Responsive Product Card.

```
function ProductCard() {
  return (
    <div className="max-w-sm bg-white rounded-xl shadow-lg overflow-hidden border">

      <img
        src="/product.jpg"
        alt="Product"
```

```
        className="w-full h-52 object-cover"
      />

      <div className="p-5">

        <h2 className="text-xl font-bold text-gray-800">
          Wireless Headphones
        </h2>

        <p className="text-gray-500 mt-2">
          High-quality wireless headphones.
        </p>

        <div className="flex justify-between items-center mt-4">

          <span className="text-2xl font-bold text-green-600">
            ₹1,999
          </span>

          <button className="bg-blue-600 text-white px-4 py-2 rounded-lg hover:bg-
blue-700 transition">
            Buy Now
          </button>

        </div>

      </div>
    </div>
```

```
  );
}

export default ProductCard;
```

Interview Explanation:

This component uses Tailwind utility classes to style a product card with an image, title, description, price, and action button.

Q33. How do you create a responsive Dashboard Layout?

```
function Dashboard() {
  return (
    <div className="min-h-screen bg-gray-100">

      <div className="grid grid-cols-1 md:grid-cols-[250px_1fr]">

        <aside className="bg-gray-900 text-white p-5">
          <h2 className="text-xl font-bold">
            Dashboard
          </h2>

          <nav className="mt-6 flex flex-col gap-4">
            <a href="#">Overview</a>
            <a href="#">Users</a>
            <a href="#">Settings</a>
          </nav>
        </aside>

        <main className="p-5">
          <h1 className="text-2xl font-bold">
```

```
            Welcome Back
          </h1>

          <div className="grid grid-cols-1 sm:grid-cols-2 xl:grid-cols-4 gap-5 mt-6">

            <div className="bg-white p-5 rounded-xl shadow">
              Total Users: 120
            </div>

            <div className="bg-white p-5 rounded-xl shadow">
              Total Sales: ₹50,000
            </div>

            <div className="bg-white p-5 rounded-xl shadow">
              Orders: 250
            </div>

            <div className="bg-white p-5 rounded-xl shadow">
              Revenue: ₹75,000
            </div>

          </div>
        </main>

      </div>
    </div>
  );
}
```

export default Dashboard;

Interview Explanation:

This layout uses CSS Grid through Tailwind utilities. It creates a single-column layout on small screens and a sidebar-plus-content layout from the medium breakpoint.

9. Tailwind CSS Tricky Interview Questions

Question	Correct Answer
Is Tailwind CSS a CSS framework?	Yes. It is a utility-first CSS framework.
Does Tailwind replace CSS?	No. It generates CSS and supports custom CSS too.
Is Tailwind mobile-first?	Yes.
What does p-4 mean?	Padding on all sides using spacing scale value 4.
What does mx-auto do?	Applies auto horizontal margins.
What does w-full mean?	Width: 100%.
What does h-screen mean?	Height: 100vh.
What does flex do?	Creates a flex container.
What does grid do?	Creates a grid container.
What does rounded-full do?	Applies fully rounded corners.
What does shadow-lg do?	Applies a large shadow.
What does hover: mean?	Applies styles on hover.
What does md: mean?	Applies styles at the medium breakpoint and above.
What does dark: mean?	Applies dark-mode styles.
What does hidden mean?	Applies display: none.
What does invisible mean?	Applies visibility: hidden.
What does @apply do?	Combines utility classes inside CSS.

What does @theme do?	Defines theme variables in Tailwind v4.
What are arbitrary values?	Custom values written inside square brackets.
Why avoid dynamic class construction?	Full class names may not be detected by the scanner.

10. Tailwind CSS Output-Based Questions

Q34. What will be the output?

```
<div className="text-red-500 text-blue-500">
  Hello World
</div>
```

Answer: Do not assume that the last class written in the HTML string wins.

Explanation: Tailwind CSS utilities are generated into a stylesheet. The winning declaration depends on the CSS cascade and generated rule order, not simply the order of class names in the attribute.

Q35. What does this class do?

```
<div className="hidden md:block">
  Hello
</div>
```

Answer:

- Below 768px – Hidden.
- 768px and above – Displayed as block.

Q36. What does this class do?

```
<div className="flex flex-col md:flex-row">
  Content
</div>
```

Answer:

- Mobile – Vertical layout.
- Medium screens and above – Horizontal layout.

Q37. What is the output?

  Content

Answer:

The element takes full available width up to the max-w-md maximum and is centered horizontally when space is available.

Part 4: JavaScript Interview Questions & Answers (Beginner to Advanced)

1. JavaScript Fundamentals

Q1. What is JavaScript?

Interview Answer:

JavaScript is a high-level, interpreted programming language. It is mainly used to make web pages interactive and dynamic. JavaScript can run in browsers as well as on servers using Node.js.

मराठी Explanation:

JavaScript ही programming language आहे. HTML वेबपेजची structure तयार करते, CSS design करते आणि JavaScript त्या webpage ला dynamic आणि interactive बनवते.

Example:

```
<button onclick="showMessage()">Click Me</button>


<script>
 function showMessage() {
  alert("Hello JavaScript!");
 }
</script>
```

Industry Use:

- Button click events.
- Form validation.
- API calls.
- Dynamic UI updates.
- Backend development using Node.js.

Q2. What are the features of JavaScript?

Interview Answer:

JavaScript has several important features:

1. Lightweight programming language.

2. Dynamically typed.
3. Supports object-oriented programming.
4. Supports functional programming.
5. Single-threaded execution model.
6. Asynchronous programming support.
7. Runs in browsers and servers.

मराठी Explanation:

JavaScript मध्ये variable चा data type आधी declare करावा लागत नाही. ती runtime वर type ठरवते. तसेच asynchronous operations वापरून API calls सारखी कामे हाताळता येतात.

Q3. Is JavaScript compiled or interpreted?

Interview Answer:

JavaScript is often described as an interpreted language, but modern JavaScript engines use Just-In-Time (JIT) compilation. They compile frequently executed code into machine code to improve performance.

मराठी Explanation:

JavaScript engine code execute करताना गरजेनुसार compilation सुद्धा करते. Chrome मध्ये V8 engine JavaScript execute करते.

Q4. What is the difference between JavaScript and Java?

JavaScript	**Java**
Dynamically typed	Statically typed
Mainly used for web development	Used for enterprise, Android and backend systems
Runs in browsers and Node.js	Runs on JVM
Uses prototype-based inheritance	Uses class-based inheritance
Commonly uses JIT engines	Compiles to bytecode and runs on JVM

Interview Tip: Java आणि JavaScript ही वेगवेगळी programming languages आहेत. नाव सारखे असले तरी दोन्ही एकच नाहीत.

2. Variables: var, let and const

Q5. What is a variable in JavaScript?

Interview Answer:

A variable is a named container used to store data in a program. JavaScript provides three keywords to declare variables: var, let, and const.

मराठी Explanation:

Variable म्हणजे data store करण्यासाठी वापरलेले नाव. उदाहरणार्थ username, age, email, price इत्यादी values variables मध्ये ठेवता येतात.

```
let username = "Manoj";

let age = 22;

const country = "India";


console.log(username);

console.log(age);

console.log(country);
```

Q6. What is the difference between var, let and const?

हा अत्यंत महत्त्वाचा interview question आहे.

Feature	**var**	**let**	**const**
Scope	Function	Block	Block
Re-declaration	Allowed	Not allowed in same scope	Not allowed in same scope
Re-assignment	Allowed	Allowed	Not allowed
Hoisting	Yes, initialized as undefined	Yes, but inaccessible before initialization	Yes, but inaccessible before initialization

Example:

```
var a = 10;

var a = 20; // Allowed


let b = 10;
```

```
b = 20; // Allowed


const c = 10;
// c = 20; // TypeError


console.log(a); // 20
console.log(b); // 20
console.log(c); // 10
```

मराठी Explanation:

- var: जुना variable declaration keyword. Function-scoped असतो.
- let: Value बदलायची असेल तर वापरतो. Block-scoped असतो.
- const: Variable पुन्हा assign करायचा नसेल तर वापरतो. Block-scoped असतो.

Industry Use:

```
const API_URL = "https://api.example.com";


let cartCount = 0;


cartCount = cartCount + 1;
```

साधारणपणे const ला preference देतात. Value पुन्हा assign करायची असल्यास let वापरतात. var शक्यतो टाळतात.

Q7. Can we change the value of a const object?

Interview Answer:

Yes. We can modify the properties of a const object, but we cannot assign a completely new object to the same variable.

Example:

```
const user = {
 name: "Manoj",
 age: 22
```

```
};

user.age = 23; // Allowed

console.log(user);

// Not allowed:
// user = { name: "Rahul" };
```

मराठी Explanation:

const object मधील properties बदलू शकतो. पण त्या variable ला नवीन object assign करू शकत नाही.

3. Data Types

Q8. What are data types in JavaScript?

Interview Answer:

Data types define the type of value stored in a variable. JavaScript has two main categories: Primitive and Non-Primitive data types.

1. Primitive Data Types

- String
- Number
- Boolean
- Undefined
- Null
- BigInt
- Symbol

2. Non-Primitive Data Types

- Object
- Array
- Function

Example:

```
let name = "Manoj";      // String
let age = 22;            // Number
let isDeveloper = true;  // Boolean
let city;                // Undefined
let salary = null;       // Null

let numbers = [10, 20, 30]; // Array

let user = {
  name: "Manoj",
  age: 22
}; // Object
```

Q9. What is the difference between null and undefined?

undefined	**null**
Variable declared but value not assigned	Intentional absence of value
Automatically assigned in certain cases	Usually assigned by developer
Type is undefined	typeof null returns object (historical JavaScript behavior)

Example:

```
let a;
let b = null;

console.log(a); // undefined
console.log(b); // null
```

```
console.log(typeof a); // undefined

console.log(typeof b); // object
```

मराठी Explanation:

- undefined: Variable तयार आहे पण त्याला value दिलेली नाही.
- null: Value मुद्दाम रिकामी ठेवली आहे.

Q10. What is the difference between primitive and reference data types?

Interview Answer:

Primitive values are copied by value, while objects and arrays are assigned and copied by reference.

Example: Primitive

```
let a = 10;

let b = a;


b = 20;


console.log(a); // 10

console.log(b); // 20
```

Example: Reference

```
let user1 = {
  name: "Manoj"
};


let user2 = user1;


user2.name = "Rahul";


console.log(user1.name); // Rahul

console.log(user2.name); // Rahul
```

मराठी Explanation:

Primitive मध्ये value ची स्वतंत्र copy तयार होते. Object किंवा array दुसऱ्या variable ला assign केल्यावर दोन्ही variables एकाच object चा reference वापरतात.

Q11. What is typeof operator?

typeof operator वापरून एखाद्या value चा type तपासता येतो.

```
console.log(typeof "Hello");   // string
console.log(typeof 100);       // number
console.log(typeof true);      // boolean
console.log(typeof undefined); // undefined
console.log(typeof null);      // object
console.log(typeof {});        // object
console.log(typeof []);        // object
console.log(typeof function(){}); // function
```

Important: Array तपासण्यासाठी typeof पुरेसा नाही.

```
console.log(Array.isArray([1, 2, 3])); // true
```

4. Operators

Q12. What are operators in JavaScript?

Operators are symbols used to perform operations on values and variables.

Types:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Ternary Operator
6. Nullish Coalescing Operator
7. Optional Chaining Operator

Example:

```
let a = 10;

let b = 5;


console.log(a + b);  // 15

console.log(a - b);  // 5

console.log(a * b);  // 50

console.log(a / b);  // 2

console.log(a % b);  // 0
```

Q13. What is the difference between == and ===?

Interview Answer:

== compares values after type conversion, while === compares both value and data type without type conversion.

```
console.log(10 == "10");  // true

console.log(10 === "10"); // false
```

मराठी Explanation:

== मध्ये JavaScript काही वेळा data type convert करून comparison करते.

=== मध्ये value आणि data type दोन्ही check केले जातात.

Industry Tip: शक्यतो === आणि !== वापरावेत.

Q14. What are truthy and falsy values?

Interview Answer:

Truthy values are values that behave like true in a Boolean context. Falsy values behave like false.

Falsy values in JavaScript:

```
false

0

-0

0n

""
```

null

undefined

NaN

बाकी बहुतेक values truthy असतात.

```
console.log(Boolean("Hello")); // true

console.log(Boolean(100));    // true

console.log(Boolean([]));     // true

console.log(Boolean({}));     // true


console.log(Boolean(""));     // false

console.log(Boolean(0));      // false

console.log(Boolean(null));   // false
```

Important Interview Point: Empty array [] आणि empty object {} हे truthy असतात.

Q15. What is the ternary operator?

Ternary operator is a short way to write an if-else condition.

```
let age = 20;


let result = age >= 18 ? "Adult" : "Minor";


console.log(result); // Adult
```

Syntax:

```
condition ? expressionIfTrue : expressionIfFalse;
```

Q16. What is the difference between || and ?? operators?

|| वापरतो जेव्हा left-side value falsy असेल.

?? वापरतो जेव्हा left-side value फक्त null किंवा undefined असेल.

```
let a = 0;

let b = a || 100;
```

```
let c = a ?? 100;


console.log(b); // 100

console.log(c); // 0
```

Industry Use: User ने price 0 दिली असेल तर ?? मुळे ती value योग्य प्रकारे retain करता येते.

Q17. What is optional chaining?

Optional chaining (?.) allows us to safely access nested object properties without getting an error when an intermediate property is null or undefined.

```
const user = {
 name: "Manoj",
 address: {
  city: "Kolhapur"
 }
};


console.log(user.address?.city);  // Kolhapur

console.log(user.contact?.phone); // undefined
```

Industry Use: API response मधील nested properties access करताना उपयोगी.

5. Scope and Hoisting

Q18. What is scope in JavaScript?

Interview Answer:

Scope defines where a variable can be accessed in a program.

Types of Scope:

1. Global Scope
2. Function Scope
3. Block Scope
4. Module Scope

Example:

```
let globalName = "Manoj";


function showData() {
  let localName = "Developer";


  console.log(globalName); // Allowed
  console.log(localName);  // Allowed
}


showData();


// console.log(localName); // Error
```

मराठी Explanation:

Scope म्हणजे variable कुठून access करता येईल याची मर्यादा.

Q19. What is block scope?

A block is a section of code written inside { }. Variables declared using let and const inside a block cannot be accessed outside that block.

```
{
  let a = 10;
  const b = 20;


  console.log(a);
  console.log(b);
}


// console.log(a); // ReferenceError
// console.log(b); // ReferenceError
```

Q20. What is hoisting in JavaScript?

Interview Answer:

Hoisting is JavaScript's behavior of processing declarations before code execution. The behavior differs for var, let, const, and function declarations.

Example: var

```
console.log(a); // undefined


var a = 10;
```

JavaScript याचा अर्थ साधारणपणे असा हाताळते:

```
var a;


console.log(a);


a = 10;
```

Example: let and const

```
console.log(a); // ReferenceError


let a = 10;
```

Example: Function Declaration

```
greet();


function greet() {
  console.log("Hello");
}
```

Output:

Hello

मराठी Explanation:

Hoisting मध्ये declarations execution आधी process होतात. पण याचा अर्थ पूर्ण code वर हलवला जातो असा नाही.

Q21. What is the Temporal Dead Zone (TDZ)?

Interview Answer:

The Temporal Dead Zone is the time between entering a scope and initializing a let or const variable. Accessing that variable during this period causes a ReferenceError.

```
{
  // TDZ starts

  // console.log(name); // ReferenceError

  let name = "Manoj";

  // TDZ ends
  console.log(name);
}
```

मराठी Explanation:

let किंवा const variable declare होण्यापूर्वी access केला तर ReferenceError येतो. या मधल्या कालावधीला TDZ म्हणतात.

6. Functions

Q22. What is a function in JavaScript?

Interview Answer:

A function is a reusable block of code designed to perform a specific task. It helps reduce code duplication and improves code organization.

```
function add(a, b) {
  return a + b;
}

let result = add(10, 20);
```

```
console.log(result); // 30
```

मराठी Explanation:

Function म्हणजे एखादे specific काम करण्यासाठी लिहिलेला reusable code block.

Q23. What are the different types of functions?

1. Function Declaration

```
function greet() {
  console.log("Hello");
}

greet();
```

2. Function Expression

```
const greet = function() {
  console.log("Hello");
};

greet();
```

3. Arrow Function

```
const greet = () => {
  console.log("Hello");
};

greet();
```

4. Anonymous Function

```
setTimeout(function() {
  console.log("Hello");
}, 1000);
```

5. Immediately Invoked Function Expression (IIFE)

```
(function() {
```

```
 console.log("IIFE executed");
})();
```

6. Callback Function

```
function processUser(callback) {
 callback();
}

processUser(function() {
 console.log("User processed");
});
```

Q24. What is the difference between normal function and arrow function?

Normal Function	**Arrow Function**
Has its own this based on how it is called	Inherits this from surrounding scope
Can be used as a constructor with new	Cannot be used as a constructor
Has arguments object	Does not have its own arguments
Can be hoisted when declared as a function declaration	Arrow function expression is not usable before initialization

Example:

```
function add(a, b) {
 return a + b;
}

const multiply = (a, b) => a * b;

console.log(add(10, 20));     // 30
console.log(multiply(10, 20)); // 200
```

Q25. What is a callback function?

Interview Answer:

A callback is a function passed as an argument to another function and executed by that function.

```
function greet(name, callback) {
  console.log("Hello " + name);

  callback();
}

function welcome() {
  console.log("Welcome to JavaScript");
}

greet("Manoj", welcome);
```

Output:

Hello Manoj

Welcome to JavaScript

मराठी Explanation:

एका function मध्ये दुसरा function argument म्हणून pass केला आणि नंतर execute केला तर त्याला callback function म्हणतात.

Industry Use:

- Event handling.
- Array methods.
- API response handling.
- Timers.

Q26. What is a higher-order function?

Interview Answer:

A higher-order function is a function that takes another function as an argument or returns a function.

```
function calculate(a, b, operation) {
  return operation(a, b);
}

function add(x, y) {
  return x + y;
}

console.log(calculate(10, 20, add)); // 30
```

मराठी Explanation:

जो function दुसरा function accept करतो किंवा function return करतो त्याला Higher-Order Function म्हणतात.

Examples: map(), filter(), reduce(), forEach().

Q27. What is a closure in JavaScript?

Very Important Interview Question

Interview Answer:

A closure is created when an inner function remembers and can access variables from its outer function, even after the outer function has finished executing.

Example:

```
function outerFunction() {
  let count = 0;

  function innerFunction() {
    count++;
    console.log(count);
  }

  return innerFunction;
```

```
}


const counter = outerFunction();


counter(); // 1

counter(); // 2

counter(); // 3
```

मराठी Explanation:

Inner function ला त्याच्या outer function मधील variables लक्षात राहतात. Outer function execute होऊन संपल्यानंतरसुद्धा inner function त्या variables ला access करू शकतो.

Industry Use:

- Private variables.
- Counter functions.
- Data encapsulation.
- Function factories.
- React closures and hooks behavior.

Q28. What is recursion?

Interview Answer:

Recursion is a programming technique in which a function calls itself until a base condition is reached.

```
function factorial(n) {
 if (n === 1) {
  return 1;
 }


 return n * factorial(n - 1);
}
```

```
console.log(factorial(5)); // 120
```

मराठी Explanation:

Function स्वतःलाच पुन्हा call करतो त्याला recursion म्हणतात. Recursion थांबण्यासाठी base condition आवश्यक असते.

7. Arrays

Q29. What is an array in JavaScript?

Interview Answer:

An array is a special object used to store multiple values in a single variable. Array elements are accessed using their index, starting from zero.

```
const fruits = ["Apple", "Banana", "Mango"];


console.log(fruits[0]); // Apple

console.log(fruits[1]); // Banana

console.log(fruits.length); // 3
```

Q30. What are the important array methods?

Method	Purpose
push()	Adds element at end
pop()	Removes last element
unshift()	Adds element at beginning
shift()	Removes first element
map()	Creates a new transformed array
filter()	Creates a new filtered array
reduce()	Reduces array to a single result
find()	Returns first matching element
findIndex()	Returns index of first matching element
includes()	Checks if an element exists

some()	Checks if at least one element matches
every()	Checks if all elements match
forEach()	Executes a function for each element
slice()	Returns a shallow copy of a portion
splice()	Adds/removes elements in original array
sort()	Sorts elements in place
join()	Converts array elements into a string

Q31. What is the difference between map() and forEach()?

Interview Answer:

map() creates and returns a new array after applying a function to each element.
forEach() executes a function for each element but returns undefined.

```
const numbers = [1, 2, 3, 4];


const result1 = numbers.map(num => num * 2);


const result2 = numbers.forEach(num => {
  console.log(num * 2);
});


console.log(result1); // [2, 4, 6, 8]
console.log(result2); // undefined
```

मराठी Explanation:

- map() वापरून नवीन array तयार करतो.
- forEach() वापरून प्रत्येक element वर operation करतो; नवीन array return करत नाही.

Industry Use: React मध्ये products किंवा users ची list render करण्यासाठी map() वापरतात.

Q32. What is the difference between filter() and find()?

```
const numbers = [10, 20, 30, 40];
```

```
const result1 = numbers.filter(num => num > 20);

const result2 = numbers.find(num => num > 20);


console.log(result1); // [30, 40]

console.log(result2); // 30
```

- filter() सर्व matching elements असलेली array return करतो.
- find() पहिला matching element return करतो.

Q33. What is reduce() in JavaScript?

Interview Answer:

The reduce() method executes a reducer function on each array element and produces a single accumulated result.

```
const numbers = [10, 20, 30, 40];


const total = numbers.reduce((sum, num) => {
  return sum + num;
}, 0);


console.log(total); // 100
```

मराठी Explanation:

Array मधील सर्व elements एकत्र करून single result तयार करण्यासाठी reduce() वापरतो.

Industry Use:

- Shopping cart total.
- Total expenses.
- Product quantity calculation.
- Grouping data.

Q34. What is the difference between slice() and splice()?

```
const arr1 = [10, 20, 30, 40];
```

```
const result1 = arr1.slice(1, 3);

console.log(result1); // [20, 30]
console.log(arr1);    // [10, 20, 30, 40]
```

slice() original array बदलत नाही.

```
const arr2 = [10, 20, 30, 40];

const result2 = arr2.splice(1, 2);

console.log(result2); // [20, 30]
console.log(arr2);    // [10, 40]
```

splice() original array बदलतो.

Q35. How do you remove duplicate values from an array?

```
const numbers = [10, 20, 10, 30, 20, 40];

const uniqueNumbers = [...new Set(numbers)];

console.log(uniqueNumbers); // [10, 20, 30, 40]
```

Explanation: Set मध्ये unique values साठवल्या जातात. Spread operator वापरून पुन्हा array तयार करतो.

Q36. How do you sort numbers in JavaScript?

```
const numbers = [40, 10, 100, 5];

numbers.sort((a, b) => a - b);

console.log(numbers); // [5, 10, 40, 100]
```

Descending:

numbers.sort((a, b) => b - a);

Important: Default sort() elements ना string म्हणून compare करू शकतो. Numeric sorting साठी comparator function वापरा.

8. Objects

Q37. What is an object in JavaScript?

Interview Answer:

An object is a collection of key-value pairs. It is used to store related data and functionality together.

```
const user = {
  name: "Manoj",
  age: 22,
  role: "Developer",

  greet: function() {
    console.log("Hello");
  }
};

console.log(user.name);
console.log(user.age);

user.greet();
```

Q38. How can we access object properties?

```
const user = {
  name: "Manoj",
  age: 22
};
```

```
// Dot notation
console.log(user.name);

// Bracket notation
console.log(user["age"]);
```

Bracket notation dynamic property names साठी उपयोगी आहे.

```
const key = "name";

console.log(user[key]); // Manoj
```

Q39. What is object destructuring?

Interview Answer:

Object destructuring is a feature that allows us to extract properties from an object into variables.

```
const user = {
  name: "Manoj",
  age: 22,
  city: "Kolhapur"
};

const { name, age, city } = user;

console.log(name);
console.log(age);
console.log(city);
```

मराठी Explanation:

Object मधील properties थेट वेगळ्या variables मध्ये घेण्याच्या पद्धतीला destructuring म्हणतात.

Q40. What is the spread operator?

Spread operator (...) expands elements of an iterable or properties of an object.

```
const arr1 = [10, 20];

const arr2 = [30, 40];


const result = [...arr1, ...arr2];


console.log(result); // [10, 20, 30, 40]
```

Object example:

```
const user = {
  name: "Manoj",
  age: 22
};


const updatedUser = {
  ...user,
  age: 23,
  city: "Kolhapur"
};


console.log(updatedUser);
```

Q41. What is the rest operator?

Rest operator collects multiple values into a single array or object.

```
function addNumbers(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}


console.log(addNumbers(10, 20, 30)); // 60
```

Difference:

- Spread: Values expand करतो.
- Rest: Multiple values collect करतो.

दोन्हींसाठी ... syntax वापरले जाते. Context नुसार त्यांचा अर्थ ठरतो.

Q42. What is object cloning?

Object cloning म्हणजे existing object ची copy तयार करणे.

Shallow Copy:

```
const user = {
 name: "Manoj",
 address: {
  city: "Kolhapur"
 }
};

const copy = { ...user };

copy.name = "Rahul";

console.log(user.name); // Manoj
```

पण nested object साठी shallow copy पुरेशी नसते:

```
copy.address.city = "Pune";

console.log(user.address.city); // Pune
```

Deep Copy:

```
const deepCopy = structuredClone(user);

deepCopy.address.city = "Mumbai";
```

```
console.log(user.address.city); // Kolhapur
```

structuredClone() supported data types साठी deep copy तयार करते. Functions सारख्या सर्व प्रकारच्या values clone करता येत नाहीत.

9. this Keyword

Q43. What is this keyword in JavaScript?

Interview Answer:

The this keyword refers to the execution context of a function. Its value depends on how the function is called.

Example: Object Method

```
const user = {
  name: "Manoj",

  greet() {
    console.log(this.name);
  }
};

user.greet(); // Manoj
```

Example: Normal Function

```
function show() {
  console.log(this);
}

show();
```

Non-strict browser script मध्ये this global object असू शकतो; strict mode आणि ES modules मध्ये साधारणपणे undefined असतो.

Example: Arrow Function

```
const user = {
```

```
  name: "Manoj",

  greet: () => {
    console.log(this.name);
  }
};

user.greet();
```

Arrow function स्वतःचा this तयार करत नाही. तो surrounding scope मधून घेतो.

Q44. What are call(), apply() and bind()?

या methods चा उपयोग function चा this context ठरवण्यासाठी होतो.

call():

```
function greet(city) {
  console.log(this.name + " from " + city);
}

const user = {
  name: "Manoj"
};

greet.call(user, "Kolhapur");
```

apply():

```
greet.apply(user, ["Pune"]);
```

bind():

```
const newFunction = greet.bind(user, "Mumbai");

newFunction();
```

Method	Description
call()	Function लगेच execute करतो; arguments वेगवेगळे pass करतो
apply()	Function लगेच execute करतो; arguments array मध्ये pass करतो
bind()	नवीन function return करतो; नंतर execute करू शकतो

10. DOM Manipulation

Q45. What is DOM in JavaScript?

Interview Answer:

DOM stands for Document Object Model. It represents an HTML document as a tree of objects. JavaScript uses the DOM to access and modify HTML elements, content, attributes, and styles.

मराठी Explanation:

Browser HTML page ला object tree स्वरूपात represent करतो. JavaScript च्या मदतीने आपण त्या elements मध्ये बदल करू शकतो.

DOM Tree: How JavaScript manipulates web pages | Lavare Nitin posted on the topic | LinkedIn

linkedin.com

Example:

```
<h1 id="title">Hello</h1>


<script>
 const heading = document.getElementById("title");


 heading.textContent = "Welcome to JavaScript";
</script>
```

Q46. What are the methods used to select DOM elements?

Method	Purpose
getElementById()	Selects element by ID

getElementsByClassName()	Selects elements by class
getElementsByTagName()	Selects elements by tag
querySelector()	Selects first matching CSS selector
querySelectorAll()	Selects all matching CSS selectors

```
const title = document.getElementById("title");

const firstBox = document.querySelector(".box");

const allBoxes = document.querySelectorAll(".box");
```

Q47. What is the difference between innerHTML, innerText and textContent?

```
<div id="box">
  Hello <b>Manoj</b>
</div>
const box = document.getElementById("box");

console.log(box.innerHTML);
// Hello <b>Manoj</b>

console.log(box.innerText);
// Hello Manoj

console.log(box.textContent);
// Hello Manoj
```

Property	Purpose
innerHTML	Reads or sets HTML markup
innerText	Reads or sets rendered text

textContent Reads or sets text content, including hidden text

Security Tip: Untrusted user input साठी innerHTML वापरल्यास XSS vulnerability निर्माण होऊ शकते. Text दाखवण्यासाठी textContent अधिक सुरक्षित पर्याय आहे.

Q48. What is event handling?

Interview Answer:

Event handling is the process of responding to user actions such as clicks, keyboard input, mouse movement, and form submission.

```
<button id="btn">Click Me</button>


<script>
 const button = document.getElementById("btn");


 button.addEventListener("click", function() {
  console.log("Button clicked");
 });
</script>
```

Q49. What is event bubbling?

Interview Answer:

Event bubbling is a process in which an event starts from the target element and propagates upward through its parent elements.

Capture VS Bubble - DEV Community

dev.to

```
<div id="parent">
 Parent
 <button id="child">Click</button>
</div>


<script>
```

```
 document.getElementById("parent")
  .addEventListener("click", () => {
   console.log("Parent clicked");
  });

 document.getElementById("child")
  .addEventListener("click", () => {
   console.log("Child clicked");
  });
</script>
```

Button click केल्यावर:

Child clicked

Parent clicked

Q50. What is event capturing?

Event capturing is the opposite propagation phase of bubbling. The event travels from the outermost ancestor toward the target element.

element.addEventListener("click", handler, true);

Third argument true दिल्यास capturing phase मध्ये listener execute होतो.

Q51. What is event delegation?

Interview Answer:

Event delegation is a technique where we attach one event listener to a parent element instead of attaching separate listeners to multiple child elements.

```
<ul id="list">
 <li>HTML</li>
 <li>CSS</li>
 <li>JavaScript</li>
</ul>
```

```
<script>
 document.getElementById("list")
  .addEventListener("click", function(event) {
   if (event.target.tagName === "LI") {
    console.log(event.target.textContent);
   }
  });
</script>
```

Industry Use:

- Dynamic lists.
- Product lists.
- Table rows.
- Large menus.

11. Asynchronous JavaScript

Q52. What is synchronous programming?

Interview Answer:

Synchronous programming executes code one statement at a time. Each operation waits for the previous operation to finish.

```
console.log("First");
console.log("Second");
console.log("Third");
```

Output:

First

Second

Third

Q53. What is asynchronous programming?

Interview Answer:

Asynchronous programming allows certain operations to complete later without blocking the main JavaScript execution flow.

```
console.log("First");


setTimeout(() => {
  console.log("Second");
}, 2000);


console.log("Third");
```

Output:

First

Third

Second

मराठी Explanation:

setTimeout() सारखे काम पूर्ण होण्यासाठी वेळ लागू शकतो. पण JavaScript त्याची वाट पाहत बसत नाही; पुढचा synchronous code execute करते.

Q54. What is the event loop?

Very Important

Interview Answer:

The event loop is a mechanism that coordinates the call stack and task queues to execute asynchronous callbacks when the call stack is empty.

Important Components:

1. Call Stack
2. Web APIs / Runtime APIs
3. Task Queue (Macrotask Queue)
4. Microtask Queue
5. Event Loop

JavaScript Event Loop Explained: Macrotasks, Microtasks, and Async/Await Made Simple - DEV Community

dev.to

मराठी Explanation:

JavaScript चा मुख्य execution model single-threaded आहे. Asynchronous कामे browser किंवा Node.js runtime हाताळते. काम पूर्ण झाल्यावर callbacks queues मध्ये येतात. Event loop call stack रिकामा झाल्यावर callbacks execute करण्यास मदत करतो.

Example:

```
console.log("Start");


setTimeout(() => {
  console.log("Timeout");
}, 0);


Promise.resolve().then(() => {
  console.log("Promise");
});


console.log("End");
```

Output:

Start

End

Promise

Timeout

Why?

- Synchronous code आधी execute होतो.
- Promise callback microtask queue मध्ये जातो.
- setTimeout callback task queue मध्ये जातो.
- Microtasks आधी process होतात.

Q55. What is a Promise?

Interview Answer:

A Promise is an object that represents the eventual completion or failure of an asynchronous operation.

Promise States:

1. Pending
2. Fulfilled
3. Rejected

```
const promise = new Promise((resolve, reject) => {
 let success = true;

 if (success) {
  resolve("Data loaded successfully");
 } else {
  reject("Something went wrong");
 }
});

promise
 .then(result => {
  console.log(result);
 })
 .catch(error => {
  console.log(error);
 })
 .finally(() => {
  console.log("Operation completed");
 });
```

मराठी Explanation:

Promise म्हणजे भविष्यात मिळणाऱ्या result चे representation. Operation यशस्वी झाल्यास fulfilled आणि fail झाल्यास rejected होते.

Q56. What is the difference between then(), catch() and finally()?

Method	Purpose
then()	Handles fulfilled result
catch()	Handles rejection/error
finally()	Runs after settlement, regardless of outcome

Q57. What is async/await?

Interview Answer:

Async/await is syntax built on Promises that makes asynchronous code easier to read and write.

```
async function getData() {
 try {
   const response = await fetch(
    "https://jsonplaceholder.typicode.com/users"
   );

   const data = await response.json();

   console.log(data);
 } catch (error) {
   console.log(error);
 }
}

getData();
```

मराठी Explanation:

- async function नेहमी Promise return करतो.
- await Promise settle होईपर्यंत त्या async function मधील पुढील execution थांबवतो.
- try/catch वापरून errors handle करू शकतो.

Industry Use: Backend API calls, database operations, authentication, file uploads.

Q58. What is the difference between Promise.all() and Promise.allSettled()?

Promise.all():

सर्व promises fulfilled झाल्यास results देते. एखादा promise reject झाला तर Promise.all reject होतो.

```
const p1 = Promise.resolve("A");
const p2 = Promise.resolve("B");

Promise.all([p1, p2]).then(result => {
  console.log(result); // ["A", "B"]
});
```

Promise.allSettled():

सर्व promises settle होईपर्यंत wait करतो आणि प्रत्येकाचा status व result देतो.

```
const p1 = Promise.resolve("Success");
const p2 = Promise.reject("Failed");

Promise.allSettled([p1, p2]).then(result => {
  console.log(result);
});
```

Other Promise Methods:

- Promise.race() – सर्वात आधी settle झालेल्या Promise चा result.
- Promise.any() – सर्वात आधी fulfilled झालेल्या Promise चा result; सर्व reject झाल्यास AggregateError.

12. ES6+ Features

Q59. What is ES6?

Interview Answer:

ES6 stands for ECMAScript 2015. It introduced many important JavaScript features that make code cleaner and easier to maintain.

Important ES6 Features:

1. let and const
2. Arrow Functions
3. Template Literals
4. Destructuring
5. Spread and Rest Operators
6. Default Parameters
7. Classes
8. Modules
9. Promises
10. Enhanced Object Literals

Q60. What are template literals?

Template literals allow us to create strings using backticks and insert expressions using ${}.

```
const name = "Manoj";

const age = 22;


console.log(` My name is ${name} and my age is ${age}` );
```

Q61. What are default parameters?

Default parameters provide default values to function parameters when no value or undefined is passed.

```
function greet(name = "Guest") {
  console.log(` Hello ${name}` );
}
```

```
greet("Manoj"); // Hello Manoj

greet();       // Hello Guest
```

Q62. What are JavaScript modules?

Modules allow us to split JavaScript code into separate files and reuse functionality through imports and exports.

Export:

```
// math.js


export function add(a, b) {
  return a + b;
}
```

Import:

```
// app.js


import { add } from "./math.js";


console.log(add(10, 20));
```

मराठी Explanation:

मोठ्या application मध्ये सर्व code एका file मध्ये न ठेवता वेगवेगळ्या files मध्ये divide करतो. त्यासाठी import आणि export वापरतो.

13. Object-Oriented Programming (OOP)

Q63. What is OOP in JavaScript?

Interview Answer:

Object-Oriented Programming is a programming approach based on objects. It helps organize code into reusable and manageable structures.

Four Main Pillars:

1. Encapsulation
2. Inheritance

3. Polymorphism
4. Abstraction

Q64. What is a class?

A class is a blueprint for creating objects.

```
class User {
 constructor(name, age) {
  this.name = name;
  this.age = age;
 }

 greet() {
  console.log(`Hello ${this.name}`);
 }
}

const user1 = new User("Manoj", 22);

user1.greet();
```

Q65. What is a constructor?

A constructor is a special method that runs automatically when an object is created using the new keyword.

```
class Student {
 constructor(name) {
  this.name = name;
 }
}

const student = new Student("Manoj");
```

```
console.log(student.name);
```

Q66. What is inheritance?

Inheritance allows one class to reuse properties and methods of another class.

```
class Animal {
  eat() {
    console.log("Animal is eating");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog is barking");
  }
}

const dog = new Dog();

dog.eat();
dog.bark();
```

Q67. What is encapsulation?

Encapsulation means keeping data and related methods together and controlling access to internal data.

```
class BankAccount {
  #balance = 1000;

  deposit(amount) {
    if (amount > 0) {
```

```
    this.#balance += amount;
  }
 }

 getBalance() {
  return this.#balance;
 }
}

const account = new BankAccount();

account.deposit(500);

console.log(account.getBalance()); // 1500
```

#balance हा private class field आहे.

Q68. What is polymorphism?

Polymorphism means the same method name can have different behavior in different classes.

```
class Animal {
 sound() {
  console.log("Animal makes a sound");
 }
}

class Dog extends Animal {
 sound() {
  console.log("Dog barks");
 }
```

```
}

class Cat extends Animal {
  sound() {
    console.log("Cat meows");
  }
}

new Dog().sound(); // Dog barks
new Cat().sound(); // Cat meows
```

14. Error Handling

Q69. What is error handling in JavaScript?

Error handling is the process of detecting and managing errors so that the application can respond properly instead of failing unexpectedly.

Example:

```
try {
  let result = 10 / 0;

  console.log(result);
} catch (error) {
  console.log("Something went wrong");
} finally {
  console.log("Execution completed");
}
```

Note: JavaScript मध्ये 10 / 0 मुळे Infinity मिळते; error throw होत नाही. हा फक्त try/catch syntax चा example आहे.

Better example:

```
try {
```

```
  throw new Error("Invalid user data");
} catch (error) {
  console.log(error.message);
}
```

Q70. What is the difference between throw and try/catch?

- throw manually error create करून फेकतो.
- try मध्ये error येऊ शकणारा code ठेवतो.
- catch error handle करतो.
- finally cleanup code चालवतो.

```
function checkAge(age) {
  if (age < 18) {
    throw new Error("Age must be 18 or above");
  }

  return "Allowed";
}

try {
  console.log(checkAge(16));
} catch (error) {
  console.log(error.message);
}
```

15. Local Storage, Session Storage and Cookies

Q71. What is localStorage?

Interview Answer:

LocalStorage is a browser storage mechanism that stores key-value pairs with no automatic expiration. Data remains available even after the browser is closed, until it is cleared.

```
localStorage.setItem("username", "Manoj");

const name = localStorage.getItem("username");

console.log(name);

localStorage.removeItem("username");

localStorage.clear();
```

Q72. What is sessionStorage?

SessionStorage stores key-value pairs for a browser tab session. Data is generally removed when that tab or window session ends.

```
sessionStorage.setItem("token", "abc123");

console.log(sessionStorage.getItem("token"));
```

Q73. What is the difference between localStorage, sessionStorage and cookies?

Feature	**localStorage**	**sessionStorage**	**Cookies**
Storage	Browser	Browser tab session	Browser
Expiration	No automatic expiry	Tab session	Depends on expiry settings
Typical capacity	Several MB	Several MB	Around 4 KB per cookie
Sent with HTTP requests	No	No	Yes, matching cookies are sent automatically
Access	JavaScript	JavaScript	Depends on HttpOnly setting

Security Important:

Authentication tokens साठी localStorage/sessionStorage मध्ये संवेदनशील token ठेवण्याचे security risks आहेत. योग्य backend architecture मध्ये HttpOnly, Secure, SameSite cookie

वापरणे उपयुक्त ठरू शकते. पण CSRF protection आणि deployment configuration सुद्धा योग्य असणे आवश्यक आहे.

16. JavaScript Advanced Concepts

Q74. What is the difference between shallow copy and deep copy?

- Shallow copy: Top-level properties copy होतात; nested objects चे references share होतात.
- Deep copy: Nested objects सुद्धा स्वतंत्रपणे copy होतात.

हे concept Object cloning मध्ये दाखवले आहे. Interview मध्ये nested object चा example नक्की तयार ठेव.

Q75. What is debouncing?

Interview Answer:

Debouncing delays the execution of a function until a specified time has passed since the last event.

Example:

```
function debounce(callback, delay) {
 let timer;

 return function(...args) {
  clearTimeout(timer);

  timer = setTimeout(() => {
   callback(...args);
  }, delay);
 };
}

const search = debounce((value) => {
 console.log("Searching:", value);
```

```
}, 500);


search("M");

search("Ma");

search("Man");

search("Mano");

search("Manoj");
```

इथे शेवटचा input आल्यानंतर 500ms ने callback execute होतो.

Industry Use:

- Search suggestions.
- Input validation.
- Auto-save.
- Resize events.

Q76. What is throttling?

Interview Answer:

Throttling ensures that a function executes at most once during a specified time interval.

Example:

```
function throttle(callback, delay) {
 let lastTime = 0;


 return function(...args) {
  const now = Date.now();


  if (now - lastTime >= delay) {
   lastTime = now;
   callback(...args);
  }
```

```
  };
}

const handleScroll = throttle(() => {
  console.log("Scroll event");
}, 1000);

window.addEventListener("scroll", handleScroll);
```

Difference:

Debouncing	**Throttling**
Last event नंतर function execute	ठरावीक interval मध्ये जास्तीत जास्त एकदा execute
Search input	Scroll event
Resize handling	Mouse movement

Q77. What is memory leak in JavaScript?

Interview Answer:

A memory leak happens when memory that is no longer needed remains reachable and cannot be garbage-collected.

Common Causes:

- Unremoved event listeners.
- Unnecessary global variables.
- Timers not cleared.
- Unwanted references.
- Detached DOM elements still referenced.

Example:

```
function startTimer() {
  const timer = setInterval(() => {
    console.log("Running");
  }, 1000);
```

```
  return () => clearInterval(timer);
}

const stopTimer = startTimer();

// Stop when no longer needed
stopTimer();
```

Q78. What is garbage collection?

Garbage collection is the automatic process of freeing memory used by objects that are no longer reachable by the program.

मराठी Explanation:

आपल्या application मध्ये एखाद्या object चा वापर संपला आणि त्याला कुठूनही access करता येत नसेल तर JavaScript engine त्या memory ला reclaim करू शकतो.

Q79. What is strict mode?

Strict mode enables stricter parsing and error handling in JavaScript.

```
"use strict";

x = 10; // ReferenceError
```

Strict mode मुळे काही चुका लवकर समजतात आणि unsafe patterns टाळण्यास मदत होते. ES modules आणि class code strict mode मध्येच execute होतात.

Q80. What is the difference between setTimeout() and setInterval()?

setTimeout()	**setInterval()**
Function एकदा execute करतो	ठरावीक interval ने पुन्हा execute करतो
clearTimeout() ने cancel करतो	clearInterval() ने cancel करतो

```
setTimeout(() => {
  console.log("After 2 seconds");
```

```
}, 2000);
const timer = setInterval(() => {
  console.log("Every 2 seconds");
}, 2000);

// Stop interval
clearInterval(timer);
```

17. JavaScript in Real MERN Projects

हे प्रश्न MERN Developer interview साठी महत्त्वाचे आहेत.

Q81. How does JavaScript communicate with an API?

JavaScript fetch() किंवा Axios सारख्या libraries वापरून HTTP requests पाठवू शकते.

```
async function getUsers() {
  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );

    if (!response.ok) {
      throw new Error("Failed to fetch users");
    }

    const users = await response.json();

    console.log(users);
  } catch (error) {
    console.log(error.message);
  }
}
```

getUsers();

मराठी Explanation:

Frontend वरून API request पाठवतो. Backend response देतो. JavaScript त्या response मधील data process करते.

Q82. What is JSON?

Interview Answer:

JSON stands for JavaScript Object Notation. It is a lightweight data format used to exchange data between applications.

```
const user = {
 name: "Manoj",
 age: 22
};

const jsonData = JSON.stringify(user);

console.log(jsonData);

const normalObject = JSON.parse(jsonData);

console.log(normalObject.name);
```

- JSON.stringify() – JavaScript value चे JSON string मध्ये conversion.
- JSON.parse() – JSON string चे JavaScript value मध्ये conversion.

Q83. What is CORS?

Interview Answer:

CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that allows a server to specify which origins can access its resources.

मराठी Explanation:

समजा React frontend localhost:5173 वर आणि Express backend localhost:5000 वर चालत आहे. दोन्हींचे origins वेगळे आहेत. Backend ने योग्य CORS policy allow केली तर browser frontend ला API response access करू देतो.

Express Example:

```
import express from "express";

import cors from "cors";


const app = express();


app.use(cors({

 origin: "http://localhost:5173",

 credentials: true

}));
```

Q84. What is XSS?

Interview Answer:

XSS stands for Cross-Site Scripting. It is a security vulnerability where an attacker injects malicious scripts into a webpage viewed by other users.

Protection:

- Escape untrusted output.
- Avoid unsafe innerHTML.
- Sanitize HTML when HTML input is required.
- Use Content Security Policy (CSP).
- Use secure frameworks correctly.

Q85. What is the difference between authentication and authorization?

Authentication	**Authorization**
Checks who the user is	Checks what the user can access
Login process	Role/permission checking

Example: Email and password Example: Admin-only dashboard

MERN Example:

- Authentication: User login करतो आणि backend त्याची identity verify करतो.
- Authorization: Admin ला users delete करण्याची permission असते; normal user ला नसते.

18. Output-Based JavaScript Interview Questions

या section चा interview मध्ये खूप उपयोग होईल. आधी स्वतः output सांगायचा प्रयत्न कर.

Q86. What is the output?

```
console.log(a);


var a = 10;
```

Answer: undefined

Reason: var declaration hoist होते आणि initialization आधी value undefined असते.

Q87. What is the output?

```
console.log(typeof null);

console.log(typeof []);

console.log(typeof {});
```

Answer:

object

object

object

Reason: typeof null हा JavaScript मधील historical behavior आहे. Array आणि object दोन्ही object दाखवतात.

Q88. What is the output?

```
console.log(1 + "2" + 3);

console.log(1 + 2 + "3");
```

Answer:

123

33

Reason: String concatenation आणि numeric addition यांचा क्रम महत्त्वाचा आहे.

Q89. What is the output?

console.log(true + true);

console.log(false + 10);

Answer:

2

10

Reason: Numeric context मध्ये true चे conversion 1 आणि false चे 0 होते.

Q90. What is the output?

let a = [1, 2, 3];

let b = a;

b.push(4);

console.log(a);

Answer:

[1, 2, 3, 4]

Reason: दोन्ही variables एकाच array चा reference वापरतात.

Q91. What is the output?

console.log([] == false);

console.log([] === false);

Answer:

true

false

Reason: == मध्ये type coercion होते; === मध्ये type आणि value दोन्ही तपासले जातात.

Q92. What is the output?

```
console.log("5" - 2);

console.log("5" + 2);
```

Answer:

3

52

Reason: - numeric conversion करतो; + string concatenation करू शकतो.

Q93. What is the output?

```
console.log(a);


let a = 10;
```

Answer: ReferenceError

Reason: let variable TDZ मध्ये access केला आहे.

Q94. What is the output?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 1000);
}
```

Answer:

3

3

3

Reason: var function-scoped आहे. तीनही callbacks एकाच i variable ला access करतात. Loop पूर्ण झाल्यावर i = 3 असतो.

Solution using let:

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => {
```

```
  console.log(i);
 }, 1000);
}
```

Output:

0

1

2

Q95. What is the output?

```
console.log("A");

Promise.resolve().then(() => {
 console.log("B");
});

setTimeout(() => {
 console.log("C");
}, 0);

console.log("D");
```

Answer:

A

D

B

C

Reason: Synchronous code → Microtask → Macrotask.

19. Practical JavaScript Coding Interview Questions

हे coding questions Junior Software Developer / MERN Developer interview साठी prepare कर.

Q96. Reverse a string.

```
function reverseString(str) {
  return str.split("").reverse().join("");
}

console.log(reverseString("Manoj")); // jo naM
```

Correct output:

jonaM

Q97. Check whether a string is palindrome.

```
function isPalindrome(str) {
  const reversed = str.split("").reverse().join("");

  return str === reversed;
}

console.log(isPalindrome("madam")); // true
console.log(isPalindrome("hello")); // false
```

Q98. Find the largest number in an array.

```
function findLargest(numbers) {
  return Math.max(...numbers);
}

console.log(findLargest([10, 50, 20, 90, 30]));
// 90
```

Q99. Find the second largest number in an array.

```
function secondLargest(numbers) {
  const unique = [...new Set(numbers)];
```

```
  unique.sort((a, b) => b - a);

  return unique.length >= 2 ? unique[1] : null;
}

console.log(secondLargest([10, 50, 20, 90, 30]));
// 50
```

Q100. Count vowels in a string.

```
function countVowels(str) {
  const matches = str.match(/[aeiou]/gi);

  return matches ? matches.length : 0;
}

console.log(countVowels("JavaScript")); // 3
```

Q101. Find duplicate elements in an array.

```
function findDuplicates(arr) {
  return arr.filter((item, index) => {
    return arr.indexOf(item) !== index;
  });
}

console.log(findDuplicates([1, 2, 3, 2, 4, 1]));
// [2, 1]
```

Q102. Find the sum of array elements.

```
const numbers = [10, 20, 30, 40];

const sum = numbers.reduce((total, num) => total + num, 0);
```

```
console.log(sum); // 100
```

Q103. Check whether a number is prime.

```
function isPrime(num) {
  if (num <= 1) return false;

  for (let i = 2; i <= Math.sqrt(num); i++) {
    if (num % i === 0) {
      return false;
    }
  }

  return true;
}

console.log(isPrime(7));  // true
console.log(isPrime(10)); // false
```

Q104. Find factorial of a number.

```
function factorial(num) {
  let result = 1;

  for (let i = 1; i <= num; i++) {
    result *= i;
  }

  return result;
}
```

```
console.log(factorial(5)); // 120
```

Q105. Remove duplicate objects based on id.

```
const users = [
 { id: 1, name: "Manoj" },
 { id: 2, name: "Rahul" },
 { id: 1, name: "Manoj" },
 { id: 3, name: "Amit" }
];

const uniqueUsers = [
  ...new Map(users.map(user => [user.id, user])).values()
];

console.log(uniqueUsers);
```

Industry Use: API मधून आलेल्या duplicate records remove करण्यासाठी.

Part 5: Git & GitHub Interview Questions and Answers (Beginner to Advanced)

1. Git Fundamentals

Q1. What is Git?

Interview Answer:

Git is a free and open-source distributed version control system. It is used to track changes in source code and helps developers manage different versions of a project.

मराठी Explanation:

Git हे Version Control System आहे. आपण project मध्ये केलेले changes track करण्यासाठी Git वापरतो.

उदाहरणार्थ, तू एखादा MERN project तयार करत आहेस. आज Login Module तयार केला आणि उद्या Payment Module तयार केला. Git वापरल्यामुळे कोणत्या दिवशी काय changes केले हे track करता येते.

Main Uses:

- Track code changes.
- Restore previous versions.
- Work with multiple developers.
- Manage different project versions.
- Compare code changes.

Q2. What is Version Control System (VCS)?

Interview Answer:

A Version Control System is a tool that records changes made to files over time. It allows developers to track, compare, and restore previous versions of their code.

Types of VCS:

1. Local Version Control System.
2. Centralized Version Control System.
3. Distributed Version Control System.

मराठी Explanation:

समजा तुझ्या project मध्ये Login functionality आधी व्यवस्थित काम करत होती. नवीन changes केल्यानंतर ती बंद पडली. Version Control मुळे आपण आधीच्या working version कडे परत जाऊ शकतो.

Q3. What is GitHub?

Interview Answer:

GitHub is a cloud-based platform used to host Git repositories. It helps developers collaborate, review code, manage projects, and share source code.

मराठी Explanation:

Git हे आपल्या computer वरसुद्धा काम करते. GitHub हे online platform आहे जिथे आपण Git repositories upload आणि manage करू शकतो.

Icona Logo Git di Those Icons – scarica in PNG, SVG

Git

Tracks changes in source code.

GitHub Vs. Bitbucket: Which Code Hosting Platform is Best? - Hatica

GitHub

Hosts repositories and supports collaboration.

GitHub Uses:

- Hosting repositories.
- Sharing projects.
- Team collaboration.
- Pull Requests and code reviews.
- Issue tracking.
- GitHub Actions for CI/CD.

Q4. What is the difference between Git and GitHub?

Git	**GitHub**
Version control system	Cloud-based hosting platform
Works locally	Works online

Tracks code changes	Hosts Git repositories
Developed by Linus Torvalds	Owned by Microsoft
Command-line tool and GUI support	Web interface and integrations

Interview Tip: Git आणि GitHub एकच नाहीत. Git हे tool आहे आणि GitHub हे Git repositories host करण्यासाठी platform आहे.

Q5. What is a repository in Git?

Interview Answer:

A repository is a storage location where project files and their version history are maintained.

Types:

1. Local Repository.
2. Remote Repository.

Example:

git init

हा command current folder मध्ये Git repository initialize करतो.

मराठी Explanation:

Repository म्हणजे project च्या files आणि त्यांचा history manage करणारी जागा.

2. Git Installation and Configuration

Q6. How do you install Git?

Interview Answer:

Git can be installed from its official website. After installation, we can verify it using the terminal.

Official Website: Download Git

git --version

Output:

git version 2.x.x

Q7. How do you configure username and email in Git?

Git मध्ये commit करणाऱ्या developer ची identity configure करण्यासाठी हे commands वापरतात.

git config --global user.name "Manoj Chougule"

git config --global user.email "your-email@example.com"

Check configuration:

git config --global --list

मराठी Explanation:

Git commit मध्ये कोणत्या developer ने changes केले याची माहिती username आणि email मधून मिळते.

Important: Git email आणि GitHub account email जुळवून ठेवणे उपयोगी ठरते.

Q8. What is the difference between global, local and system configuration?

Configuration	Meaning
--system	Applies to all users on the system
--global	Applies to the current operating system user
--local	Applies only to the current repository

Example:

git config --global user.name "Manoj"

git config --local user.name "Manoj Developer"

Local configuration current repository साठी global configuration override करू शकते.

3. Important Git Commands

Q9. What is git init?

Interview Answer:

The git init command initializes a new Git repository in the current directory.

git init

Use: नवीन project Git ने track करण्यासाठी.

Q10. What is git status?

The git status command shows the current state of the working directory and staging area.

git status

यातून कोणत्या files modified, untracked किंवा staged आहेत हे कळते.

Q11. What is git add?

The git add command moves changes from the working directory to the staging area.

git add index.html

All files:

git add .

मराठी Explanation:

File मध्ये changes केले की ते लगेच commit होत नाहीत. आधी staging area मध्ये add करावे लागतात.

Q12. What is git commit?

Interview Answer:

The git commit command saves the staged changes as a new snapshot in the repository history.

git commit -m "Added login functionality"

मराठी Explanation:

Commit म्हणजे staged changes चा saved snapshot. प्रत्येक meaningful feature किंवा fix नंतर commit करणे चांगली practice आहे.

Q13. What is git log?

The git log command displays the commit history of a repository.

git log

Short history:

git log --oneline

एका commit मध्ये कोणते changes झाले हे पाहण्यासाठी:

git show

Q14. What is git diff?

Interview Answer:

The git diff command shows the differences between file versions or different Git states.

git diff

Staged changes:

git diff --staged

Use: Commit करण्यापूर्वी नेमके कोणते changes झाले हे तपासण्यासाठी.

Q15. What is git rm?

git rm command removes a file from the working directory and stages the removal.

git rm old-file.js

फक्त Git tracking मधून file काढायची पण computer वर ठेवायची असेल:

git rm --cached config.js

Q16. What is git mv?

Git repository मध्ये file rename किंवा move करण्यासाठी वापरतात.

git mv old.js new.js

4. Git Workflow and Architecture

Q17. Explain the Git workflow.

Git workflow मध्ये मुख्यतः तीन महत्त्वाचे areas असतात.

EP206: Git Workflow: Essential Commands

blog.bytebytego.com

1. Working Directory

इथे आपण project files create किंवा modify करतो.

2. Staging Area

Commit करण्यासाठी निवडलेल्या changes इथे ठेवतो.

3. Local Repository

इथे commits आणि project history save होते.

4. Remote Repository

GitHub सारख्या remote platform वर repository ठेवली जाते.

Complete Workflow:

git init

git status

git add .

git commit -m "Initial commit"

git remote add origin YOUR_REPOSITORY_URL

git push -u origin main

Q18. What is the difference between git add, git commit and git push?

Command	Purpose
git add	Changes staging area मध्ये ठेवतो
git commit	Local repository मध्ये snapshot save करतो
git push	Local commits remote repository वर पाठवतो

लक्षात ठेव:

Working Directory

|

git add

↓

Staging Area

|

git commit

↓

Local Repository

|

git push

↓

Remote Repository (GitHub)

Q19. What is a remote repository?

Interview Answer:

A remote repository is a version of a project hosted on a server or platform such as GitHub. It allows developers to share and synchronize their code.

git remote -v

Remote repository add करणे:

git remote add origin https://github.com/username/project.git

origin हे remote repository चे default नाव असते. ते बदलू शकते.

Q20. What is git push?

The git push command uploads local commits to a remote repository.

git push origin main

पहिल्यांदा upstream सेट करताना:

git push -u origin main

Q21. What is git pull?

Interview Answer:

The git pull command fetches changes from a remote repository and integrates them into the current local branch.

git pull origin main

मराठी Explanation:

GitHub वर दुसऱ्या developer ने नवीन changes upload केले असतील तर ते आपल्या local project मध्ये आणण्यासाठी git pull वापरतो.

Q22. What is git fetch?

git fetch downloads new commits and references from a remote repository without integrating them into the current branch.

git fetch origin

Difference:

- git fetch: Changes download करतो; current branch लगेच merge होत नाही.
- git pull: Fetch करून changes integrate करतो; default behavior मध्ये merge होतो, पण configuration नुसार rebase सुद्धा होऊ शकतो.

5. Branching and Merging

Very Important for Team Interviews

Q23. What is a branch in Git?

Interview Answer:

A branch is an independent line of development in Git. It allows developers to work on features or fixes without directly changing the main branch.

मराठी Explanation:

समजा तुझ्या project मध्ये login, payment आणि dashboard features तयार करायचे आहेत. प्रत्येक feature साठी वेगळी branch तयार करू शकतो.

Git Process - NeoITO Playbook

playbook.neoito.com

Q24. How do you create a branch?

git branch feature-login

सर्व branches पाहण्यासाठी:

git branch

Q25. How do you switch branches?

git switch feature-login

जुना command:

git checkout feature-login

Q26. How do you create and switch to a branch in one command?

git switch -c feature-login

जुना पर्याय:

git checkout -b feature-login

Q27. What is git merge?

Interview Answer:

Git merge combines changes from one branch into another branch.

Example:

git switch main

git merge feature-login

मराठी Explanation:

Feature branch मधील changes main branch मध्ये आणण्यासाठी merge वापरतो.

Q28. What is a merge conflict?

Interview Answer:

A merge conflict occurs when Git cannot automatically combine changes from different branches, usually because conflicting changes were made to the same part of a file.

Example:

Branch A मध्ये:

const username = "Manoj";

Branch B मध्ये:

const username = "Rahul";

दोन्ही branches मधील same line वर वेगवेगळे changes असतील तर conflict येऊ शकतो.

Conflict file मध्ये असे दिसते:

<<<<<<< HEAD

const username = "Manoj";

=======

const username = "Rahul";

>>>>>>> feature-login

How to resolve?

1. Conflicted file open कर.
2. योग्य code निवड किंवा दोन्ही changes combine कर.
3. Conflict markers remove कर.
4. File test कर.
5. Changes stage कर.
6. Merge commit पूर्ण कर.

git add .

git commit -m "Resolved merge conflict"

Important: Merge conflict म्हणजे Git error नेहमीच असे नाही. तो developer ने resolve करायचा असलेला conflict असतो.

Q29. What is git rebase?

Interview Answer:

Git rebase moves or replays commits from one branch onto another base commit. It creates a cleaner, more linear project history in many workflows.

git switch feature-login

git rebase main

मराठी Explanation:

Feature branch चे commits main branch च्या latest commit नंतर पुन्हा apply केले जातात.

Merge vs Rebase:

Merge	**Rebase**
Branch histories combine करतो	Commits नवीन base वर replay करतो
Merge commit तयार होऊ शकतो	Linear history तयार होऊ शकते
Existing commits बदलत नाही	Replayed commits चे hashes बदलतात

Shared branches साठी सामान्यतः सुरक्षित	Published shared history rebase करताना काळजी घ्यावी

Interview Tip: Public/shared branch वर इतरांनी वापरलेले commits rebase करून rewrite करू नयेत, जोपर्यंत team ने स्पष्टपणे ठरवलेले नाही.

6. Undo and Restore Commands

Q30. What is git restore?

git restore command वापरून working directory मधील unstaged changes discard करू शकतो.

git restore index.js

Warning: या command मुळे त्या file मधील unstaged changes गमावू शकतो.

Q31. What is git reset?

Git reset command वापरून HEAD, staging area आणि काही परिस्थितींमध्ये working directory मागील state मध्ये आणता येते.

Types:

1. Soft Reset

git reset --soft HEAD~1

Last commit हटवतो, पण changes staged ठेवतो.

2. Mixed Reset

git reset --mixed HEAD~1

Last commit हटवतो आणि changes unstaged ठेवतो. हा default mode आहे.

3. Hard Reset

git reset --hard HEAD~1

Last commit हटवतो आणि tracked files मधील changes discard करतो.

Destructive Command

--hard काळजीपूर्वक वापर. Uncommitted changes कायमचे गमावले जाऊ शकतात.

Q32. What is git revert?

Interview Answer:

The git revert command creates a new commit that reverses the changes introduced by an earlier commit.

git revert COMMIT_ID

मराठी Explanation:

समजा चुकीचा commit GitHub वर push झाला आहे. जुना commit history मधून delete न करता त्याचे changes उलट करणारा नवीन commit तयार करण्यासाठी revert वापरतो.

Reset vs Revert:

Reset	**Revert**
HEAD/history बदलू शकतो	नवीन reverse commit तयार करतो
Local/private history साठी उपयोगी	Shared/published history साठी सुरक्षित पर्याय
Commit pointer मागे नेऊ शकतो	Existing commit history जतन करतो

Q33. What is git stash?

Interview Answer:

Git stash temporarily saves uncommitted changes so that we can work on another branch without committing those changes.

git stash

Stash पुन्हा apply करणे:

git stash apply

Stash apply करून remove करणे:

git stash pop

Stash list:

git stash list

Industry Example:

तू Payment Module वर काम करत आहेस. अचानक Login Module मध्ये urgent bug fix करायचा आहे. Payment चे incomplete changes stash करून दुसऱ्या branch वर काम करू शकतोस.

7. GitHub Fundamentals

Q34. How do you create a repository on GitHub?

Steps:

1. GitHub website open कर.
2. Login कर.
3. New Repository वर click कर.
4. Repository name enter कर.
5. Public किंवा Private select कर.
6. Create Repository वर click कर.

Official Website: GitHub

Q35. What is the difference between public and private repositories?

Public Repository	**Private Repository**
Anyone can view it	Only authorized users can access
Open-source projects साठी उपयुक्त	Company/private projects साठी उपयुक्त
Code publicly visible	Code restricted

Important: Public repository मध्ये passwords, API keys किंवा database credentials कधीही upload करू नयेत.

Q36. What is README.md?

Interview Answer:

README.md is a Markdown file that provides information about a project, including its purpose, features, installation steps, usage, and technologies.

Example:

```
# Project Name


A MERN Stack Application.


## Features

- User Authentication

- Dashboard
```

- CRUD Operations

Technologies

- React

- Node.js

- Express.js

- MongoDB

Installation

npm install

Run Project

npm run dev

Industry Use: Developer ला project समजण्यासाठी README महत्त्वाचा असतो.

Q37. What is .gitignore?

Interview Answer:

A .gitignore file specifies files and directories that Git should ignore and not track as untracked files.

Example:

node_modules/

.env

dist/

build/

*.log

.DS_Store

मराठी Explanation:

आपल्या project मधील काही files GitHub वर upload करायच्या नसतात. त्यासाठी .gitignore वापरतो.

MERN Project Example:

my-project/

│

├── client/

│ ├── node_modules/

│ └── .env

│

├── server/

│ ├── node_modules/

│ └── .env

│

└── .gitignore

Root .gitignore:

node_modules/

.env

client/.env

server/.env

dist/

Important: आधीच tracked झालेली file .gitignore मध्ये टाकल्याने ती आपोआप untrack होत नाही.

Q38. What is a Pull Request (PR)?

Interview Answer:

A Pull Request is a GitHub feature that allows developers to propose changes from one branch into another branch. Team members can review, discuss, and approve the changes before merging.

मराठी Explanation:

Developer ने feature branch वर काम पूर्ण केले की तो Pull Request तयार करतो. Team मधील दुसरे developers code review करतात. Approve झाल्यानंतर changes main branch मध्ये merge केले जातात.

Developing Software In a Team: Code Review – Intermediate Research Software Development

softwaresaved.github.io

Typical PR Workflow:

Create Feature Branch

↓

Write Code

↓

Commit Changes

↓

Push Branch to GitHub

↓

Create Pull Request

↓

Code Review

↓

Approve

↓

Merge into Main

Q39. What is a Fork in GitHub?

Interview Answer:

A fork is a personal copy of another user's repository on GitHub. It allows developers to make changes independently without modifying the original repository.

मराठी Explanation:

दुसऱ्याच्या public repository ची स्वतःच्या GitHub account मध्ये copy तयार करणे म्हणजे Fork.

Use:

- Open-source contribution.
- Experimenting with projects.
- Submitting changes to another repository.

Q40. What is the difference between Fork and Clone?

Fork	Clone
GitHub वर repository ची copy तयार करतो	Repository local computer वर download करतो
GitHub account मध्ये copy तयार होते	Local machine वर copy तयार होते
Web-based action	Git command

git clone https://github.com/username/project.git

Q41. What is GitHub Issues?

GitHub Issues is used to track bugs, tasks, feature requests, and discussions related to a project.

Example:

Issue #1: Login page is not working

Issue #2: Add password reset feature

Issue #3: Fix mobile responsive layout

Q42. What is GitHub Actions?

Interview Answer:

GitHub Actions is an automation platform that helps developers automate workflows such as testing, building, and deploying applications.

मराठी Explanation:

Code GitHub वर push केल्यानंतर automatically tests run करणे किंवा deployment करणे यासाठी GitHub Actions वापरतात.

Example Uses:

- Run tests on every push.
- Build React application.
- Deploy application.

- Check code quality.

8. Git Authentication and Security

Q43. What is SSH in Git?

Interview Answer:

SSH stands for Secure Shell. It provides a secure way to authenticate and communicate with remote Git repositories.

SSH URL example:

git@github.com:username/project.git

HTTPS URL example:

https://github.com/username/project.git

Q44. What is the difference between HTTPS and SSH?

HTTPS	SSH
Uses HTTPS URL	Uses SSH URL
Authentication through supported credentials/token or credential manager	Uses SSH key pair
Easy to set up	Requires SSH key setup
Useful for quick setup	Useful for regular development

Q45. What is a Personal Access Token (PAT)?

Interview Answer:

A Personal Access Token is a token used to authenticate with GitHub when password-based Git authentication is not supported.

मराठी Explanation:

GitHub ने Git operations साठी account password वापरण्याऐवजी supported authentication methods जसे PAT किंवा SSH वापरण्याची पद्धत दिली आहे.

Security Tip: PAT हा password सारखाच sensitive असतो. तो code मध्ये hardcode करू नये किंवा public repository मध्ये upload करू नये.

Q46. What is a GitHub Secret?

GitHub Secrets are encrypted values used to store sensitive information for GitHub Actions workflows.

Example:

MONGODB_URI

JWT_SECRET

DEPLOYMENT_TOKEN

मराठी Explanation:

CI/CD workflow मध्ये API keys किंवा deployment tokens वापरायचे असतील तर GitHub Secrets मध्ये ठेवू शकतो.

9. Advanced Git Concepts

Q47. What is HEAD in Git?

Interview Answer:

HEAD is a reference that points to the current commit or the currently checked-out branch.

git log --oneline

Current HEAD:

git show HEAD

मराठी Explanation:

सध्या आपण Git history मधील कोणत्या commit वर काम करत आहोत हे HEAD दर्शवतो.

Q48. What is a detached HEAD state?

A detached HEAD state occurs when HEAD points directly to a commit instead of a branch.

git checkout COMMIT_ID

मराठी Explanation:

आपण branch ऐवजी specific commit वर checkout केले तर detached HEAD state येऊ शकते.

नंतर त्या state मध्ये commits तयार केले तर ते एखाद्या branch ला attach केले नसल्यास हरवण्याची शक्यता असते. त्यामुळे काम सुरू ठेवायचे असल्यास नवीन branch तयार करावी.

git switch -c new-branch

Q49. What is cherry-pick?

Interview Answer:

Git cherry-pick applies a specific commit from one branch onto the current branch.

git cherry-pick COMMIT_ID

Industry Example:

Feature branch मधील एक bug fix commit main branch मध्ये आणायचा आहे, पण संपूर्ण feature branch merge करायची नाही. अशावेळी cherry-pick वापरता येतो.

Q50. What is git tag?

Git tags are used to mark specific commits, commonly for release versions.

git tag v1.0.0

Tags list:

git tag

Remote वर push:

git push origin v1.0.0

Industry Use: Project releases जसे v1.0.0, v2.0.0 mark करण्यासाठी.

Q51. What is git bisect?

Git bisect helps identify the commit that introduced a bug by performing a binary search through commit history.

git bisect start

git bisect bad

git bisect good COMMIT_ID

Git वेगवेगळे commits checkout करतो. प्रत्येक version good किंवा bad आहे हे सांगितल्यावर faulty commit शोधण्यास मदत होते.

Q52. What is git reflog?

git reflog records changes to HEAD and branch references in the local repository.

git reflog

Industry Use: चुकून reset किंवा branch change केल्यामुळे हरवलेला commit शोधण्यासाठी reflog उपयोगी ठरू शकतो.

10. Real Industry Git Workflow

Q53. How do developers work together using Git and GitHub?

Interview Answer:

Developers usually work on separate feature branches. They commit and push their changes to GitHub, create pull requests, review code, resolve conflicts, and merge approved changes into the main branch.

Practical Example:

समजा तू एका company मध्ये MERN Developer आहेस. तुला Login Module तयार करायचा आहे.

Step 1: Clone Project

git clone https://github.com/company/project.git

Step 2: Enter Project

cd project

Step 3: Pull Latest Changes

git pull origin main

Step 4: Create Feature Branch

git switch -c feature-login

Step 5: Develop Feature

Create Login Component

Create Login API

Add Validation

Connect Frontend and Backend

Step 6: Check Changes

git status

git diff

Step 7: Stage Changes

git add .

Step 8: Commit Changes

git commit -m "Added user login functionality"

Step 9: Push Branch

git push -u origin feature-login

Step 10: Create Pull Request

GitHub वर Pull Request तयार कर आणि team review ची वाट पाहा.

Step 11: Merge

Approval नंतर feature branch main मध्ये merge होते.

11. Git Commit Best Practices

Q54. What is a good commit message?

Interview Answer:

A good commit message clearly describes the purpose of the changes in a short and meaningful way.

Good Examples:

git commit -m "Added user authentication"

git commit -m "Fixed login validation error"

git commit -m "Updated dashboard UI"

git commit -m "Added product search functionality"

Bad Examples:

git commit -m "update"

git commit -m "changes"

git commit -m "final"

git commit -m "done"

Q55. What is Conventional Commits?

Conventional Commits is a specification for writing structured and meaningful commit messages.

Format:

type(scope): description

Examples:

feat(auth): add user login

fix(cart): correct total price calculation

docs(readme): update installation steps

style(header): improve navbar spacing

refactor(api): simplify user controller

test(auth): add login test cases

chore(deps): update dependencies

Type	Meaning
feat	New feature
fix	Bug fix
docs	Documentation
style	Formatting changes
refactor	Code restructuring
test	Testing changes

chore Maintenance tasks

12. Git Practical Problem-Solving Questions

Q56. How do you upload an existing local project to GitHub?

cd my-project

git init

git add .

git commit -m "Initial commit"

git branch -M main

git remote add origin https://github.com/username/project.git

git push -u origin main

Q57. How do you download an existing GitHub project?

git clone https://github.com/username/project.git

Q58. How do you check the current branch?

git branch --show-current

Q59. How do you rename a branch?

Current branch rename:

git branch -m new-branch-name

Q60. How do you delete a local branch?

git branch -d feature-login

Force delete:

git branch -D feature-login

Q61. How do you delete a remote branch?

git push origin --delete feature-login

Q62. What happens if you push without pulling the latest changes?

Interview Answer:

If the remote branch contains commits that are not present locally, Git may reject the push as a non-fast-forward update. We should fetch and integrate the remote changes before pushing again.

git pull --rebase origin main

git push origin main

मराठी Explanation:

समजा तू आणि दुसरा developer एकाच branch वर काम करत आहात. त्याने आधी changes push केले. तू जुने code वापरून push केलेस तर conflict किंवा non-fast-forward rejection येऊ शकतो.

13. Output / Command-Based Interview Questions

Q63. What does this command do?

git add .

Answer: Current directory मधील changes staging area मध्ये add करतो.

Q64. What does this command do?

git commit -am "Updated login"

Answer: Already tracked files मधील modified किंवा deleted changes stage करून commit करतो. नवीन untracked files automatically add होत नाहीत.

Q65. What does this command do?

git push -u origin main

Answer: Local main branch remote origin वर push करतो आणि upstream tracking set करतो.

Q66. What does this command do?

git pull --rebase origin main

Answer: Remote main branch मधील changes fetch करून current branch चे local commits त्या updated base वर replay करतो.

Q67. What does this command do?

git reset --soft HEAD~1

Answer: Last commit मागे नेतो; changes staging area मध्ये ठेवतो.

Q68. What does this command do?

git stash

Answer: Uncommitted changes तात्पुरते stash मध्ये save करतो.

14. Important Git vs GitHub Revision Table

Concept	Meaning
Git	Version Control System
GitHub	Git repository hosting platform
Repository	Project and version history
Commit	Saved snapshot
Branch	Independent development line
Merge	Combines branch changes
Rebase	Replays commits on another base
Clone	Copies remote repository locally
Fork	Copies another GitHub repository to your account
Pull	Fetches and integrates remote changes
Fetch	Downloads remote updates
Push	Uploads local commits
Stash	Temporarily saves uncommitted changes
Revert	Creates a commit that reverses earlier changes
Reset	Moves HEAD and optionally resets other states
Cherry-pick	Applies a specific commit
Pull Request	Proposes changes for review and merging

.gitignore	Excludes specified untracked files
README	Project documentation
HEAD	Current commit/branch reference

Part 7: React.js Interview Questions & Answers (Beginner to Advanced)

Beginner to Advanced

Industry Level

1. React.js Fundamentals

Q1. What is React.js?

Interview Answer:

React.js is a JavaScript library used to build user interfaces. It helps developers create reusable UI components. React is mainly used for building single-page applications.

मराठी स्पष्टीकरण:

React.js ही JavaScript ची Library आहे. तिचा वापर Website चा Frontend तयार करण्यासाठी केला जातो.

उदाहरणार्थ, E-commerce Website मध्ये:

- Navbar
- Product Card
- Login Form
- Shopping Cart
- Footer

हे वेगवेगळे Components म्हणून तयार करता येतात आणि पुन्हा वापरता येतात.

Q2. What are the main features of React?

Interview Answer:

The main features of React are:

1. Component-based architecture.
2. Virtual DOM.
3. Reusable components.
4. One-way data binding.
5. JSX.
6. Hooks.
7. Fast UI updates.

मराठी स्पष्टीकरण:

- Component-based: Website छोटे-छोटे भाग करून बनवता येते.
- Virtual DOM: UI मधील बदल efficiently update करण्यासाठी React ची पद्धत.
- Reusable Components: एकदा तयार केलेला component अनेक ठिकाणी वापरता येतो.
- One-way Data Flow: Data साधारणपणे Parent कडून Child कडे Props द्वारे जातो.
- JSX: JavaScript मध्ये HTML सारखी syntax लिहिता येते.
- Hooks: Functional Components मध्ये state आणि इतर React features वापरता येतात.

Q3. What is the difference between React and JavaScript?

Interview Answer:

JavaScript is a programming language, while React is a JavaScript library used to build user interfaces.

मराठी स्पष्टीकरण:

JavaScript ही Programming Language आहे. React ही JavaScript वापरून तयार केलेली UI Library आहे.

JavaScript	**React**
Programming language	UI library
General-purpose	Mainly UI development
Direct DOM manipulation possible	React manages UI updates
Can work without React	Requires JavaScript

Q4. What is a Single Page Application (SPA)?

Interview Answer:

A Single Page Application is a web application that loads a single HTML page and updates its content dynamically without reloading the entire page.

मराठी स्पष्टीकरण:

SPA मध्ये प्रत्येक वेळी नवीन Page पूर्णपणे Load होत नाही. फक्त आवश्यक Content बदलतो.

उदाहरण: Gmail मध्ये Inbox मधून Sent मध्ये गेल्यावर पूर्ण Website पुन्हा Load होत नाही.

Q5. What is the difference between SPA and MPA?

SPA	MPA
Single main HTML page	Multiple HTML pages
Content updates dynamically	New pages are loaded
Often smoother navigation	Navigation may reload pages
Example: Gmail	Traditional multi-page websites

Q6. What is JSX?

Interview Answer:

JSX stands for JavaScript XML. It allows us to write HTML-like syntax inside JavaScript. React uses JSX to describe the UI.

मराठी स्पष्टीकरण:

JSX मुळे JavaScript मध्ये HTML सारखा Code लिहिता येतो.

```
function App() {
 return (
  <div>
   <h1>Hello Manoj</h1>
   <p>Welcome to React</p>
  </div>
 );
}

export default App;
```

JSX चे महत्त्वाचे Rules:

1. एका Component ने एक Parent Element return करावा.
2. class ऐवजी className वापरावे.
3. JavaScript Expressions {} मध्ये लिहाव्यात.
4. प्रत्येक Tag properly close करावा.

5. Inline styles object स्वरूपात द्यावेत.

```
const name = "Manoj";


function App() {
  return <h1>Hello {name}</h1>;
}
```

Q7. What is a React Component?

Interview Answer:

A component is a reusable and independent part of the user interface. We can create components to divide a large application into smaller parts.

मराठी स्पष्टीकरण:

Component म्हणजे Website चा एक छोटा भाग.

उदाहरण:

```
function Navbar() {
  return <h2>My Website</h2>;
}


function Footer() {
  return <p>Copyright 2026</p>;
}


function App() {
  return (
    <>
      <Navbar />
      <h1>Home Page</h1>
      <Footer />
    </>
```

);

}

Types of Components:

- Functional Components — सध्याच्या React मध्ये मुख्यतः वापरले जातात.
- Class Components — जुना प्रकार; काही Legacy Projects मध्ये आढळतात.

Q8. What is the difference between Functional and Class Components?

Functional Component	**Class Component**
JavaScript function	JavaScript class
Uses Hooks	Uses lifecycle methods
Less code	More boilerplate
Common in modern React	Common in older React code

Q9. What is Virtual DOM?

Interview Answer:

The Virtual DOM is a lightweight representation of the real DOM. React uses it to calculate UI changes and update the necessary parts of the real DOM.

मराठी स्पष्टीकरण:

Real DOM म्हणजे Browser मधील actual HTML structure.

Virtual DOM म्हणजे React कडून UI चे ठेवलेले lightweight representation.

जेव्हा State बदलते:

1. React नवीन UI representation तयार करते.
2. मागील आणि नवीन representation ची तुलना करते.
3. आवश्यक DOM changes commit करते.

यामुळे संपूर्ण DOM manually पुन्हा तयार करण्याची गरज पडत नाही.

Q10. What is Reconciliation in React?

Interview Answer:

Reconciliation is the process React uses to compare the previous UI tree with the new UI tree and determine what changes are needed.

मराठी स्पष्टीकरण:

State किंवा Props बदलल्यानंतर आधीचे UI आणि नवीन UI यांची तुलना करून कोणते बदल करायचे ते React ठरवते. या प्रक्रियेला Reconciliation म्हणतात.

Q11. What is React Fiber?

Interview Answer:

React Fiber is the internal reconciliation architecture of React. It helps React manage rendering work and prioritize updates.

मराठी स्पष्टीकरण:

React Fiber हे React चे Internal Architecture आहे. मोठ्या UI मध्ये Rendering Work व्यवस्थित manage करण्यासाठी आणि गरजेनुसार कामाला Priority देण्यासाठी त्याचा उपयोग होतो.

Q12. What is the difference between Real DOM and Virtual DOM?

Real DOM	**Virtual DOM**
Browser DOM structure	React's UI representation
Directly represents the document	Lightweight representation
DOM updates can be costly	Helps calculate necessary updates
Managed by browser	Managed by React

2. Props, State and Data Handling

Q13. What are Props in React?

Interview Answer:

Props are used to pass data from a parent component to a child component. Props are read-only.

मराठी स्पष्टीकरण:

Props म्हणजे Parent Component कडून Child Component ला Data पाठवण्याची पद्धत.

```
function User({ name, age }) {
 return (
  <div>
```

```
    <h2>{name}</h2>
    <p>{age}</p>
   </div>
  );
}

function App() {
  return <User name="Manoj" age={22} />;
}
```

इथे:

- App हा Parent Component.
- User हा Child Component.
- name आणि age हे Props आहेत.

Q14. What is State in React?

Interview Answer:

State is a built-in React feature used to store data inside a component. When state changes, React schedules an update to render the component again.

मराठी स्पष्टीकरण:

State म्हणजे Component मधील बदलणारा Data.

उदाहरणार्थ:

- Counter Value
- Login Status
- Form Input
- Shopping Cart Items
- Toggle Button

```
import { useState } from "react";
```

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>{count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}

export default Counter;
```

समजून घे:

- count → Current State.
- setCount → State Update करण्यासाठी Function.
- useState(0) → Initial Value 0.

Q15. What is the difference between Props and State?

Props	State
Passed from parent	Managed by component
Read-only for receiving component	Updated using setter or React APIs
Used for communication	Used for changing data
Parent controls the passed value	Component manages its own state

Q16. Why should we not directly modify State?

Interview Answer:

We should not modify state directly because React may not detect the change correctly. We should use the state setter function to update state.

मराठी स्पष्टीकरण:

State Direct बदलल्यामुळे React ला Update योग्य प्रकारे समजू शकत नाही. म्हणून setState किंवा useState चा setter वापरतो.

❌ Wrong:

```
count = count + 1;
```

❌ Also wrong:

```
user.name = "Rahul";
```

✅ Correct:

```
setCount(prev => prev + 1);
```

Object State साठी:

```
setUser(prev => ({
  ...prev,
  name: "Rahul"
}));
```

Q17. What is State Lifting?

Interview Answer:

State lifting means moving state from a child component to its common parent component so that multiple child components can share the same data.

मराठी स्पष्टीकरण:

दोन Child Components ना एकच Data लागणार असेल तर तो State त्यांच्या Common Parent मध्ये ठेवतो. याला Lifting State Up म्हणतात.

Q18. What is Prop Drilling?

Interview Answer:

Prop drilling means passing data through multiple components just to reach a deeply nested child component.

मराठी स्पष्टीकरण:

Parent पासून एखाद्या खूप आतल्या Child पर्यंत Data पोहोचवण्यासाठी मध्ये असलेल्या अनेक Components मधून Props Pass करावे लागतात. याला Prop Drilling म्हणतात.

उपाय:

- Context API
- Redux Toolkit
- Zustand

Q19. What is the difference between Controlled and Uncontrolled Components?

Interview Answer:

A controlled component is managed by React state, while an uncontrolled component manages its value through the DOM, usually accessed using a ref.

मराठी स्पष्टीकरण:

Controlled Input मध्ये Input ची Value React State मध्ये ठेवली जाते.

Controlled Example:

```
import { useState } from "react";


function Login() {
 const [email, setEmail] = useState("");


 return (
  <input
   value={email}
   onChange={(e) => setEmail(e.target.value)}
  />
 );
}
```

Uncontrolled Example:

```
import { useRef } from "react";
```

```
function Login() {
 const emailRef = useRef(null);

 function handleSubmit() {
  console.log(emailRef.current.value);
 }

 return (
  <>
   <input ref={emailRef} />
   <button onClick={handleSubmit}>Submit</button>
  </>
 );
}
```

Q20. What is the difference between useState and normal variables?

Interview Answer:

A normal variable does not trigger a React re-render when its value changes. Updating state using useState schedules a component update.

मराठी स्पष्टीकरण:

Normal Variable बदलला तर React ला आपोआप UI Update करण्याची सूचना मिळत नाही. State बदलल्यावर React Update Schedule करते.

3. React Hooks

Q21. What are Hooks in React?

Interview Answer:

Hooks are special functions that allow functional components to use React features such as state, effects, context, and references.

मराठी स्पष्टीकरण:

Hooks मुळे Functional Component मध्ये State, Lifecycle-सारखे Effects, Context आणि इतर React Features वापरता येतात.

महत्त्वाचे Hooks:

Hook	Use
useState	State manage करणे
useEffect	Side effects handle करणे
useContext	Context मधील Data वापरणे
useRef	Reference जपणे
useMemo	Calculation result cache करणे
useCallback	Function reference cache करणे
useReducer	Complex State manage करणे
useId	Unique IDs तयार करणे
useTransition	Non-urgent updates manage करणे
useDeferredValue	Expensive UI updates defer करणे

Q22. What is useEffect?

Interview Answer:

useEffect is a React Hook used to synchronize a component with external systems, such as APIs, subscriptions, timers, and browser APIs.

मराठी स्पष्टीकरण:

useEffect चा उपयोग Component आणि बाहेरील Systems यांच्यात Synchronization ठेवण्यासाठी होतो.

उदाहरण:

- API Call
- Event Listener
- Timer

- WebSocket Subscription
- Document Title Update

```
import { useEffect, useState } from "react";

function Users() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    async function fetchUsers() {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );

      const data = await response.json();
      setUsers(data);
    }

    fetchUsers();
  }, []);

  return (
    <div>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}
```

Dependency Array चे प्रकार:

Syntax	Behavior
useEffect(() => {})	प्रत्येक committed render नंतर
useEffect(() => {}, [])	Mount झाल्यावर setup
useEffect(() => {}, [count])	count बदलल्यावर setup पुन्हा

टीप: Development मधील Strict Mode मध्ये काही Effects अतिरिक्त setup/cleanup cycle मधून जाऊ शकतात.

Q23. What is the Cleanup Function in useEffect?

Interview Answer:

A cleanup function is returned from useEffect to remove subscriptions, clear timers, or clean up resources before the effect runs again or the component unmounts.

मराठी स्पष्टीकरण:

Component बंद होताना किंवा Effect पुन्हा चालण्याआधी Timer, Event Listener, Subscription वगैरे Cleanup करणे आवश्यक असते.

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("Running...");
  }, 1000);

  return () => {
    clearInterval(timer);
  };
}, []);
```

Q24. What is useRef?

Interview Answer:

useRef is a Hook that stores a mutable value between renders. Updating a ref does not trigger a re-render. It is also used to access DOM elements.

मराठी स्पष्टीकरण:

useRef चा उपयोग दोन महत्त्वाच्या गोष्टींसाठी होतो:

1. DOM Element Access करणे.
2. Re-render न करता Value Store करणे.

```
import { useRef } from "react";


function Search() {
 const inputRef = useRef(null);


 function focusInput() {
  inputRef.current.focus();
 }


 return (
  <>
   <input ref={inputRef} />
   <button onClick={focusInput}>Focus</button>
  </>
 );
}
```

Q25. What is the difference between useState and useRef?

useState	**useRef**
Stores component state	Stores mutable reference
Updating it schedules a render	Updating it does not trigger render
Used for UI data	Used for DOM refs and persistent values

Q26. What is useMemo?

Interview Answer:

useMemo is a React Hook that caches the result of a calculation between renders until its dependencies change.

मराठी स्पष्टीकरण:

मोठे Calculation प्रत्येक Render वेळी पुन्हा करावे लागू नये म्हणून useMemo वापरतो.

```
import { useMemo, useState } from "react";


function ProductList({ products }) {
 const [search, setSearch] = useState("");


 const filteredProducts = useMemo(() => {
  return products.filter(product =>
   product.name.toLowerCase().includes(search.toLowerCase())
  );
 }, [products, search]);


 return (
  <>
   <input
    value={search}
    onChange={e => setSearch(e.target.value)}
   />


   {filteredProducts.map(product => (
    <p key={product.id}>{product.name}</p>
   ))}
  </>
 );
}
```

Q27. What is useCallback?

Interview Answer:

useCallback is a React Hook that caches a function reference between renders until its dependencies change.

मराठी स्पष्टीकरण:

Component पुन्हा Render झाला तरी Function ची Reference अनावश्यकपणे बदलू नये यासाठी useCallback वापरू शकतो.

```
import { useCallback, useState } from "react";


function Parent() {
 const [count, setCount] = useState(0);


 const handleClick = useCallback(() => {
  console.log("Button clicked");
 }, []);


 return (
  <>
   <button onClick={() => setCount(c => c + 1)}>
    {count}
   </button>


   <Child onClick={handleClick} />
  </>
 );
}


function Child({ onClick }) {
```

```
  return <button onClick={onClick}>Child Button</button>;
}
```

Q28. Difference between useMemo and useCallback?

useMemo	useCallback
Caches calculation result	Caches function reference
Returns calculated value	Returns function
Used for expensive calculations	Useful when function identity matters

Q29. What is useContext?

Interview Answer:

useContext is a React Hook that allows a component to read data from a Context without passing props through every intermediate component.

मराठी स्पष्टीकरण:

अनेक Components मध्ये एकच Data लागतो, तेव्हा Context API वापरून तो Data Share करता येतो.

उदाहरण: Theme, Authenticated User, Language.

```
import { createContext, useContext } from "react";


const ThemeContext = createContext("light");


function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Dashboard />
    </ThemeContext.Provider>
  );
}
```

```
function Dashboard() {
  const theme = useContext(ThemeContext);

  return <h1>Current Theme: {theme}</h1>;
}
```

Q30. What is useReducer?

Interview Answer:

useReducer is a React Hook used to manage state with a reducer function. It is useful when state logic becomes complex or involves multiple related actions.

मराठी स्पष्टीकरण:

State मध्ये अनेक प्रकारचे Actions असतील, तर useReducer उपयोगी ठरतो.

```
import { useReducer } from "react";

function reducer(state, action) {
  switch (action.type) {
    case "INCREMENT":
      return { count: state.count + 1 };

    case "DECREMENT":
      return { count: state.count - 1 };

    case "RESET":
      return { count: 0 };

    default:
      return state;
  }
}
```

```
function Counter() {
  const [state, dispatch] = useReducer(reducer, {
    count: 0
  });

  return (
    <>
      <h2>{state.count}</h2>

      <button onClick={() => dispatch({ type: "INCREMENT" })}>
        +
      </button>

      <button onClick={() => dispatch({ type: "DECREMENT" })}>
        -
      </button>

      <button onClick={() => dispatch({ type: "RESET" })}>
        Reset
      </button>
    </>
  );
}
```

Q31. What are the Rules of Hooks?

Interview Answer:

There are two main rules:

1. Call Hooks only at the top level of React function components or custom Hooks.

2. Do not call Hooks inside loops, conditions, or nested functions.

मराठी स्पष्टीकरण:

Hooks नेहमी Component च्या Top Level वर वापरावेत. if, for, किंवा Nested Function मध्ये Hooks Call करू नयेत.

Q32. What is a Custom Hook?

Interview Answer:

A custom Hook is a reusable JavaScript function that uses React Hooks to share stateful logic between components.

मराठी स्पष्टीकरण:

एकच Logic अनेक Components मध्ये वापरायचे असेल तर ते Custom Hook मध्ये ठेवता येते.

उदाहरण: API Fetching, Form Handling, Authentication.

```
import { useEffect, useState } from "react";


function useUsers() {
 const [users, setUsers] = useState([]);
 const [loading, setLoading] = useState(true);


 useEffect(() => {
  async function getUsers() {
   try {
    const response = await fetch(
     "https://jsonplaceholder.typicode.com/users"
    );


    const data = await response.json();
    setUsers(data);
   } finally {
    setLoading(false);
```

```
    }
  }

  getUsers();
 }, []);

 return { users, loading };
}
```

वापरणे:

```
function UserList() {
 const { users, loading } = useUsers();

 if (loading) return <p>Loading...</p>;

 return users.map(user => (
  <p key={user.id}>{user.name}</p>
 ));
}
```

4. Rendering, Lists and Events

Q33. What is Conditional Rendering?

Interview Answer:

Conditional rendering means displaying different UI elements based on a condition.

मराठी स्पष्टीकरण:

Condition नुसार वेगवेगळे UI दाखवणे म्हणजे Conditional Rendering.

```
function Dashboard({ isLoggedIn }) {
 return (
  <div>
```

```
    {isLoggedIn ? (
      <h1>Welcome Back</h1>
    ) : (
      <h1>Please Login</h1>
    )}
   </div>
  );
}
```

Q34. How do you render a list in React?

Interview Answer:

We use the map() method to render a list of elements in React.

```
const users = [
 { id: 1, name: "Manoj" },
 { id: 2, name: "Rahul" },
 { id: 3, name: "Amit" }
];

function App() {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

Q35. Why are Keys important in React?

Interview Answer:

Keys help React identify list items between renders. They help React understand which items were added, removed, or changed.

मराठी स्पष्टीकरण:

List मधील प्रत्येक Item ला Unique Key द्यावी लागते. त्यामुळे React ला कोणता Item बदलला हे ओळखता येते.

```
{users.map(user => (
  <UserCard key={user.id} user={user} />
))}
```

Important: शक्यतो Array Index ला Key म्हणून वापरू नये, विशेषतः List मध्ये Add, Delete किंवा Reorder होत असेल तर.

Q36. What is Event Handling in React?

Interview Answer:

Event handling means responding to user actions such as clicks, typing, form submission, and mouse events.

```
function App() {
  function handleClick() {
    alert("Button clicked!");
  }

  return (
    <button onClick={handleClick}>
      Click Me
    </button>
  );
}
```

React मध्ये Event Names camelCase असतात, उदाहरणार्थ onClick, onChange, onSubmit.

Q37. What is the difference between onClick= and onClick=?

Interview Answer:

onClick={handleClick} passes the function reference, while onClick={handleClick()} calls the function immediately during rendering.

```
// Correct
<button onClick={handleClick}>Click</button>

// Calls immediately — usually incorrect
<button onClick={handleClick()}>Click</button>
```

Arguments पाठवायचे असल्यास:

```
<button onClick={() => handleClick(10)}>
  Click
</button>
```

Q38. What is Synthetic Event?

Interview Answer:

A Synthetic Event is React's cross-browser wrapper around native browser events. It provides a consistent event interface.

मराठी स्पष्टीकरण:

Browser नुसार Native Events मध्ये काही फरक असू शकतात. React Event Handling साठी Consistent Interface देते, त्याला Synthetic Event म्हणतात.

Q39. What is React Fragment?

Interview Answer:

A Fragment lets us group multiple elements without adding an extra DOM element.

```
function App() {
  return (
    <>
      <h1>Hello</h1>
      <p>Welcome</p>
    </>
  );
```

}

Q40. What is dangerouslySetInnerHTML?

Interview Answer:

dangerouslySetInnerHTML is a React property used to insert raw HTML into a DOM element. It must be used carefully because untrusted HTML can cause XSS attacks.

मराठी स्पष्टीकरण:

काही वेळा HTML String थेट Render करायची असते. त्यासाठी याचा उपयोग होतो. पण User कडून आलेला Unsanitized HTML थेट Render केल्यास XSS Attack होऊ शकतो.

5. React Router

Q41. What is React Router?

Interview Answer:

React Router is a routing library used to navigate between different views in a React application without reloading the entire page.

मराठी स्पष्टीकरण:

React Application मध्ये वेगवेगळे URLs आणि Pages Manage करण्यासाठी React Router वापरतो.

उदाहरण:

- / → Home
- /about → About
- /login → Login
- /dashboard → Dashboard

Q42. How do you implement routing in React?

```
import {
  BrowserRouter,
  Routes,
  Route,
  Link
} from "react-router-dom";
```

```
function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Q43. Difference between Link and useNavigate?

Link	useNavigate

Used for navigation through UI links	Used for programmatic navigation
User clicks a link	Code triggers navigation
Example: Navbar	Example: After successful login

```
import { useNavigate } from "react-router-dom";

function Login() {
 const navigate = useNavigate();

 function handleLogin() {
  // Login logic
  navigate("/dashboard");
 }

 return <button onClick={handleLogin}>Login</button>;
}
```

Q44. What are Dynamic Routes?

Interview Answer:

Dynamic routes are routes that contain parameters, allowing us to display different data based on the URL.

```
<Route path="/users/:id" element={<UserDetails />} />
```

उदाहरण: /users/101

```
import { useParams } from "react-router-dom";

function UserDetails() {
 const { id } = useParams();

 return <h1>User ID: {id}</h1>;
}
```

Q45. What are Protected Routes?

Interview Answer:

Protected routes are routes that allow access only to authorized users.

मराठी स्पष्टीकरण:

Dashboard किंवा Admin Panel फक्त Login केलेल्या किंवा योग्य Permission असलेल्या Users ना दाखवण्यासाठी Protected Routes वापरतो.

```
import { Navigate, Outlet } from "react-router-dom";


function ProtectedRoute({ isAuthenticated }) {
  return isAuthenticated ? (
    <Outlet />
  ) : (
    <Navigate to="/login" replace />
  );
}
```

टीप: Frontend Protected Route हा फक्त UI-Level Access Control आहे. Backend API वरही Authentication आणि Authorization तपासणे आवश्यक आहे.

6. API Integration and Forms

Q46. How do you call an API in React?

Interview Answer:

We can use fetch() or Axios to make HTTP requests from a React application. We commonly handle API calls using useEffect or event handlers.

```
import { useEffect, useState } from "react";


function Products() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
```

```
useEffect(() => {
  async function fetchProducts() {
    try {
      const response = await fetch(
        "https://fakestoreapi.com/products"
      );

      if (!response.ok) {
        throw new Error("Failed to fetch products");
      }

      const data = await response.json();
      setProducts(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  fetchProducts();
}, []);

if (loading) return <p>Loading...</p>;
if (error) return <p>{error}</p>;

return products.map(product => (
```

```
  <h3 key={product.id}>{product.title}</h3>
 ));
}
```

Q47. What is Axios?

Interview Answer:

Axios is a promise-based HTTP client used to make API requests from browsers and Node.js applications.

मराठी स्पष्टीकरण:

Axios चा वापर Backend API ला GET, POST, PUT, PATCH, DELETE Requests पाठवण्यासाठी होतो.

```
import axios from "axios";

const response = await axios.get("/api/products");

console.log(response.data);
```

Q48. Difference between Fetch and Axios?

Fetch	Axios
Built into modern browsers	External package
Requires checking response.ok	Rejects on non-2xx HTTP status by default
JSON parsing is explicit	JSON response is generally parsed automatically
Supports AbortController	Supports AbortController

Q49. What is CORS?

Interview Answer:

CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether a web page can access resources from a different origin.

मराठी स्पष्टीकरण:

समजा React Frontend localhost:5173 वर आहे आणि Express Backend localhost:5000 वर आहे. दोन्हींचे Origin वेगळे असल्यामुळे Browser ला Backend कडून CORS Permission लागते.

Express Example:

```
import cors from "cors";


app.use(cors({
  origin: "http://localhost:5173",
  credentials: true
}));
```

Q50. How do you handle Forms in React?

Interview Answer:

We can handle forms using controlled components, where input values are stored in React state and updated through event handlers.

```
import { useState } from "react";


function Register() {
  const [form, setForm] = useState({
    name: "",
    email: ""
  });


  function handleChange(e) {
    setForm({
      ...form,
      [e.target.name]: e.target.value
    });
  }


  function handleSubmit(e) {
    e.preventDefault();
```

```
  console.log(form);
 }

 return (
  <form onSubmit={handleSubmit}>
   <input
    name="name"
    placeholder="Enter name"
    value={form.name}
    onChange={handleChange}
   />

   <input
    name="email"
    placeholder="Enter email"
    value={form.email}
    onChange={handleChange}
   />

   <button type="submit">Register</button>
  </form>
 );
}
```

Q51. What is e.preventDefault()?

Interview Answer:

preventDefault() stops the browser's default behavior for an event.

मराठी स्पष्टीकरण:

Form Submit केल्यावर Browser Page Reload करतो. तो Default Behavior थांबवण्यासाठी:

```
function handleSubmit(e) {
  e.preventDefault();
}
```

Q52. What is Debouncing?

Interview Answer:

Debouncing delays the execution of a function until a specified time has passed without another call.

मराठी स्पष्टीकरण:

Search Input मध्ये प्रत्येक अक्षर टाइप झाल्यावर API Call करण्याऐवजी User टाइप करणे थांबवल्यानंतर ठरावीक वेळाने API Call करणे म्हणजे Debouncing.

```
import { useEffect, useState } from "react";

function Search() {
  const [search, setSearch] = useState("");

  useEffect(() => {
    const timer = setTimeout(() => {
      if (search.trim()) {
        console.log("API call:", search);
      }
    }, 500);

    return () => clearTimeout(timer);
  }, [search]);

  return (
    <input
      value={search}
```

```
    onChange={e => setSearch(e.target.value)}
    placeholder="Search products..."
   />
  );
}
```

Q53. What is Throttling?

Interview Answer:

Throttling limits how frequently a function can execute within a specific time interval.

मराठी स्पष्टीकरण:

एखादे Function सतत Trigger होत असेल तर ठरावीक Interval मध्ये ते जास्तीत जास्त एकदाच Execute होईल अशी व्यवस्था म्हणजे Throttling.

उदाहरण:

- Scroll Events
- Resize Events
- Mouse Movement

Debouncing vs Throttling:

Debouncing	**Throttling**
Executes after calls stop for a delay	Executes at most once per interval
Search suggestions	Scroll handling
Waits for inactivity	Controls execution frequency

7. State Management

Q54. What is Context API?

Interview Answer:

Context API is a built-in React feature used to share data across components without passing props through every level.

मराठी स्पष्टीकरण:

Prop Drilling कमी करण्यासाठी Context API वापरतो.

उदाहरण:

- Theme Management
- Authentication
- Language Settings

Q55. What is Redux?

Interview Answer:

Redux is a state management library used to manage application-wide state in a predictable way.

मराठी स्पष्टीकरण:

मोठ्या Application मध्ये अनेक Components ना एकच State लागतो. तो Centralized पद्धतीने Manage करण्यासाठी Redux वापरतात.

Q56. What is Redux Toolkit?

Interview Answer:

Redux Toolkit is the recommended way to write Redux logic. It reduces boilerplate code and provides utilities such as configureStore and createSlice.

मराठी स्पष्टीकरण:

Traditional Redux मध्ये जास्त Boilerplate Code लिहावा लागत होता. Redux Toolkit मुळे Code कमी आणि व्यवस्थित होतो.

```
import { createSlice, configureStore } from "@reduxjs/toolkit";


const counterSlice = createSlice({
 name: "counter",


 initialState: {
  value: 0
 },


 reducers: {
```

```
    increment: state => {
      state.value += 1;
    },

    decrement: state => {
      state.value -= 1;
    }
  }
});

export const { increment, decrement } =
  counterSlice.actions;

const store = configureStore({
  reducer: {
    counter: counterSlice.reducer
  }
});

export default store;
```

टीप: Redux Toolkit मध्ये Reducer च्या आत असे दिसणारे Mutation Syntax वापरले तरी Immer मुळे प्रत्यक्ष State Immutable पद्धतीने Update होते.

Q57. What is Zustand?

Interview Answer:

Zustand is a lightweight state management library for React. It provides a simple way to create and access shared state.

मराठी स्पष्टीकरण:

Zustand हा Redux च्या तुलनेत सोपा आणि Lightweight State Management पर्याय आहे.

```
import { create } from "zustand";

const useCounterStore = create((set) => ({
 count: 0,

 increment: () =>
  set(state => ({
   count: state.count + 1
  }))
}));

function Counter() {
 const { count, increment } = useCounterStore();

 return (
  <>
   <h2>{count}</h2>
   <button onClick={increment}>Increase</button>
  </>
 );
}
```

Q58. Difference between Context API, Redux Toolkit and Zustand?

Context API	Redux Toolkit	Zustand
Built into React	External library	External library
Simple shared data	Complex application state	Simple global state
Provider-based	Central store	Store-based
No built-in async middleware	Middleware ecosystem	Middleware available

Interview Tip: प्रत्येक Application ला Redux लागतोच असे नाही. State च्या Complexity नुसार योग्य पर्याय निवडतो.

8. Performance Optimization

Q59. What is React.memo?

Interview Answer:

React.memo is a higher-order component that can skip re-rendering a component when its props are unchanged.

```
import React from "react";


const User = React.memo(function User({ name }) {
  console.log("User rendered");


  return <h2>{name}</h2>;
});
```

मराठी स्पष्टीकरण:

Parent Render झाल्यावर Child Component चे Props बदलले नसतील तर React.memo मुळे अनावश्यक Re-render टाळता येऊ शकतो.

टीप: React Compiler वापरल्यास काही Manual Memoization आवश्यक राहत नाही. React.memo वापरण्यापूर्वी Performance गरज तपासावी.

Q60. What is Lazy Loading?

Interview Answer:

Lazy loading means loading a component or resource only when it is needed.

```
import { lazy, Suspense } from "react";


const Dashboard = lazy(() => import("./Dashboard"));


function App() {
  return (
```

```
  <Suspense fallback={<p>Loading...</p>}>
    <Dashboard />
  </Suspense>
 );
}
```

मराठी स्पष्टीकरण:

संपूर्ण Application चा JavaScript Code एकाच वेळी Download करण्याऐवजी गरजेनुसार Component Load करणे म्हणजे Lazy Loading.

Q61. What is Code Splitting?

Interview Answer:

Code splitting divides the application bundle into smaller chunks that can be loaded when required.

मराठी स्पष्टीकरण:

मोठ्या JavaScript Bundle ला छोटे भाग करून आवश्यकतेनुसार Load करणे म्हणजे Code Splitting.

Q62. What is Bundle Size?

Interview Answer:

Bundle size is the size of the JavaScript and other assets generated for an application.

मराठी स्पष्टीकरण:

Application Build झाल्यानंतर Browser ला Download करावा लागणारा JavaScript Code आणि इतर Assets यांचा आकार म्हणजे Bundle Size.

Bundle कमी करण्याचे मार्ग:

- Code Splitting
- Lazy Loading
- Unused Packages काढणे
- Tree Shaking
- Image Optimization

Q63. What is React StrictMode?

Interview Answer:

StrictMode is a development tool that helps identify potential problems in a React application. It can run certain functions and effect setup/cleanup cycles extra times in development.

मराठी स्पष्टीकरण:

StrictMode मुळे Development दरम्यान काही संभाव्य चुका लक्षात येतात. यामुळे काही Effects दोनदा चालल्यासारखे दिसू शकतात. Production मध्ये हा Development-only अतिरिक्त व्यवहार नसतो.

Q64. What is an Error Boundary?

Interview Answer:

An Error Boundary is a React component that catches rendering errors in its child component tree and displays fallback UI.

मराठी स्पष्टीकरण:

एखाद्या Child Component मध्ये Rendering Error आल्यास संपूर्ण Application Crash होण्याऐवजी Fallback UI दाखवण्यासाठी Error Boundary वापरतो.

टीप: Traditional Error Boundaries Class Components वापरून तयार केले जातात. Event Handler किंवा सामान्य asynchronous callback मधील Errors ते आपोआप पकडत नाहीत.

9. Advanced React Concepts

Q65. What is Higher-Order Component (HOC)?

Interview Answer:

A Higher-Order Component is a function that takes a component and returns a new enhanced component.

मराठी स्पष्टीकरण:

एका Component ला दुसऱ्या Function च्या मदतीने Extra Functionality देणे म्हणजे HOC.

```
function withLoading(Component) {
 return function EnhancedComponent({ loading, ...props }) {
  if (loading) {
   return <p>Loading...</p>;
  }
```

```
  return <Component {...props} />;
 };
}
```

Q66. What is Code Reusability in React?

Interview Answer:

Code reusability means creating components or custom Hooks that can be used in multiple parts of an application.

मराठी स्पष्टीकरण:

एकच Navbar, Button, Modal किंवा Form अनेक ठिकाणी वापरता येणे म्हणजे Code Reusability.

Q67. What is Portal in React?

Interview Answer:

A Portal allows us to render a component into a DOM node outside its parent component's DOM hierarchy.

मराठी स्पष्टीकरण:

सामान्य Component Structure च्या बाहेर DOM मध्ये UI Render करायचा असेल तर Portal वापरतो.

उदाहरण: Modal, Dialog, Tooltip.

```
import { createPortal } from "react-dom";


function Modal() {
 return createPortal(
  <div className="modal">
   <h2>Hello</h2>
  </div>,
  document.getElementById("modal-root")
 );
}
```

Q68. What is Hydration?

Interview Answer:

Hydration is the process of attaching React behavior and event handlers to HTML that was already rendered on the server.

मराठी स्पष्टीकरण:

Server कडून HTML आधीच तयार होऊन Browser मध्ये आलेला असतो. React त्या HTML ला Interactive बनवण्यासाठी Event Handlers आणि React Behavior जोडते, याला Hydration म्हणतात.

Q69. What is Server-Side Rendering (SSR)?

Interview Answer:

Server-Side Rendering means generating HTML on the server and sending it to the browser.

मराठी स्पष्टीकरण:

Browser मध्ये JavaScript ने UI तयार करण्याऐवजी Server वर HTML तयार करून Browser ला पाठवणे म्हणजे SSR.

Q70. What is Client-Side Rendering (CSR)?

Interview Answer:

Client-Side Rendering means the browser uses JavaScript to render the user interface.

मराठी स्पष्टीकरण:

Browser JavaScript चालवून UI तयार करतो. साधारण Vite + React SPA मध्ये CSR वापरले जाते.

Q71. Difference between CSR and SSR?

CSR	**SSR**
UI rendered in browser	HTML rendered on server
Initial content may depend on JS loading	Server can send ready HTML
Common in Vite React SPA	Common in frameworks such as Next.js
Browser does more rendering work	Server handles HTML generation

Q72. What is React Server Component?

Interview Answer:

React Server Components are components that render on the server and can access server-side resources without sending their component code to the client.

मराठी स्पष्टीकरण:

Server Components Server वर Execute होतात. त्यांचा Component Code Browser ला पाठवण्याची गरज नसते. ते Server-side Data Access साठी उपयोगी असतात.

टीप: Server Components आणि SSR एकच गोष्ट नाहीत. Vite वापरून तयार केलेल्या साध्या React SPA मध्ये Server Components आपोआप मिळत नाहीत.

Q73. What is useTransition?

Interview Answer:

useTransition is a React Hook that lets us mark some state updates as non-urgent transitions.

मराठी स्पष्टीकरण:

मोठ्या UI Update मुळे User Interface अडकू नये म्हणून काही Updates कमी Priority ने Process करता येतात.

```
import { useState, useTransition } from "react";


function Search() {
 const [query, setQuery] = useState("");
 const [isPending, startTransition] = useTransition();


 function handleChange(e) {
  const value = e.target.value;


  startTransition(() => {
   setQuery(value);
  });
 }
```

```
 return (
   <>
    <input onChange={handleChange} />
    {isPending && <p>Updating...</p>}
    <p>{query}</p>
   </>
 );
}
```

10. Real-World MERN Project Questions

हे प्रश्न तुझ्या MERN Developer Interview साठी विशेष महत्त्वाचे आहेत.

Q74. How does React communicate with Node.js and Express?

Interview Answer:

React communicates with the Express backend using HTTP requests. The backend processes the request, interacts with MongoDB if needed, and sends a response back to React.

मराठी स्पष्टीकरण:

MERN Application मध्ये Flow असा असतो:

React.js Frontend

User Interface / API Request

Express.js / Node.js

Routes / Middleware / Business Logic

MongoDB

Database Operations

JSON Response

Back to React UI

Q75. How do you manage Authentication in React?

Interview Answer:

I manage authentication by sending login credentials to the backend. The backend verifies the user and creates an authentication session or token. The frontend uses the authentication response to manage the user's login state.

मराठी स्पष्टीकरण:

Login Process:

1. User Email आणि Password टाकतो.
2. React Backend ला Request पाठवतो.
3. Backend User Verify करतो.
4. Backend Session किंवा JWT तयार करतो.
5. Authentication Cookie वापरली असल्यास Browser ती Store करतो.
6. React Login State Update करतो.
7. Protected UI दाखवतो.

Important: JWT किंवा Session Validation Backend वरच करणे आवश्यक आहे. फक्त React State वर विश्वास ठेवू नये.

Q76. What is JWT?

Interview Answer:

JWT stands for JSON Web Token. It is a compact token format used to securely transmit claims between parties.

मराठी स्पष्टीकरण:

JWT वापरून User Authentication आणि काही माहिती Token स्वरूपात पाठवता येते.

JWT चे तीन भाग:

1. Header
2. Payload
3. Signature

उदाहरणार्थ:

header.payload.signature

JWT मध्ये Password किंवा Sensitive Data ठेवू नये. Payload सामान्यतः Encode केलेला असतो, Encrypt केलेला असेलच असे नाही.

Q77. What is the difference between Local Storage, Session Storage and Cookies?

Local Storage	Session Storage	Cookies
Persists until cleared	Usually lasts for the tab session	Depends on expiry settings
Larger storage capacity	Smaller storage capacity	Small storage capacity
Not automatically sent to server	Not automatically sent to server	Can be sent with HTTP requests
Accessible through JavaScript	Accessible through JavaScript	Can be HttpOnly

Industry Tip: Authentication साठी HttpOnly, Secure आणि योग्य SameSite settings असलेली Cookie उपयोगी ठरते. CSRF Risk सुद्धा विचारात घ्यावा.

Q78. What is Axios Interceptor?

Interview Answer:

Axios interceptors allow us to run code before a request is sent or after a response is received.

मराठी स्पष्टीकरण:

प्रत्येक API Request मध्ये Common Headers किंवा प्रत्येक Response मध्ये Common Error Handling करायचे असेल तर Interceptor वापरतो.

```
import axios from "axios";


const api = axios.create({
  baseURL: "http://localhost:5000/api"
});


api.interceptors.request.use(
  config => {
```

```
  console.log("Request sent");
  return config;
 },
 error => Promise.reject(error)
);

export default api;
```

Q79. How do you upload images in React?

Interview Answer:

We can upload images using a file input and FormData. The image is sent to the backend, which can store it in a cloud storage service such as Cloudinary.

```
async function uploadImage(file) {
 const formData = new FormData();

 formData.append("image", file);

 const response = await fetch("/api/upload", {
  method: "POST",
  body: formData
 });

 return response.json();
}
```

मराठी स्पष्टीकरण:

React मधून Image निवडतो → FormData मध्ये टाकतो → Backend ला पाठवतो → Backend Cloudinary वर Upload करतो → Image URL परत देतो.

Q80. How do you handle Loading, Error and Empty States?

Interview Answer:

I maintain separate loading, error, and data states. This helps users understand whether data is loading, failed to load, or is not available.

मराठी स्पष्टीकरण:

Professional UI मध्ये फक्त Data दाखवणे पुरेसे नाही.

- Loading State — Data येत आहे.
- Error State — API Request Failed.
- Empty State — Data उपलब्ध नाही.
- Success State — Data योग्य प्रकारे आला.

11. React Output-Based Interview Questions

हे प्रश्न Interview मध्ये तुझी Concept Clarity तपासतात.

Q81. What will be the output?

```
function App() {
 const [count, setCount] = useState(0);

 function handleClick() {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);
 }

 return (
  <>
   <h1>{count}</h1>
   <button onClick={handleClick}>Click</button>
  </>
 );
}
```

Answer: 1

कारण: एका Event Handler मधील Updates Batch होतात. प्रत्येक setCount(count + 1) मध्ये त्याच Render मधील count ची Value 0 असते.

Q82. How can you increase count by 3?

```
function handleClick() {
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
}
```

Answer: 3

कारण: Functional Updater ला मागील Update ची नवीनतम State Value मिळते.

Q83. What will be the output?

```
function App() {
  const [count, setCount] = useState(0);

  console.log("Rendered");

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

Answer: Initial Render वेळी आणि प्रत्येक वास्तविक State Update नंतर Rendered Console मध्ये दिसेल. StrictMode Development मध्ये अतिरिक्त Render दिसू शकतो.

Q84. What is wrong with this code?

```
function App() {
  const [user, setUser] = useState({
```

```
  name: "Manoj"
 });

 function updateUser() {
  user.name = "Rahul";
  setUser(user);
 }

 return <button onClick={updateUser}>Update</button>;
}
```

Answer: State Object Direct Mutate केले आहे आणि त्याच Object Reference ने State Update करण्याचा प्रयत्न केला आहे.

Correct:

```
setUser(prev => ({
 ...prev,
 name: "Rahul"
}));
```

Q85. What is the output?

```
const items = ["A", "B", "C"];

function App() {
 return (
  <>
   {items.map(item => (
    <h2>{item}</h2>
   ))}
  </>
 );
```

}

Answer: A, B, C हे तीन Items दिसतील, पण प्रत्येक List Item ला key न दिल्यामुळे Development मध्ये Warning येईल.

Correct:

{items.map(item => (

<h2 key={item}>{item}</h2>

))}

12. React Coding Interview Questions

खालील Coding Questions स्वतः करून बघ. हे Frontend Developer Interview साठी उपयोगी आहेत.

No.	Coding Question	Concept
1	Create a Counter App	useState
2	Create a Show/Hide Password	State, Conditional Rendering
3	Build a Todo App	CRUD, State
4	Create a Search Filter	Array Filter
5	Fetch Users from API	useEffect, Fetch
6	Create a Login Form	Controlled Inputs
7	Create a Dark/Light Theme	Context API
8	Create Pagination	State, Array Methods
9	Create a Modal	Conditional Rendering, Portal
10	Create a Shopping Cart	State Management
11	Create Protected Routes	React Router
12	Create Debounced Search	useEffect, Timer
13	Create Image Preview	useRef, FileReader
14	Create Infinite Scroll	API, Scroll Handling
15	Create Custom useFetch Hook	Custom Hooks

13. Common React Interview Mistakes

Mistake	Correct Approach
Directly changing State	Use state setter
Missing list keys	Give stable unique keys
Calling Hooks inside conditions	Call at top level
Using handleClick() in onClick	Pass function reference
Missing Effect cleanup	Clean timers/listeners
Unnecessary useEffect	Derive values during render when possible
Ignoring API errors	Handle loading/error states
Storing sensitive tokens in localStorage without considering risks	Use an appropriate secure authentication design
Using Index as key for changing lists	Use stable IDs
Trusting frontend authorization	Validate permissions on backend

14. React Quick Revision

Topic	Remember
React	JavaScript UI Library
JSX	HTML-like JavaScript Syntax
Component	Reusable UI Part
Props	Parent to Child Data
State	Component Data
Virtual DOM	UI Representation
Reconciliation	Compare UI Trees
useState	Manage State
useEffect	Synchronize External Systems

useRef	Store Reference
useMemo	Cache Calculation
useCallback	Cache Function Reference
useContext	Share Context Data
useReducer	Manage Complex State
React.memo	Skip Some Unnecessary Renders
React Router	Navigation
Redux Toolkit	Global State Management
Zustand	Lightweight State Management
Lazy Loading	Load When Needed
Error Boundary	Handle Rendering Errors
Portal	Render Outside Parent DOM
Hydration	Attach React to Server HTML

15. React.js Interview Practice Test

आता स्वतःची तयारी तपास. प्रत्येक प्रश्नाला उत्तर निवडल्यावर लगेच योग्य उत्तर आणि Explanation दिसेल.

React.js MCQ Test

0/10 answered

Q1. What is React.js?

Programming Language

JavaScript UI Library

Database

Backend Framework

Q2. Which Hook is used to manage component state?

useEffect

useRef

useState

useMemo

Q3. Why are keys used in React lists?

For styling

To identify list items

For API calls

For routing

Q4. Which Hook is used to access a DOM element?

useMemo

useRef

useContext

useReducer

Q5. What does useEffect handle?

Only CSS

External system synchronization

Database creation

Routing only

Q6. What is Prop Drilling?

Passing props through many component levels

Deleting props

Changing CSS

Creating routes

Q7. Which method is commonly used to render lists?

filter() only

map()

push()

sort()

Q8. Which tool is used for React navigation?

Mongoose

React Router

Express

MongoDB

Q9. What does React.memo help with?

Database queries

Skipping some re-renders

Creating APIs

Changing routes

Q10. What is the purpose of a cleanup function in useEffect?

Add CSS

Clean timers and subscriptions

Create state

Render lists

Submit Answers

पुढील अभ्यासासाठी महत्त्वाचा सल्ला

React मध्ये फक्त Definitions पाठ करून उपयोग नाही. खालील 5 गोष्टी Code करून समजून घे:

1. useState आणि useEffect वापरून CRUD Application.
2. React Router वापरून Login आणि Protected Dashboard.
3. Axios वापरून Express API Integration.
4. Redux Toolkit किंवा Zustand वापरून Shopping Cart.
5. Custom Hooks वापरून Reusable API Logic.

Interview मध्ये सर्वात महत्त्वाचे: एखादा Hook काय आहे एवढेच सांगू नकोस; तो कोणत्या परिस्थितीत वापरतोस आणि तुझ्या Project मध्ये कुठे वापरला आहे हेही सांगता आले पाहिजे.

Redux Toolkit Interview Questions & Answers (Beginner to Advanced)

Redux Toolkit (RTK) हा React Developer Interview मधील महत्त्वाचा Topic आहे. विशेषतः MERN Stack Developer साठी Redux Toolkit, Redux Flow, Store, Slice, Reducers, Actions, Middleware आणि Async Thunks समजणे आवश्यक आहे.

आपण प्रत्येक Topic सोप्या English Interview Answer + मराठी Explanation + Practical Code + Industry Examples सह शिकणार आहोत.

Beginner to Advanced

Practical + Interview

1. Redux Fundamentals

Q1. What is Redux?

Interview Answer:

Redux is a state management library used to manage application-wide state in a predictable way. It allows different components to access shared data from a central store.

मराठी स्पष्टीकरण:

Redux ही State Management Library आहे. मोठ्या React Application मध्ये अनेक Components ना एकच Data लागतो. तो Data प्रत्येक Component ला Props मधून पाठवण्याऐवजी एका Central Store मध्ये ठेवता येतो.

उदाहरणार्थ, E-commerce Website मध्ये:

- Logged-in User
- Shopping Cart
- Wishlist
- User Preferences
- Notifications

हा Data अनेक Components मध्ये वापरायचा असेल तर Redux उपयोगी पडतो.

Q2. Why do we use Redux?

Interview Answer:

We use Redux to manage shared application state, avoid prop drilling, and keep state updates predictable and organized.

मराठी स्पष्टीकरण:

समजा E-commerce Application आहे.

- Navbar मध्ये Cart Count दाखवायचा आहे.
- Product Page वर Add to Cart Button आहे.
- Cart Page वर सर्व Products दाखवायचे आहेत.

- Checkout Page वर Total Amount पाहिजे.

या सर्व Components मध्ये Cart Data लागतो. Redux Store मध्ये Cart Data ठेवल्यास सर्व Components तो Access करू शकतात.

Q3. What is Redux Toolkit?

Interview Answer:

Redux Toolkit is the official recommended way to write Redux logic. It simplifies Redux development by reducing boilerplate code and providing utilities such as configureStore, createSlice, and createAsyncThunk.

मराठी स्पष्टीकरण:

Traditional Redux मध्ये Store, Actions, Action Types आणि Reducers साठी जास्त Code लिहावा लागत होता.

Redux Toolkit मुळे हे सर्व सोप्या पद्धतीने करता येते.

मुख्य Features:

1. configureStore() — Redux Store तयार करणे.
2. createSlice() — State आणि Reducers तयार करणे.
3. createAsyncThunk() — Asynchronous API Operations करणे.
4. createListenerMiddleware() — Actions किंवा State Changes वर Side Effects चालवणे.
5. Immer Integration — Reducers मध्ये सोप्या Mutation सारख्या Syntax ने Immutable Updates करणे.

Q4. What is the difference between Redux and Redux Toolkit?

Redux (Traditional)	**Redux Toolkit**
More boilerplate code	Less boilerplate code
Manually configure store	configureStore
Separate action creators	createSlice generates them
Immutable updates need more code	Immer is included
Middleware setup is manual	Common middleware is configured automatically

Interview Tip: Redux Toolkit हा Redux चाच अधिक सोपा आणि Recommended वापर आहे. तो Redux चा वेगळा पर्याय नाही.

Q5. What is State in Redux?

Interview Answer:

State in Redux is the data stored in the Redux Store. It represents the current state of the application.

मराठी स्पष्टीकरण:

Redux Store मध्ये ठेवलेल्या संपूर्ण Application Data ला Redux State म्हणतात.

```
const initialState = {
  user: null,
  cart: [],
  isLoggedIn: false
};
```

2. Redux Toolkit Architecture

Q6. What are the main parts of Redux Toolkit?

Interview Answer:

The main parts of Redux Toolkit are:

1. Store
2. Slice
3. Actions
4. Reducers
5. Dispatch
6. Selectors
7. Middleware

मराठी स्पष्टीकरण:

Concept	**काम**
Store	Application State ठेवतो

Slice	एका Feature चा State आणि Reducers ठेवतो
Action	कोणता बदल करायचा ते सांगतो
Reducer	State कसा बदलायचा ते ठरवतो
Dispatch	Action Store कडे पाठवतो
Selector	Store मधून Data वाचतो
Middleware	Dispatch आणि Reducer यांच्या दरम्यान अतिरिक्त Logic चालवतो

Q7. What is Redux Store?

Interview Answer:

The Redux Store is a central object that holds the application state and provides methods to access and update it.

मराठी स्पष्टीकरण:

Store म्हणजे Application चा Central Data Storage.

```
import { configureStore } from "@reduxjs/toolkit";


const store = configureStore({
  reducer: {}
});


export default store;
```

Q8. What is a Slice in Redux Toolkit?

Interview Answer:

A slice is a collection of Redux reducer logic and actions for a specific feature of the application.

मराठी स्पष्टीकरण:

Slice म्हणजे एका Feature संबंधित State, Reducers आणि Actions यांचा Group.

उदाहरण:

- userSlice
- cartSlice
- authSlice
- productSlice

```
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
 name: "counter",

 initialState: {
  count: 0
 },

 reducers: {
  increment: (state) => {
   state.count += 1;
  },

  decrement: (state) => {
   state.count -= 1;
  }
 }
});

export const { increment, decrement } =
 counterSlice.actions;

export default counterSlice.reducer;
```

Q9. What is createSlice()?

Interview Answer:

createSlice() is a Redux Toolkit function that creates a slice containing the initial state, reducer functions, and automatically generated action creators.

मराठी स्पष्टीकरण:

createSlice() वापरल्यावर आपल्याला वेगळे Action Types आणि Action Creators लिहावे लागत नाहीत.

यामध्ये तीन महत्त्वाच्या गोष्टी असतात:

```
createSlice({
 name: "counter",
 initialState: { count: 0 },
 reducers: {
  increment: (state) => {
   state.count += 1;
  }
 }
});
```

- name — Slice चे नाव.
- initialState — सुरुवातीचा State.
- reducers — State Update करणारी Functions.

Q10. What is a Reducer?

Interview Answer:

A reducer is a function that receives the current state and an action, then determines the next state.

मराठी स्पष्टीकरण:

Reducer म्हणजे State कसा Update करायचा हे ठरवणारे Function.

उदाहरण:

```
increment: (state) => {
  state.count += 1;
}
```

इथे increment Reducer आहे.

Q11. What is an Action?

Interview Answer:

An action is an object that describes what happened in the application. It usually contains a type and may contain a payload.

मराठी स्पष्टीकरण:

Action म्हणजे Application मध्ये कोणती घटना घडली ते सांगणारा Object.

उदाहरण:

```
{
  type: "counter/increment"
}
```

किंवा Payload सोबत:

```
{
  type: "cart/addToCart",
  payload: {
    id: 1,
    name: "Laptop"
  }
}
```

Q12. What is Dispatch in Redux?

Interview Answer:

Dispatch is a function used to send an action to the Redux Store so that the corresponding reducer can process it.

मराठी स्पष्टीकरण:

Reducer थेट Call न करता Action Store कडे पाठवण्यासाठी dispatch() वापरतो.

```
import { useDispatch } from "react-redux";
import { increment } from "./counterSlice";

function Counter() {
  const dispatch = useDispatch();

  return (
    <button onClick={() => dispatch(increment())}>
      Increase
    </button>
  );
}
```

Q13. What is Payload in Redux?

Interview Answer:

Payload is the data sent along with an action to provide the reducer with the information needed to update the state.

मराठी स्पष्टीकरण:

Action सोबत पाठवलेल्या Data ला Payload म्हणतात.

```
reducers: {
  addUser: (state, action) => {
    state.users.push(action.payload);
  }
}
```

इथे action.payload मध्ये नवीन User चा Data येतो.

Q14. What is Selector in Redux?

Interview Answer:

A selector is a function used to read specific data from the Redux Store.

मराठी स्पष्टीकरण:

Store मधून Data वाचण्यासाठी Selector वापरतो.

```
import { useSelector } from "react-redux";


function Counter() {
  const count = useSelector(
    (state) => state.counter.count
  );


  return <h2>{count}</h2>;
}
```

3. Redux Toolkit Installation and Setup

Q15. How do you install Redux Toolkit?

Interview Answer:

We install Redux Toolkit and React Redux using npm.

```
npm install @reduxjs/toolkit react-redux
```

- @reduxjs/toolkit — Redux logic आणि Store Management.
- react-redux — React Components आणि Redux Store यांना Connect करते.

Q16. How do you configure Redux Toolkit in React?

Industry मध्ये साधारणपणे Feature-based Folder Structure वापरतात.

```
src
app
store.js
features
counter
counterSlice.js
App.jsx
main.jsx
```

Step 1: Create Slice

src/features/counter/counterSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
 name: "counter",

 initialState: {
  count: 0
 },

 reducers: {
  increment: (state) => {
   state.count += 1;
  },

  decrement: (state) => {
   state.count -= 1;
  },

  reset: (state) => {
   state.count = 0;
  }
 }
});

export const { increment, decrement, reset } =
 counterSlice.actions;
```

```
export default counterSlice.reducer;
```

Step 2: Create Store

src/app/store.js

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../features/counter/counterSlice";

const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});

export default store;
```

Step 3: Provide Store to React

src/main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";

import App from "./App";
import store from "./app/store";

ReactDOM.createRoot(
  document.getElementById("root")
).render(
  <Provider store={store}>
    <App />
```

```
  </Provider>
);
```

Step 4: Use Redux State in Component

src/App.jsx

```
import { useSelector, useDispatch } from "react-redux";

import {
  increment,
  decrement,
  reset
} from "./features/counter/counterSlice";

function App() {
  const count = useSelector(
    (state) => state.counter.count
  );

  const dispatch = useDispatch();

  return (
    <div>
      <h1>Counter: {count}</h1>

      <button onClick={() => dispatch(increment())}>
        +
      </button>

      <button onClick={() => dispatch(decrement())}>
```

```
      -
    </button>

    <button onClick={() => dispatch(reset())}>
      Reset
    </button>
   </div>
  );
}

export default App;
```

Q17. What is Provider in Redux?

Interview Answer:

Provider is a component from React Redux that makes the Redux Store available to the React component tree.

मराठी स्पष्टीकरण:

Provider मुळे त्याच्या आत असलेल्या सर्व Components ना Redux Store Access करता येतो.

```
<Provider store={store}>
  <App />
</Provider>
```

Q18. What is useSelector()?

Interview Answer:

useSelector is a React Redux Hook used to read data from the Redux Store.

```
const user = useSelector((state) => state.auth.user);
```

मराठी स्पष्टीकरण:

Store मधील State मधून आवश्यक Data मिळवण्यासाठी useSelector() वापरतो.

Q19. What is useDispatch()?

Interview Answer:

useDispatch is a React Redux Hook that returns the dispatch function used to send actions to the Redux Store.

const dispatch = useDispatch();

dispatch(increment());

मराठी स्पष्टीकरण:

Redux Action Dispatch करण्यासाठी useDispatch() वापरतो.

Q20. Difference between useSelector() and useDispatch()?

useSelector	**useDispatch**
Reads data from Store	Sends actions to Store
Accesses current State	Updates State through Actions
Returns selected data	Returns dispatch function

4. Redux Data Flow

Q21. Explain Redux Data Flow.

Interview Answer:

Redux follows a predictable unidirectional data flow. A component dispatches an action, the reducer calculates the next state, the store updates, and subscribed components receive the updated state.

मराठी स्पष्टीकरण:

Redux चा Data Flow असा असतो:

React Component

User clicks button

Dispatch Action

dispatch(increment())

Reducer

Calculates the next state

Redux Store

Stores updated state

Component Re-render

UI displays updated data

लक्षात ठेव:

Component → Dispatch → Action → Reducer → Store → UI Update

Q22. Is Redux State Immutable?

Interview Answer:

Redux follows immutable state update principles. Redux Toolkit uses Immer internally, allowing us to write mutation-like code while producing immutable updates.

मराठी स्पष्टीकरण:

Redux मध्ये State Direct बदलण्याऐवजी नवीन State तयार होणे अपेक्षित असते.

Traditional Redux मध्ये Spread Operator वापरावा लागत असे.

Redux Toolkit मध्ये:

```
increment: (state) => {
  state.count += 1;
}
```

असे लिहू शकतो कारण Immer हे Immutable Update तयार करते.

महत्त्वाचे: हे Mutation सारखे Syntax फक्त Redux Toolkit च्या Immer-enabled Reducers मध्ये सुरक्षित आहे. सामान्य React State मध्ये असे Direct Mutation करू नये.

5. Async Redux and API Calls

Q23. What is Asynchronous Action in Redux?

Interview Answer:

An asynchronous action performs operations that take time, such as API requests. Redux Toolkit provides createAsyncThunk to handle common asynchronous workflows.

मराठी स्पष्टीकरण:

API Call, Server Request किंवा Database मधून Data आणण्यासाठी वेळ लागतो. अशा Operations ना Asynchronous Operations म्हणतात.

उदाहरण:

- Login API
- Fetch Products
- Register User
- Delete Product

Q24. What is createAsyncThunk()?

Interview Answer:

createAsyncThunk is a Redux Toolkit function used to create asynchronous action creators. It automatically generates pending, fulfilled, and rejected action types.

मराठी स्पष्टीकरण:

API Call साठी तीन States असतात:

State	अर्थ
Pending	API Request सुरू आहे
Fulfilled	API Request Successful
Rejected	API Request Failed

createAsyncThunk() या तीन Action Types आपोआप तयार करते.

Q25. How do you fetch API data using Redux Toolkit?

हा अत्यंत महत्त्वाचा Practical Interview Question आहे.

Step 1: Create Async Thunk and Slice

src/features/product/productSlice.js

```
import {
  createSlice,
  createAsyncThunk
} from "@reduxjs/toolkit";


export const fetchProducts = createAsyncThunk(
  "products/fetchProducts",
```

```
  async (_, thunkAPI) => {
    try {
      const response = await fetch(
        "https://fakestoreapi.com/products",
        {
          signal: thunkAPI.signal
        }
      );

      if (!response.ok) {
        throw new Error("Failed to fetch products");
      }

      return await response.json();

    } catch (error) {
      return thunkAPI.rejectWithValue(
        error.message || "Something went wrong"
      );
    }
  }
);

const productSlice = createSlice({
  name: "products",

  initialState: {
    products: [],
```

```
  loading: false,
  error: null
 },

 reducers: {},

 extraReducers: (builder) => {
  builder
   .addCase(fetchProducts.pending, (state) => {
    state.loading = true;
    state.error = null;
   })

   .addCase(fetchProducts.fulfilled, (state, action) => {
    state.loading = false;
    state.products = action.payload;
   })

   .addCase(fetchProducts.rejected, (state, action) => {
    state.loading = false;
    state.error = action.payload || action.error.message;
   });
 }
});

export default productSlice.reducer;
```

Step 2: Add Reducer to Store

src/app/store.js

```
import { configureStore } from "@reduxjs/toolkit";

import productReducer from "../features/product/productSlice";

const store = configureStore({
  reducer: {
    products: productReducer
  }
});

export default store;
```

Step 3: Use API Data in Component

```
import { useEffect } from "react";
import { useSelector, useDispatch } from "react-redux";

import { fetchProducts } from "./features/product/productSlice";

function Products() {
  const dispatch = useDispatch();

  const { products, loading, error } = useSelector(
    (state) => state.products
  );

  useEffect(() => {
    const promise = dispatch(fetchProducts());

    return () => {
```

```
    promise.abort();
  };
 }, [dispatch]);

 if (loading) return <h2>Loading...</h2>;

 if (error) return <h2>{error}</h2>;

 return (
  <div>
   {products.map(product => (
    <div key={product.id}>
     <h3>{product.title}</h3>
     <p>${product.price}</p>
    </div>
   ))}
  </div>
 );
}

export default Products;
```

समजून घे:

1. Component मध्ये dispatch(fetchProducts()) Call होतो.
2. pending Action Dispatch होते.
3. API Request चालते.
4. Success झाल्यास fulfilled Action Dispatch होते.
5. Failure झाल्यास rejected Action Dispatch होते.

6. Reducer State Update करतो.
7. useSelector() नवीन State वाचतो.
8. UI Update होते.

Q26. What is extraReducers?

Interview Answer:

extraReducers allows a slice to respond to actions generated outside its own reducers, including createAsyncThunk lifecycle actions.

मराठी स्पष्टीकरण:

reducers मध्ये आपण स्वतःचे Actions तयार करतो. extraReducers वापरून दुसऱ्या Slice किंवा createAsyncThunk() ने तयार केलेल्या Actions वर प्रतिक्रिया देऊ शकतो.

```
extraReducers: (builder) => {
 builder
  .addCase(fetchProducts.pending, (state) => {
   state.loading = true;
  });
}
```

Q27. Difference between reducers and extraReducers?

reducers	**extraReducers**
Defines slice's own actions	Handles external actions
Generates action creators	Does not generate action creators
Used for normal state updates	Used for thunk and shared actions

Q28. What is rejectWithValue?

Interview Answer:

rejectWithValue is used inside createAsyncThunk to return a custom error payload when an asynchronous operation fails.

मराठी स्पष्टीकरण:

Backend कडून आलेला Custom Error Message Redux च्या rejected Action मध्ये पाठवण्यासाठी वापरतो.

```
return thunkAPI.rejectWithValue(
  "Invalid email or password"
);
```

नंतर:

```
.addCase(loginUser.rejected, (state, action) => {
  state.error = action.payload;
});
```

Q29. What is Redux Thunk?

Interview Answer:

Redux Thunk is middleware that allows us to write functions for asynchronous operations and access dispatch and getState.

मराठी स्पष्टीकरण:

Redux मध्ये सामान्य Actions सोबत Asynchronous Logic हाताळण्यासाठी Thunk Middleware वापरले जाते.

Redux Toolkit च्या configureStore() मध्ये Thunk Middleware Default ने समाविष्ट असते.

Q30. What is the difference between createAsyncThunk and RTK Query?

Interview Answer:

createAsyncThunk is useful for writing custom asynchronous workflows, while RTK Query is a data-fetching and caching solution designed to manage API requests and server data.

createAsyncThunk	**RTK Query**
Manually manage API state	Automatically manages request states
More control over custom workflows	Built-in caching
Define pending/fulfilled/rejected logic	Provides query and mutation APIs
Useful for custom async operations	Useful for server data fetching

6. RTK Query

Q31. What is RTK Query?

Interview Answer:

RTK Query is a data-fetching and caching tool included with Redux Toolkit. It simplifies API requests, caching, loading states, and cache invalidation.

मराठी स्पष्टीकरण:

RTK Query वापरून API Integration चे काम सोपे होते.

Features:

- API Requests
- Loading State
- Error State
- Data Caching
- Automatic Cache Updates
- Query आणि Mutation

Q32. What are Query and Mutation?

Interview Answer:

A query is used to fetch data from a server, while a mutation is used to create, update, or delete server data.

मराठी स्पष्टीकरण:

- Query — Data मिळवणे.
- Mutation — Data Create, Update किंवा Delete करणे.

उदाहरण:

Operation	**Type**
Get Products	Query
Get User Profile	Query
Add Product	Mutation
Update Product	Mutation

Delete Product Mutation

Q33. How do you implement RTK Query?

Step 1: Create API Service

src/services/productApi.js

```
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";

export const productApi = createApi({
 reducerPath: "productApi",

 baseQuery: fetchBaseQuery({
  baseUrl: "https://fakestoreapi.com/"
 }),

 endpoints: (builder) => ({
  getProducts: builder.query({
   query: () => "products"
  }),

  getProductById: builder.query({
   query: (id) => `products/${id}`
  })
 })
});

export const {
 useGetProductsQuery,
 useGetProductByIdQuery
} = productApi;
```

Step 2: Configure Store

```
import { configureStore } from "@reduxjs/toolkit";
import { productApi } from "../services/productApi";

export const store = configureStore({
  reducer: {
    [productApi.reducerPath]: productApi.reducer
  },

  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(productApi.middleware)
});
```

Step 3: Use in Component

```
import { useGetProductsQuery } from "./services/productApi";

function Products() {
  const {
    data,
    error,
    isLoading
  } = useGetProductsQuery();

  if (isLoading) return <p>Loading...</p>;

  if (error) return <p>Error loading products</p>;

  return (
    <div>
```

```
    {data?.map(product => (
      <h3 key={product.id}>
        {product.title}
      </h3>
    ))}
  </div>
 );
}
```

7. Advanced Redux Toolkit Concepts

Q34. What is Middleware in Redux?

Interview Answer:

Middleware is a function that intercepts dispatched actions before they reach the reducer. It can be used for logging, asynchronous operations, and other side effects.

मराठी स्पष्टीकरण:

Action Dispatch झाल्यानंतर Reducer पर्यंत पोहोचण्याआधी Middleware मध्ये अतिरिक्त Logic Execute करता येते.

उदाहरण:

- Logging
- API Calls
- Error Monitoring
- Analytics

Q35. What is configureStore()?

Interview Answer:

configureStore() is a Redux Toolkit function used to create the Redux Store with good default settings, including useful middleware and development checks.

```
const store = configureStore({
  reducer: {
    auth: authReducer,
```

```
  cart: cartReducer
 }
});
```

Q36. What is createEntityAdapter?

Interview Answer:

createEntityAdapter is a Redux Toolkit utility that helps manage normalized collections of data using a standard structure and generated reducer functions and selectors.

मराठी स्पष्टीकरण:

मोठ्या Array मध्ये अनेक Users किंवा Products Manage करताना Data id नुसार व्यवस्थित Store करण्यासाठी वापरता येतो.

उदाहरण:

```
import {
 createEntityAdapter,
 createSlice
} from "@reduxjs/toolkit";

const usersAdapter = createEntityAdapter();

const usersSlice = createSlice({
 name: "users",

 initialState: usersAdapter.getInitialState(),

 reducers: {
  setUsers: usersAdapter.setAll,
  addUser: usersAdapter.addOne,
  removeUser: usersAdapter.removeOne
 }
```

});

Q37. What is Normalization in Redux?

Interview Answer:

Normalization means storing relational data in a structured format using IDs instead of deeply nested objects.

मराठी स्पष्टीकरण:

Data Nested स्वरूपात ठेवण्याऐवजी ID वापरून वेगवेगळ्या Objects मध्ये ठेवणे म्हणजे Normalization.

```
const state = {
 users: {
  1: { id: 1, name: "Manoj" },
  2: { id: 2, name: "Rahul" }
 },

 allIds: [1, 2]
};
```

यामुळे मोठा Data Update आणि Access करणे सोपे होते.

Q38. What is createSelector?

Interview Answer:

createSelector is a function from Reselect that creates memoized selectors. It recalculates the result only when its input values change.

मराठी स्पष्टीकरण:

Store मधील Data वर Filter किंवा Calculation करायचे असेल आणि प्रत्येक वेळी तेच Calculation पुन्हा करायचे नसेल तर Memoized Selector वापरू शकतो.

```
import { createSelector } from "@reduxjs/toolkit";

const selectProducts = (state) => state.products.items;
```

```
export const selectAvailableProducts = createSelector(
 [selectProducts],
 (products) =>
  products.filter(product => product.stock > 0)
);
```

Q39. What is Serializability in Redux?

Interview Answer:

Serializability means that Redux state and actions should generally contain values that can be safely serialized, such as strings, numbers, arrays, and plain objects.

मराठी स्पष्टीकरण:

Redux Store मध्ये साधारणपणे JSON सारखा साधा Data ठेवावा.

उदाहरण:

- String
- Number
- Boolean
- Array
- Plain Object

शक्यतो Function, Promise, DOM Element, Class Instance यांसारखा Non-Serializable Data Store मध्ये ठेवू नये.

टीप: Date सारख्या Values साठी ISO String वापरणे अनेकदा सोयीचे असते.

Q40. What is Redux DevTools?

Interview Answer:

Redux DevTools is a browser debugging tool that helps developers inspect actions, state changes, and the history of Redux state updates.

मराठी स्पष्टीकरण:

Redux DevTools वापरून कोणता Action Dispatch झाला, State कसा बदलला आणि कोणती Error आली हे पाहता येते.

8. Real-World Industry Questions

Q41. Where do you use Redux Toolkit in your projects?

Interview Answer:

I use Redux Toolkit to manage shared application state. For example, in an e-commerce application, I can manage cart items, product filters, and user-related state using Redux Toolkit.

मराठी स्पष्टीकरण:

उदाहरणार्थ, E-commerce Project मध्ये:

- Cart Slice
- Auth Slice
- Wishlist Slice
- Product Slice

वेगवेगळे Slices तयार करून प्रत्येक Feature चा Data स्वतंत्रपणे Manage करू शकतो.

Q42. How do you manage Cart using Redux Toolkit?

Interview Answer:

I create a cart slice with reducers for adding items, removing items, updating quantities, and clearing the cart.

Practical Example:

```
import { createSlice } from "@reduxjs/toolkit";


const cartSlice = createSlice({
 name: "cart",


 initialState: {
  items: []
 },


 reducers: {
  addToCart: (state, action) => {
```

```
  const product = action.payload;

  const existingItem = state.items.find(
    item => item.id === product.id
  );

  if (existingItem) {
    existingItem.quantity += 1;
  } else {
    state.items.push({
      ...product,
      quantity: 1
    });
  }
},

removeFromCart: (state, action) => {
  state.items = state.items.filter(
    item => item.id !== action.payload
  );
},

updateQuantity: (state, action) => {
  const { id, quantity } = action.payload;

  const item = state.items.find(
    item => item.id === id
  );
```

```
    if (item && quantity > 0) {
      item.quantity = quantity;
    }
   },

   clearCart: (state) => {
    state.items = [];
   }
 }
});

export const {
 addToCart,
 removeFromCart,
 updateQuantity,
 clearCart
} = cartSlice.actions;

export default cartSlice.reducer;
```

मराठी स्पष्टीकरण:

- Product पहिल्यांदा Add झाला तर नवीन Item तयार होतो.
- तोच Product पुन्हा Add झाला तर Quantity वाढते.
- Remove केल्यावर Product Array मधून काढला जातो.
- Quantity Update करता येते.
- Clear Cart केल्यावर सर्व Items काढले जातात.

Q43. How do you calculate the total cart price?

```
const totalPrice = useSelector((state) =>
  state.cart.items.reduce(
    (total, item) =>
      total + item.price * item.quantity,
    0
  )
);
```

Interview Answer:

I use the reduce() method to calculate the total price by multiplying each product's price by its quantity and adding all the values.

Q44. Should we store API response data in Redux?

Interview Answer:

It depends on the application. Shared server data can be managed with RTK Query or other caching solutions. Local component data does not always need to be stored in Redux.

मराठी स्पष्टीकरण:

प्रत्येक API Response Redux मध्ये Store करण्याची गरज नसते.

- Shared Server Data → RTK Query.
- Complex Shared Client State → Redux Toolkit.
- Local Form State → useState.
- Small Shared State → Context किंवा Zustand देखील वापरू शकतो.

Q45. How do you persist Redux State after page refresh?

Interview Answer:

Redux state is stored in memory by default. To persist selected state across page reloads, we can use Redux Persist or browser storage with a carefully designed persistence strategy.

मराठी स्पष्टीकरण:

Browser Refresh केल्यावर Redux Store Reset होतो कारण तो Memory मध्ये असतो.

Persistence साठी:

- Redux Persist
- localStorage
- sessionStorage

यांचा वापर करता येतो.

Sensitive Authentication Data बाबतीत Storage निवडताना Security विचारात घ्यावी.

9. Redux Toolkit vs Other State Management

Q46. Redux Toolkit vs Context API?

Redux Toolkit	**Context API**
External library	Built into React
Structured state management	Context-based sharing
Middleware and DevTools ecosystem	No Redux-style middleware built in
Good for complex shared state	Good for simpler shared values

Q47. Redux Toolkit vs Zustand?

Redux Toolkit	**Zustand**
More structured architecture	Simple API
Built-in async tools like RTK Query	Additional middleware and patterns available
Strong DevTools support	DevTools middleware available
Useful for complex applications	Useful for lightweight global state

Q48. Redux Toolkit vs useState?

useState	**Redux Toolkit**
Component-level state	Shared application state
Built into React	External library
Simple local data	Complex shared data
No central store	Centralized store

10. Common Redux Toolkit Mistakes

Mistake	Correct Approach
Forgetting Provider	Wrap App with Provider
Not adding reducer to Store	Register slice reducer
Using useSelector outside Provider	Use inside Provider tree
Direct mutation outside RTK reducers	Use immutable state updates
Forgetting extraReducers	Handle async lifecycle actions
Storing unnecessary local state globally	Keep local state local
Storing API data without loading/error handling	Handle all request states
Storing functions in Redux State	Prefer serializable values
Not understanding Payload	Check action.payload
Dispatching action creator without calling dispatch	Use dispatch(action())

11. Redux Toolkit Output-Based Questions

Q49. What will be the output?

```
const counterSlice = createSlice({
 name: "counter",

 initialState: {
  count: 0
 },

 reducers: {
  increment: (state) => {
   state.count += 1;
  }
 }
});
```

```
dispatch(increment());

dispatch(increment());

dispatch(increment());
```

Answer: count = 3

कारण: प्रत्येक Action मुळे Count एकाने वाढतो.

Q50. What will be the output?

```
const initialState = {
  count: 5
};


const counterSlice = createSlice({
  name: "counter",
  initialState,


  reducers: {
    add: (state, action) => {
      state.count += action.payload;
    }
  }
});

dispatch(add(10));
```

Answer: count = 15

कारण: Initial Count 5 + Payload 10 = 15.

Q51. What is wrong with this code?

```
const count = useSelector(
  state => state.counter
);
```

जर Store मध्ये असेल:

```
{
  counter: {
    count: 10
  }
}
```

तर count Variable मध्ये पूर्ण Object येतो, Number नाही.

Correct:

```
const count = useSelector(
  state => state.counter.count
);
```

Node.js Interview Questions & Answers (Beginner to Advanced)

Beginner to Advanced

MERN Developer Interview

1. Node.js Fundamentals

Q1. What is Node.js?

Interview Answer:

Node.js is an open-source, cross-platform JavaScript runtime environment. It allows us to execute JavaScript code outside the browser, mainly for building server-side applications.

मराठी स्पष्टीकरण:

साधारणपणे JavaScript आपण Browser मध्ये वापरतो. पण Node.js मुळे JavaScript Code Browser च्या बाहेर म्हणजेच Computer किंवा Server वर Execute करता येतो.

Node.js वापरून आपण:

- Backend Applications
- REST APIs
- Real-time Chat Applications
- Authentication Systems
- File Handling
- Streaming Applications

तयार करू शकतो.

उदाहरण: React Frontend आणि Node.js Backend वापरून आपण पूर्ण MERN Application तयार करू शकतो.

Q2. Is Node.js a Programming Language?

Interview Answer:

No, Node.js is not a programming language. It is a JavaScript runtime environment that executes JavaScript outside the browser.

मराठी स्पष्टीकरण:

Node.js ही Programming Language नाही. JavaScript ही Programming Language आहे आणि Node.js हे JavaScript Execute करण्यासाठी Runtime Environment आहे.

Q3. Is Node.js a Framework?

Interview Answer:

No, Node.js is not a framework. It is a runtime environment. Express.js is a framework built on top of Node.js.

JavaScript	Node.js	Express.js
Programming Language	Runtime Environment	Web Framework
Writes application logic	Executes JavaScript outside browser	Simplifies backend development

Q4. Why do we use Node.js?

Interview Answer:

We use Node.js because it is lightweight, supports asynchronous programming, handles I/O operations efficiently, and allows developers to use JavaScript on both frontend and backend.

मराठी स्पष्टीकरण:

Node.js वापरण्याची कारणे:

1. JavaScript Frontend आणि Backend दोन्हीकडे वापरता येते.
2. Asynchronous Operations Support करतो.
3. I/O Operations Efficiently Handle करतो.
4. मोठ्या प्रमाणात npm Packages उपलब्ध आहेत.
5. Real-time Applications तयार करता येतात.
6. REST APIs तयार करणे सोपे होते.

Q5. What are the main features of Node.js?

Interview Answer:

The main features of Node.js are:

1. Asynchronous and non-blocking I/O.

2. Event-driven architecture.
3. Single JavaScript execution thread by default.
4. Fast JavaScript execution with V8.
5. Large npm ecosystem.
6. Cross-platform support.
7. Scalability for I/O-heavy applications.

मराठी स्पष्टीकरण:

Node.js मध्ये Event Loop, Non-blocking I/O आणि Event-driven Architecture हे महत्त्वाचे Concepts आहेत. यामुळे अनेक I/O Operations efficiently Handle करता येतात.

Q6. What is V8 Engine?

Interview Answer:

V8 is an open-source JavaScript engine developed by Google. It compiles and executes JavaScript code and is used by Node.js.

मराठी स्पष्टीकरण:

V8 Engine हे JavaScript Code Execute करण्याचे काम करते.

उदाहरणार्थ:

- Chrome Browser मध्ये V8 वापरले जाते.
- Node.js मध्येही V8 वापरले जाते.

Node.js मध्ये V8 सोबत Node चे Native APIs आणि Libuv सारखे Components असतात.

Q7. What is Libuv?

Interview Answer:

Libuv is a multi-platform library used by Node.js to handle asynchronous I/O operations, event loops, and a thread pool.

मराठी स्पष्टीकरण:

Libuv Node.js ला Asynchronous कामे Handle करण्यास मदत करते.

उदाहरणार्थ:

- File System Operations

- DNS Operations
- काही Crypto Operations
- Thread Pool Management
- Event Loop Support

Q8. What is the difference between Node.js and Browser JavaScript?

Browser JavaScript	**Node.js**
Runs inside browser	Runs outside browser
Has access to DOM APIs	No built-in browser DOM
Uses window object	Uses global and globalThis
Used mainly for frontend	Used mainly for backend and tooling
Browser Web APIs available	Node.js APIs available

2. Node.js Architecture

Q9. Explain Node.js Architecture.

Interview Answer:

Node.js uses an event-driven, non-blocking I/O architecture. JavaScript runs on the main thread, while asynchronous I/O operations are handled through operating system facilities or the libuv thread pool when needed.

मराठी स्पष्टीकरण:

Node.js Architecture समजून घेणे खूप महत्त्वाचे आहे.

Node.js मध्ये साधारणपणे हे Components असतात:

1. V8 Engine
2. Event Loop
3. Call Stack
4. Callback Queue
5. Libuv
6. Thread Pool
7. Operating System APIs

Node.js Architecture

JavaScript Code

V8 Engine

JavaScript execution

Call Stack + Event Loop

OS Async APIs

Network / timers

Libuv Thread Pool

File / crypto work

Event Loop Callbacks

Simplified conceptual diagram; actual Node.js execution involves multiple queues and phases.

Q10. What is the Event Loop in Node.js?

Interview Answer:

The Event Loop is a mechanism that allows Node.js to handle asynchronous operations without blocking the main JavaScript execution thread.

मराठी स्पष्टीकरण:

Event Loop हे Node.js मधील सर्वात महत्त्वाचे Concept आहे.

समजा Server वर एकाच वेळी अनेक Requests येत आहेत. प्रत्येक Request साठी JavaScript Thread थांबून राहिला तर Application Slow होईल.

Event Loop मुळे Node.js Asynchronous Operations पूर्ण झाल्यावर त्यांचे Callbacks योग्य वेळी Execute करू शकतो.

उदाहरण:

```
console.log("Start");


setTimeout(() => {
  console.log("Timer");
}, 1000);
```

```
console.log("End");
```

Output:

Start

End

Timer

कारण:

- Start लगेच Execute होते.
- setTimeout() Timer Schedule करते.
- End लगेच Execute होते.
- Timer पूर्ण झाल्यावर Callback Execute होते.

Q11. What is the Call Stack?

Interview Answer:

The Call Stack is a data structure that tracks function execution in JavaScript. It follows the Last In, First Out principle.

मराठी स्पष्टीकरण:

Function Execute होताना तो Call Stack मध्ये जातो. Function Complete झाल्यावर Stack मधून बाहेर येतो.

```
function first() {
  second();
}

function second() {
  console.log("Hello");
}

first();
```

Execution:

1. first() Stack मध्ये जातो.
2. second() Stack मध्ये जातो.
3. console.log() Execute होते.
4. second() Complete होतो.
5. first() Complete होतो.

Q12. What is Blocking and Non-blocking I/O?

Interview Answer:

Blocking I/O stops further execution until an operation completes. Non-blocking I/O allows the program to continue executing while an I/O operation is in progress.

मराठी स्पष्टीकरण:

Blocking: एखादे काम पूर्ण होईपर्यंत पुढचे JavaScript काम थांबते.

Non-blocking: काम सुरू करून पुढचे काम करता येते. काम पूर्ण झाल्यावर Callback किंवा Promise द्वारे Result मिळतो.

Example:

```
const fs = require("fs");


// Blocking
const data = fs.readFileSync("data.txt", "utf8");


console.log(data);
console.log("Next task");
const fs = require("fs");


// Non-blocking
fs.readFile("data.txt", "utf8", (err, data) => {
  if (err) {
```

```
  console.log(err);
  return;
 }

 console.log(data);
});

console.log("Next task");
```

दुसऱ्या उदाहरणात File Read होण्याची वाट पाहत JavaScript पुढचे काम थांबवत नाही.

Q13. What is Single Threaded in Node.js?

Interview Answer:

Node.js executes JavaScript callbacks on a single main thread by default. However, it can handle asynchronous I/O through the operating system and libuv's thread pool.

मराठी स्पष्टीकरण:

Node.js मध्ये JavaScript Execution सामान्यतः एका Main Thread वर होते.

परंतु याचा अर्थ Node.js फक्त एकच काम करू शकतो असा नाही.

- JavaScript Callback Execution — Main Thread.
- काही File Operations — Thread Pool.
- Network I/O — अनेकदा OS-level asynchronous facilities.
- CPU-heavy Work — Worker Threads किंवा इतर Process मध्ये देता येते.

Q14. Is Node.js good for CPU-intensive tasks?

Interview Answer:

Node.js is generally well suited for I/O-intensive applications. CPU-intensive tasks can block the main JavaScript thread, so we should use worker threads, child processes, or separate services when appropriate.

मराठी स्पष्टीकरण:

Node.js खालील कामांसाठी चांगला आहे:

- API Requests

- Database Operations
- Chat Applications
- File Uploads
- Network Communication

पण मोठे Image Processing, Video Encoding किंवा Heavy Calculations Main Thread वर केल्यास Server Block होऊ शकतो.

अशा वेळी Worker Threads वापरता येतात.

3. Node.js Core Modules

Q15. What are Core Modules in Node.js?

Interview Answer:

Core modules are built-in modules provided by Node.js. They can be used without installing external npm packages.

मराठी स्पष्टीकरण:

Node.js सोबत आधीपासून उपलब्ध असलेल्या Modules ना Core Modules म्हणतात.

महत्त्वाचे Modules:

Module	Use
fs	File System
http	HTTP Server
https	Secure HTTP
path	File Paths
os	Operating System Information
events	Event Handling
crypto	Cryptographic Operations
stream	Streaming Data
buffer	Binary Data
url	URL Handling

util	Utility Functions
child_process	Child Processes
worker_threads	Parallel JavaScript Work

Q16. What is the fs Module?

Interview Answer:

The fs module provides APIs to interact with the file system, such as creating, reading, updating, and deleting files.

मराठी स्पष्टीकरण:

File Create, Read, Update आणि Delete करण्यासाठी fs Module वापरतो.

```
const fs = require("fs");


// Create / Write file
fs.writeFile("demo.txt", "Hello Node.js", (err) => {
  if (err) throw err;

  console.log("File created");
});
```

Read File:

```
fs.readFile("demo.txt", "utf8", (err, data) => {
  if (err) throw err;

  console.log(data);
});
```

Update File:

```
fs.appendFile("demo.txt", "\nWelcome", (err) => {
  if (err) throw err;

  console.log("File updated");
```

```
});
```

Delete File:

```
fs.unlink("demo.txt", (err) => {
  if (err) throw err;

  console.log("File deleted");
});
```

Q17. Difference between readFile() and readFileSync()?

Interview Answer:

readFile() is asynchronous, while readFileSync() is synchronous and blocks further JavaScript execution until the file is read.

readFile()	**readFileSync()**
Asynchronous	Synchronous
Non-blocking	Blocking
Callback-based	Returns result directly
Better for normal server I/O	Useful for controlled startup or scripts

Q18. What is the path Module?

Interview Answer:

The path module provides utilities for working with file and directory paths.

मराठी स्पष्टीकरण:

Windows आणि Linux मध्ये File Paths वेगवेगळ्या पद्धतीने असू शकतात. Path Module वापरल्याने Path Handling सोपे होते.

```
const path = require("path");

console.log(path.join("folder", "file.txt"));

console.log(path.basename("/app/data/file.txt"));
```

console.log(path.extname("photo.png"));

Q19. What is __dirname?

Interview Answer:

In CommonJS modules, __dirname provides the absolute path of the directory containing the current module.

console.log(__dirname);

मराठी स्पष्टीकरण:

सध्या Execute होत असलेल्या CommonJS File च्या Folder चा Absolute Path मिळतो.

टीप: Native ES Modules मध्ये __dirname उपलब्ध नसते. त्यासाठी import.meta.url वापरून Path तयार करतात.

Q20. What is the http Module?

Interview Answer:

The http module provides functionality to create HTTP servers and handle HTTP requests and responses.

मराठी स्पष्टीकरण:

Node.js वापरून Basic HTTP Server तयार करण्यासाठी http Module वापरतो.

```
const http = require("http");


const server = http.createServer((req, res) => {
 res.writeHead(200, {
  "Content-Type": "text/plain"
 });


 res.end("Hello from Node.js");
});


server.listen(5000, () => {
```

```
  console.log("Server running on port 5000");
});
```

Q21. What is the os Module?

Interview Answer:

The os module provides information about the operating system, CPU, memory, and system architecture.

```
const os = require("os");


console.log(os.platform());
console.log(os.arch());
console.log(os.totalmem());
console.log(os.freemem());
```

Q22. What is the events Module?

Interview Answer:

The events module provides the EventEmitter class, which allows us to create and handle custom events.

मराठी स्पष्टीकरण:

आपले Custom Events तयार करून त्यांना Listen करण्यासाठी EventEmitter वापरतो.

```
const EventEmitter = require("events");


const emitter = new EventEmitter();


emitter.on("userCreated", (name) => {
  console.log(` ${name} created `);
});


emitter.emit("userCreated", "Manoj");
```

Output:

Manoj created

Q23. What is the crypto Module?

Interview Answer:

The crypto module provides cryptographic functionality such as hashing, encryption, decryption, and secure random value generation.

मराठी स्पष्टीकरण:

Security संबंधित कामांसाठी Crypto Module वापरतो.

उदाहरणार्थ:

- Hashing
- Encryption
- Decryption
- Random Token Generation
- Digital Signatures

```
const crypto = require("crypto");


const token = crypto.randomBytes(32).toString("hex");


console.log(token);
```

टीप: User Password साठवण्यासाठी साधा SHA-256 वापरण्याऐवजी bcrypt किंवा Argon2 सारखा Password Hashing Algorithm वापरावा.

4. Modules and Package Management

Q24. What is a Module in Node.js?

Interview Answer:

A module is a reusable piece of code that can be imported and used in different parts of an application.

मराठी स्पष्टीकरण:

मोठा Code छोटे-छोटे Reusable Files मध्ये विभागणे म्हणजे Module System.

उदाहरण:

math.js

```
function add(a, b) {
  return a + b;
}


module.exports = add;
```

app.js

```
const add = require("./math");


console.log(add(10, 20));
```

Q25. What is CommonJS?

Interview Answer:

CommonJS is a module system traditionally used in Node.js. It uses require() to import modules and module.exports to export them.

```
// Export
module.exports = {
  name: "Manoj"
};


// Import
const user = require("./user");
```

Q26. What is ES Module (ESM)?

Interview Answer:

ES Modules are the standard JavaScript module system. They use import and export statements to share code between files.

```
// Export
export const name = "Manoj";


// Import
```

import { name } from "./user.js";

Q27. Difference between CommonJS and ES Modules?

CommonJS	ES Modules
require()	import
module.exports	export
Traditionally synchronous loading	Supports static module imports
Common in older Node.js projects	Standard JavaScript module system

महत्वाचे: Node.js मध्ये ESM वापरण्यासाठी Project Configuration योग्य असणे आवश्यक आहे. सामान्यतः package.json मध्ये "type": "module" वापरतात किंवा .mjs Extension वापरतात.

Q28. What is npm?

Interview Answer:

npm stands for Node Package Manager. It is used to install, manage, and share JavaScript packages.

मराठी स्पष्टीकरण:

npm वापरून External Libraries आणि Packages Install करू शकतो.

उदाहरण:

npm install express

npm install mongoose

npm install bcryptjs

Q29. What is package.json?

Interview Answer:

package.json is a project configuration file that contains project information, scripts, dependencies, and other metadata.

मराठी स्पष्टीकरण:

Node.js Project ची Configuration आणि Dependencies यांची माहिती package.json मध्ये असते.

{

```
 "name": "my-app",
 "version": "1.0.0",
 "scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
 },
 "dependencies": {
  "express": "^5.0.0"
 },
 "devDependencies": {
  "nodemon": "^3.0.0"
 }
}
```

Q30. What is package-lock.json?

Interview Answer:

package-lock.json records the exact dependency tree and package versions installed in a project, helping ensure consistent installations.

मराठी स्पष्टीकरण:

Project मध्ये कोणत्या Package ची कोणती Exact Version आणि कोणत्या Dependencies Install झाल्या आहेत याची माहिती package-lock.json मध्ये असते.

Q31. Difference between dependencies and devDependencies?

dependencies	**devDependencies**
Required for application runtime	Mainly needed during development/build
Express	Nodemon
Mongoose	ESLint
bcryptjs	Testing tools

npm install express

npm install nodemon --save-dev

Q32. What is npx?

Interview Answer:

npx is a command-line tool provided with npm that can execute package binaries without requiring a separate global installation.

npx create-vite@latest

Q33. What is nodemon?

Interview Answer:

Nodemon is a development tool that automatically restarts the Node.js application when it detects file changes.

npm install --save-dev nodemon

```
{
  "scripts": {
    "dev": "nodemon server.js"
  }
}
```

Run:

npm run dev

5. Asynchronous JavaScript in Node.js

Q34. What is Asynchronous Programming?

Interview Answer:

Asynchronous programming allows a program to start an operation and continue executing other code without waiting for that operation to finish.

मराठी स्पष्टीकरण:

एखादे काम पूर्ण होईपर्यंत संपूर्ण Program थांबवण्याऐवजी दुसरी कामे सुरू ठेवणे म्हणजे Asynchronous Programming.

उदाहरण:

- API Call
- File Read

- Database Query
- Timer

Q35. What is a Callback Function?

Interview Answer:

A callback is a function passed as an argument to another function and executed later.

```
function greet(name, callback) {
  console.log("Hello " + name);

  callback();
}

function message() {
  console.log("Welcome");
}

greet("Manoj", message);
```

Output:

Hello Manoj

Welcome

Q36. What is Callback Hell?

Interview Answer:

Callback Hell occurs when multiple nested callbacks make code difficult to read, maintain, and debug.

मराठी स्पष्टीकरण:

एक Callback पूर्ण झाल्यावर दुसरा, त्यानंतर तिसरा अशा पद्धतीने खूप Nested Callbacks लिहिल्यामुळे Code गुंतागुंतीचा होतो.

उपाय:

- Promises

- async/await
- Separate reusable functions

Q37. What is a Promise?

Interview Answer:

A Promise is a JavaScript object that represents the eventual success or failure of an asynchronous operation.

मराठी स्पष्टीकरण:

Asynchronous Operation चे Future Result दर्शवण्यासाठी Promise वापरतो.

Promise च्या तीन States:

State	Meaning
Pending	Operation चालू आहे
Fulfilled	Operation Successful
Rejected	Operation Failed

```
const promise = new Promise((resolve, reject) => {
 const success = true;

 if (success) {
  resolve("Task completed");
 } else {
  reject("Task failed");
 }
});

promise
 .then(result => console.log(result))
 .catch(error => console.log(error));
```

Q38. What is async/await?

Interview Answer:

async/await is syntax built on Promises that makes asynchronous code easier to read and maintain.

```
async function getData() {
 try {
   const result = await Promise.resolve("Success");

   console.log(result);
 } catch (error) {
   console.log(error);
 }
}

getData();
```

मराठी स्पष्टीकरण:

async Function नेहमी Promise Return करते. await वापरून Promise Settled होईपर्यंत त्या Async Function मधील पुढील Execution थांबवता येते. पण यामुळे संपूर्ण JavaScript Thread Block होत नाही.

Q39. Difference between Promise and async/await?

Promise	**async/await**
Uses .then() and .catch()	Uses await and try/catch
Can create Promise chains	Makes async code easier to read
Promise-based API	Syntax built on Promises

Q40. What is Promise.all()?

Interview Answer:

Promise.all() takes an iterable of promises and fulfills when all promises fulfill. It rejects as soon as one of them rejects.

```
const p1 = Promise.resolve("One");
```

```
const p2 = Promise.resolve("Two");

const p3 = Promise.resolve("Three");


Promise.all([p1, p2, p3])

 .then(result => console.log(result));
```

Output:

["One", "Two", "Three"]

Q41. Difference between Promise.all(), allSettled(), race() and any()?

Method	Behavior
Promise.all()	सर्व Fulfilled झाले तर Success; एखादे Reject झाले तर Reject
Promise.allSettled()	सर्व Promises Settled होईपर्यंत Wait
Promise.race()	पहिल्यांदा Settled झालेला Result
Promise.any()	पहिला Fulfilled Result; सर्व Reject झाल्यास AggregateError

Q42. What is the Event Loop Priority between Promise and setTimeout?

Interview Answer:

Promise callbacks such as then() are microtasks, while setTimeout callbacks are processed through the timers phase. Microtasks are processed before the Event Loop continues to the next phase.

Example:

```
console.log("Start");


setTimeout(() => {

 console.log("Timeout");

}, 0);


Promise.resolve().then(() => {

 console.log("Promise");
```

```
});


console.log("End");
```

Output:

Start

End

Promise

Timeout

कारण: Synchronous Code आधी Execute होतो. त्यानंतर Promise Microtask Callback आणि मग Timer Callback Execute होतो.

6. Express.js Integration

Q43. What is Express.js?

Interview Answer:

Express.js is a lightweight web framework built on Node.js. It simplifies server creation, routing, middleware handling, and API development.

मराठी स्पष्टीकरण:

Node.js च्या http Module वापरून Server तयार करता येतो. पण Routing, Middleware, Request Handling सोपे करण्यासाठी Express.js वापरतात.

Q44. How do you create a basic Express server?

```
const express = require("express");


const app = express();


app.get("/", (req, res) => {
  res.send("Hello from Express");
});


app.listen(5000, () => {
```

```
 console.log("Server running on port 5000");
});
```

Q45. What is Middleware?

Interview Answer:

Middleware is a function that has access to the request object, response object, and the next middleware function in the request-response cycle.

मराठी स्पष्टीकरण:

Client कडून Request आल्यावर Route Handler आधी किंवा दरम्यान Execute होणाऱ्या Functions ना Middleware म्हणतात.

Middleware वापर:

- Authentication
- Logging
- Request Validation
- Error Handling
- CORS
- JSON Parsing

```
app.use((req, res, next) => {
 console.log(req.method, req.url);

 next();
});
```

Q46. What is the purpose of next()?

Interview Answer:

next() passes control to the next middleware function in the request-response cycle.

मराठी स्पष्टीकरण:

Middleware चे काम पूर्ण झाल्यावर पुढील Middleware किंवा Route Handler कडे Control पाठवण्यासाठी next() वापरतो.

Q47. What is the difference between app.use() and app.get()?

app.use()	app.get()
Registers middleware	Handles GET requests
Can apply to multiple HTTP methods	Handles only GET
Often used for common middleware	Used for GET routes

Q48. What are req and res?

Interview Answer:

req represents the incoming HTTP request, while res represents the HTTP response sent back to the client.

मराठी स्पष्टीकरण:

- req — Client कडून आलेली Request.
- res — Server कडून पाठवला जाणारा Response.

```
app.post("/users", (req, res) => {
 console.log(req.body);


 res.status(201).json({
  message: "User created"
 });
});
```

Q49. What is express.json()?

Interview Answer:

express.json() is built-in Express middleware that parses incoming JSON request bodies.

```
app.use(express.json());
```

मराठी स्पष्टीकरण:

Frontend कडून JSON Format मध्ये Data आला तर req.body मध्ये तो मिळवण्यासाठी हे Middleware वापरतो.

Q50. What is Router in Express?

Interview Answer:

Express Router is used to create modular and separate route handlers for different parts of an application.

```
const express = require("express");


const router = express.Router();


router.get("/", (req, res) => {
  res.send("All users");
});


router.get("/:id", (req, res) => {
  res.send("Single user");
});


module.exports = router;
```

मुख्य Server:

```
const userRoutes = require("./routes/userRoutes");


app.use("/api/users", userRoutes);
```

7. REST API and HTTP Concepts

Q51. What is REST API?

Interview Answer:

REST API is an architectural style for designing web APIs using resources and standard HTTP methods.

मराठी स्पष्टीकरण:

Frontend आणि Backend मध्ये Communication करण्यासाठी REST API वापरतो.

उदाहरण:

Method	Use
GET	Data Read
POST	Data Create
PUT	Full Update
PATCH	Partial Update
DELETE	Data Delete

Q52. What is the difference between PUT and PATCH?

Interview Answer:

PUT is generally used to replace a resource, while PATCH is used to apply a partial update.

मराठी स्पष्टीकरण:

समजा User मध्ये Name, Email आणि Phone आहे.

- PUT — पूर्ण Resource Replace करण्यासाठी.
- PATCH — फक्त Name किंवा Phone Update करण्यासाठी.

Q53. What are HTTP Status Codes?

Status	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Unprocessable Content

500 Internal Server Error

Q54. Difference between 401 and 403?

Interview Answer:

401 means valid authentication credentials are missing or invalid. 403 means the server understood the request but refuses access due to insufficient permission.

मराठी स्पष्टीकरण:

- 401 — Login किंवा Valid Authentication नाही.
- 403 — User Login आहे पण त्या Resource ची Permission नाही.

8. Authentication and Security

Q55. What is Authentication?

Interview Answer:

Authentication is the process of verifying the identity of a user.

मराठी स्पष्टीकरण:

User खरोखर कोण आहे हे Verify करणे म्हणजे Authentication.

उदाहरण: Login Email आणि Password तपासणे.

Q56. What is Authorization?

Interview Answer:

Authorization determines what an authenticated user is allowed to access or perform.

मराठी स्पष्टीकरण:

User ला कोणते काम करण्याची Permission आहे हे तपासणे म्हणजे Authorization.

उदाहरण:

- Admin — User Delete करू शकतो.
- Normal User — स्वतःचा Profile पाहू शकतो.

Q57. What is JWT Authentication?

Interview Answer:

JWT authentication is a method where the server issues a signed token after verifying the user. The token is then used to authenticate subsequent requests.

मराठी स्पष्टीकरण:

Login केल्यानंतर Backend JWT तयार करून Client ला देतो. त्यानंतर Client Protected API साठी Token किंवा Cookie द्वारे Authentication करतो.

Flow:

1. User Login Request पाठवतो.
2. Backend Credentials Verify करतो.
3. JWT तयार करतो.
4. Token सुरक्षितपणे Client पर्यंत पोहोचवतो.
5. पुढील Request मध्ये Token पाठवला जातो.
6. Backend Token Verify करतो.

Q58. What is bcrypt?

Interview Answer:

bcrypt is a password hashing library used to securely hash passwords and compare submitted passwords with stored password hashes.

मराठी स्पष्टीकरण:

Password Database मध्ये Plain Text स्वरूपात ठेवू नये. त्याऐवजी bcrypt वापरून Password Hash करून Store करतो.

```
const bcrypt = require("bcryptjs");


async function hashPassword(password) {
  const hashedPassword = await bcrypt.hash(password, 12);


  return hashedPassword;
}


async function comparePassword(password, hashedPassword) {
  return bcrypt.compare(password, hashedPassword);
```

}

Q59. What is Helmet?

Interview Answer:

Helmet is Express middleware that helps secure an application by setting various HTTP security headers.

```
const helmet = require("helmet");


app.use(helmet());
```

मराठी स्पष्टीकरण:

Browser ला Security संबंधित HTTP Headers पाठवण्यासाठी Helmet वापरतात.

Q60. What is Rate Limiting?

Interview Answer:

Rate limiting restricts the number of requests a client can make within a specified time period.

मराठी स्पष्टीकरण:

एकाच User किंवा IP कडून खूप जास्त Requests येत असतील तर त्यावर मर्यादा घालणे म्हणजे Rate Limiting.

उदाहरण: Login API वर एका वेळेत मर्यादित Attempts ठेवणे.

Q61. What is CORS?

Interview Answer:

CORS stands for Cross-Origin Resource Sharing. It controls whether browsers allow a frontend from one origin to access resources from another origin.

```
const cors = require("cors");


app.use(cors({
  origin: "http://localhost:5173",
  credentials: true
}));
```

मराठी स्पष्टीकरण:

React Frontend आणि Express Backend वेगवेगळ्या Ports वर चालत असतील तर Browser ला योग्य CORS Configuration लागते.

Q62. What is XSS?

Interview Answer:

Cross-Site Scripting is a security vulnerability where attackers inject malicious scripts into web pages viewed by other users.

मराठी स्पष्टीकरण:

Website मध्ये User कडून आलेला Untrusted HTML किंवा JavaScript सुरक्षित न करता Render केल्यास XSS होऊ शकतो.

Protection:

- Input Validation
- Output Encoding
- Safe Rendering
- HTML Sanitization

Q63. What is SQL Injection / NoSQL Injection?

Interview Answer:

Injection attacks happen when untrusted input changes the meaning of a database query. We should validate input and use safe query construction.

मराठी स्पष्टीकरण:

User कडून आलेला Data Database Query मध्ये चुकीच्या पद्धतीने वापरल्यास Attacker Query Manipulate करू शकतो.

MongoDB मध्ये User Input थेट Query Object म्हणून स्वीकारू नये.

9. Node.js Error Handling

Q64. What is Error Handling in Node.js?

Interview Answer:

Error handling is the process of detecting, managing, and responding to errors so that an application behaves safely and predictably.

मराठी स्पष्टीकरण:

Application मध्ये Error आल्यावर Server Crash होऊ नये आणि Client ला योग्य Error Response मिळावा यासाठी Error Handling करतो.

Q65. What is try...catch?

Interview Answer:

try...catch is used to handle exceptions in synchronous code and errors thrown while awaiting promises inside an async function.

```
async function getUser() {
 try {
  const user = await User.findById("123");

  console.log(user);
 } catch (error) {
  console.log(error.message);
 }
}
```

Q66. What is Global Error Handling Middleware?

Interview Answer:

Global error-handling middleware is a centralized Express middleware used to handle errors from different routes and middleware functions.

```
app.use((err, req, res, next) => {
 console.error(err);

 res.status(err.statusCode || 500).json({
  success: false,
  message: err.message || "Internal Server Error"
 });
});
```

महत्त्वाचे: Express मध्ये Error Middleware चे चार Parameters असतात: err, req, res, next. ते Routes आणि इतर Middleware नंतर Register करावे.

Q67. What is the difference between throw and next(error)?

Interview Answer:

throw raises an exception, while next(error) passes an error to Express error-handling middleware.

```
app.get("/user", async (req, res, next) => {
 try {
  throw new Error("User not found");
 } catch (error) {
  next(error);
 }
});
```

10. Streams and Buffers

Q68. What is a Stream in Node.js?

Interview Answer:

A stream is a way to process data piece by piece instead of loading the entire data into memory.

मराठी स्पष्टीकरण:

मोठी File एकाच वेळी Memory मध्ये Load न करता छोट्या-छोट्या Chunks मध्ये Process करणे म्हणजे Stream.

उदाहरण:

- Video Streaming
- Large File Upload
- File Download
- Large File Reading

Q69. What are the types of Streams?

Interview Answer:

There are four main types of streams in Node.js:

1. Readable
2. Writable
3. Duplex
4. Transform

Stream	Use
Readable	Data Read
Writable	Data Write
Duplex	Read + Write
Transform	Data Read + Modify + Write

Q70. What is Buffer?

Interview Answer:

A Buffer is a Node.js object used to work with binary data directly.

मराठी स्पष्टीकरण:

Image, Audio, Video किंवा इतर Binary Data Handle करण्यासाठी Buffer वापरतात.

```
const buffer = Buffer.from("Hello");


console.log(buffer);
console.log(buffer.toString());
```

Q71. Difference between Buffer and Stream?

Buffer	Stream
Holds binary data in memory	Processes data in chunks
Useful for smaller data	Useful for large data
Can require more memory	Can reduce memory usage

Q72. What is pipe()?

Interview Answer:

pipe() connects a readable stream to a writable stream and transfers data efficiently.

```
const fs = require("fs");


const readStream = fs.createReadStream("input.txt");


const writeStream = fs.createWriteStream("output.txt");


readStream.pipe(writeStream);
```

11. Advanced Node.js Concepts

Q73. What is Worker Threads?

Interview Answer:

Worker Threads allow JavaScript code to run in separate threads, making them useful for CPU-intensive tasks.

मराठी स्पष्टीकरण:

Heavy Calculations Main JavaScript Thread वर केल्यास Event Loop Block होऊ शकतो. अशा कामासाठी Worker Threads वापरता येतात.

Q74. What is Child Process?

Interview Answer:

The child_process module allows Node.js applications to create and manage separate operating system processes.

मराठी स्पष्टीकरण:

Main Node.js Process पासून वेगळी OS Process सुरू करायची असल्यास Child Process वापरतो.

मुख्य Methods:

- spawn()
- exec()
- execFile()
- fork()

Q75. Difference between Worker Threads and Child Process?

Worker Threads	Child Process
Separate JavaScript thread	Separate OS process
Can share memory through SharedArrayBuffer	Separate memory space by default
Useful for CPU-heavy JavaScript work	Useful for separate processes and commands
Lower overhead in many cases	Usually higher process overhead

Q76. What is Clustering in Node.js?

Interview Answer:

Clustering allows a Node.js application to run multiple worker processes that can share a server port.

मराठी स्पष्टीकरण:

Node.js Application अनेक CPU Cores चा उपयोग करू शकेल अशा पद्धतीने Multiple Worker Processes चालवण्यासाठी Cluster Module वापरता येते.

टीप: Production मध्ये अनेकदा Process Managers किंवा Container Orchestration वापरतात.

Q77. What is Memory Leak?

Interview Answer:

A memory leak happens when an application keeps references to objects that are no longer needed, preventing garbage collection from reclaiming their memory.

मराठी स्पष्टीकरण:

ज्या Objects ची आता गरज नाही त्यांचे References कायम ठेवल्यामुळे Memory वापर वाढत राहतो. याला Memory Leak म्हणतात.

कारणे:

- Unremoved Event Listeners
- Uncontrolled Global Variables
- Growing Caches
- Unreleased Timers
- Unnecessary References

Q78. What is Garbage Collection?

Interview Answer:

Garbage collection is the process of automatically reclaiming memory used by objects that are no longer reachable.

मराठी स्पष्टीकरण:

JavaScript मध्ये वापरात नसलेल्या Objects ची Memory V8 Engine कडून परत मोकळी केली जाते. याला Garbage Collection म्हणतात.

Q79. What is process.env?

Interview Answer:

process.env is an object that provides access to environment variables in a Node.js process.

मराठी स्पष्टीकरण:

Database URL, JWT Secret, Port आणि API Keys सारखी Configuration Values Environment Variables मध्ये ठेवतात.

console.log(process.env.PORT);

console.log(process.env.MONGO_URI);

.env File:

PORT=5000

MONGO_URI=mongodb://localhost:27017/mydb

JWT_SECRET=your-secret-value

dotenv वापरून:

require("dotenv").config();

Security Tip: .env File GitHub वर Push करू नये. .gitignore मध्ये .env ठेवावे.

Q80. What is process object?

Interview Answer:

The process object provides information about and control over the current Node.js process.

console.log(process.pid);

```
console.log(process.version);

console.log(process.platform);

console.log(process.cwd());
```

Q81. What is REPL?

Interview Answer:

REPL stands for Read-Eval-Print Loop. It is an interactive environment where we can execute JavaScript code directly.

Terminal मध्ये:

```
node
```

मग:

```
10 + 20
```

Output:

```
30
```

Q82. What is EventEmitter?

Interview Answer:

EventEmitter is a class in Node.js used to create, emit, and listen for custom events.

```
const EventEmitter = require("events");


const emitter = new EventEmitter();


emitter.on("orderPlaced", (orderId) => {
  console.log("Order:", orderId);
});


emitter.emit("orderPlaced", 101);
```

12. Node.js Performance and Best Practices

Q83. How can you improve Node.js performance?

Interview Answer:

We can improve Node.js performance by using asynchronous I/O, avoiding blocking operations, optimizing database queries, implementing caching, using streams, and scaling the application when needed.

मराठी स्पष्टीकरण:

Performance Improve करण्यासाठी:

1. Blocking Operations टाळणे.
2. Database Indexes वापरणे.
3. API Response मध्ये आवश्यक Data पाठवणे.
4. Caching वापरणे.
5. Streams वापरणे.
6. Pagination वापरणे.
7. CPU-heavy Tasks Worker Threads मध्ये देणे.
8. Proper Logging आणि Monitoring ठेवणे.

Q84. What is Caching?

Interview Answer:

Caching means storing frequently accessed data temporarily so that future requests can retrieve it faster.

मराठी स्पष्टीकरण:

वारंवार Database मधून तोच Data आणण्याऐवजी तो काही वेळासाठी Cache मध्ये ठेवणे म्हणजे Caching.

उदाहरण:

- Redis
- In-memory Cache
- CDN Cache

Q85. What is Pagination?

Interview Answer:

Pagination divides a large dataset into smaller pages so that the application can retrieve and display data efficiently.

मराठी स्पष्टीकरण:

Database मध्ये 10,000 Products असतील तर सर्व एकाच वेळी पाठवण्याऐवजी 10 किंवा 20 Products एका Page मध्ये पाठवतो.

उदाहरण:

```
const page = 1;

const limit = 10;


const skip = (page - 1) * limit;


const products = await Product.find()

 .skip(skip)

 .limit(limit);
```

Q86. What is Helmet and why is it used?

Interview Answer:

Helmet helps secure Express applications by setting HTTP security headers.

```
const helmet = require("helmet");


app.use(helmet());
```

Q87. What is a Reverse Proxy?

Interview Answer:

A reverse proxy is a server that receives client requests and forwards them to backend servers.

मराठी स्पष्टीकरण:

Client थेट Node.js Server ला Request न पाठवता आधी Nginx सारख्या Reverse Proxy कडे Request पाठवतो. त्यानंतर Nginx योग्य Backend कडे Request Forward करतो.

उदाहरण:

Client

↓

Nginx

↓

Node.js / Express

↓

MongoDB

13. Node.js Real-World MERN Interview Questions

Q88. Explain the complete request-response lifecycle in MERN.

Interview Answer:

The React frontend sends an HTTP request to the Express backend. Express middleware processes the request, the route handler executes business logic, and Mongoose communicates with MongoDB when required. The backend then sends a JSON response to React.

मराठी स्पष्टीकरण:

उदाहरणार्थ User Registration:

1. React Form मधून User Data घेतो.
2. Axios किंवा Fetch वापरून API Call करतो.
3. Express Request Receive करतो.
4. Middleware Request तपासतो.
5. Route Handler Execute होतो.
6. Controller Validation करतो.
7. Password Hash केला जातो.
8. Mongoose Database मध्ये User Save करते.
9. Server Response पाठवतो.
10. React UI Update करतो.

Q89. What is MVC Architecture?

Interview Answer:

MVC stands for Model-View-Controller. It is a software architectural pattern that separates application data, user interface, and business logic.

मराठी स्पष्टीकरण:

MVC मध्ये Application तीन भागांमध्ये विभागले जाते.

Part	Responsibility
Model	Database आणि Data Structure
View	User Interface
Controller	Request आणि Business Logic

Node.js Backend मध्ये MVC वापरल्यामुळे Code व्यवस्थित आणि Maintainable राहतो.

Q90. What is the difference between Controller and Middleware?

Interview Answer:

Middleware performs common request-processing tasks, while a controller handles the main business logic for a particular route.

मराठी स्पष्टीकरण:

- Middleware — Authentication, Logging, Validation.
- Controller — User Create, Product Update, Order Place.

Q91. What is Mongoose?

Interview Answer:

Mongoose is an ODM library for MongoDB and Node.js. It provides schemas, models, validation, and convenient database operations.

मराठी स्पष्टीकरण:

Node.js मधून MongoDB सोबत काम करणे सोपे करण्यासाठी Mongoose वापरतो.

```
const mongoose = require("mongoose");


const userSchema = new mongoose.Schema({
  name: {
    type: String,
```

```
  required: true
 },

 email: {
  type: String,
  required: true,
  unique: true
 }
});

const User = mongoose.model("User", userSchema);
```

Q92. What is Socket.IO?

Interview Answer:

Socket.IO is a library that enables real-time, bidirectional communication between clients and servers. It commonly uses WebSocket when available and can fall back to other transports.

मराठी स्पष्टीकरण:

सामान्य HTTP Request मध्ये Client Request पाठवतो आणि Server Response देतो.

पण Chat Application मध्ये Real-time Communication हवी असते. त्यासाठी Socket.IO वापरता येते.

उदाहरण:

- Real-time Chat
- Typing Indicator
- Live Notifications
- Online Status
- Live Dashboard

Q93. What is the difference between HTTP and WebSocket?

HTTP	WebSocket

Request-response communication	Bidirectional communication
Client usually initiates requests	Both sides can send messages after connection
Common for REST APIs	Common for real-time apps
Stateless request model	Persistent connection

Q94. How do you handle file uploads in Node.js?

Interview Answer:

We can handle file uploads in Express using middleware such as Multer. Files can then be stored locally or uploaded to cloud storage services.

मराठी स्पष्टीकरण:

React मधून Image किंवा File पाठवला जातो. Express मध्ये Multer वापरून तो Receive करता येतो. नंतर Cloudinary किंवा इतर Storage Service वर Upload करू शकतो.

14. Node.js Output-Based Questions

हे प्रश्न Interview मध्ये खूप विचारले जातात.

Q95. What is the output?

```
console.log("A");


setTimeout(() => {
  console.log("B");
}, 0);


console.log("C");
```

Output

A

C

B

कारण: Synchronous Code आधी Execute होतो.

Q96. What is the output?

```
console.log("Start");

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Output:

Start

End

Promise

कारण: Promise Callback Microtask Queue मध्ये जातो.

Q97. What is the output?

```
console.log("One");

setTimeout(() => {
  console.log("Two");
}, 0);

Promise.resolve().then(() => {
  console.log("Three");
});

console.log("Four");
```

Output:

One

Four

Three

Two

कारण: Synchronous Code → Promise Microtask → Timer Callback.

Q98. What is the output?

```
function test() {
  return new Promise((resolve) => {
    resolve("Success");
  });
}

test().then(result => {
  console.log(result);
});
```

Output:

Success

Q99. What is the output?

```
console.log("Start");

async function test() {
  console.log("Inside");

  await Promise.resolve();

  console.log("After Await");
}

test();

console.log("End");
```

Output:

Start

Inside

End

After Await

कारण: await नंतरचे Code Promise Microtask म्हणून पुढे Execute होते.

Q100. What is the output?

```
const fs = require("fs");


console.log("A");


fs.readFile(__filename, () => {
  console.log("B");
});


console.log("C");
```

Output:

A

C

B

कारण: File Read Asynchronous आहे. त्यामुळे Callback नंतर Execute होतो.

15. Practical Node.js Coding Interview Questions

No.	Coding Question	Concept
1	Create HTTP Server	HTTP Module
2	Read and Write File	fs Module
3	Create Custom Event	EventEmitter
4	Create REST API	Express

5	Create User Registration API	Express + MongoDB
6	Implement Login API	JWT + bcrypt
7	Create Middleware	Express Middleware
8	Create Global Error Handler	Error Handling
9	Upload Image	Multer + Cloudinary
10	Implement Pagination	MongoDB
11	Implement Search API	Query Parameters
12	Implement Rate Limiting	Security
13	Create File Stream	Streams
14	Implement Worker Thread	Multithreading
15	Create Socket.IO Chat	Real-time Communication

16. Common Node.js Interview Mistakes

Mistake	**Correct Understanding**
Node.js is a language	It is a runtime environment
Node.js is fully single-threaded	JS runs on the main thread; other work may use OS or worker threads
Async means parallel execution	Async means not waiting synchronously
setTimeout(0) executes immediately	It runs in a later timer phase
Every async task uses thread pool	Many network I/O operations use OS facilities
Express is Node.js	Express is a framework built on Node.js
Promise and async/await are different systems	async/await uses Promises
readFileSync() is non-blocking	It is blocking
All Errors are caught by try/catch	Async callbacks need their own error handling

JWT payload is encrypted by default	JWT payload is usually encoded, not encrypted

17. Node.js Quick Revision Table

Concept	Short Answer
Node.js	JavaScript Runtime
V8	JavaScript Engine
Libuv	Async I/O and Event Loop support
Event Loop	Coordinates asynchronous callbacks
Call Stack	Tracks function execution
fs	File System
http	HTTP Server
path	Path Handling
os	Operating System Information
events	EventEmitter
crypto	Cryptographic Functions
Buffer	Binary Data
Stream	Chunk-based Data Processing
npm	Package Manager
package.json	Project Configuration
CommonJS	require / module.exports
ESM	import / export
Express	Node.js Web Framework
Middleware	Request Processing Function
JWT	Signed Token Format
bcrypt	Password Hashing

Socket.IO	Real-time Communication
Worker Threads	Separate JavaScript Threads
Cluster	Multiple Node.js Processes
MVC	Model-View-Controller
REST API	Resource-based HTTP API

Express.js Interview Questions & Answers – Beginner to Advanced

Part 9 – Backend Development

1. Express.js Fundamentals

Q1. What is Express.js?

Interview Answer:

Express.js is a fast, minimal, and flexible web framework for Node.js. It helps us build web applications, REST APIs, and backend services easily.

मराठी स्पष्टीकरण:

Express.js हा Node.js वर आधारित backend framework आहे. Node.js वापरून HTTP server तयार करता येतो, पण routing, middleware, request handling आणि response handling स्वतः manage करावे लागते.

Express.js या गोष्टी सोप्या करतो.

Example:

```
const express = require("express");


const app = express();


app.get("/", (req, res) => {
  res.send("Hello Express.js");
});


app.listen(5000, () => {
  console.log("Server is running");
});
```

Q2. Why do we use Express.js?

Interview Answer:

We use Express.js because it makes backend development easier. It provides routing, middleware, request handling, error handling, and REST API support.

मराठी स्पष्टीकरण:

Express.js वापरण्याचे मुख्य फायदे:

1. Easy routing – URL नुसार वेगवेगळे operations.
2. Middleware – Request आणि response मधील processing.
3. REST API – Backend APIs तयार करणे.
4. Error handling – Errors manage करणे.
5. Database integration – MongoDB सारख्या database सोबत काम करणे.
6. Easy development – कमी code मध्ये server तयार करणे.

Q3. Is Express.js a programming language?

Interview Answer:

No, Express.js is not a programming language. It is a web framework built on top of Node.js. JavaScript is the programming language used with Express.js.

मराठी:

- JavaScript = Programming Language
- Node.js = JavaScript Runtime Environment
- Express.js = Node.js Web Framework

Q4. What is the difference between Node.js and Express.js?

Node.js	**Express.js**
JavaScript runtime	Web framework
Runs JavaScript outside browser	Runs on Node.js
Provides built-in HTTP module	Provides easier routing and middleware
Can create servers	Makes server development easier
Lower-level APIs	Higher-level abstractions

Interview Answer:

Node.js allows us to run JavaScript on the server. Express.js is a framework built on Node.js that makes it easier to create APIs and web applications.

Q5. How do you install Express.js?

Interview Answer:

First, we initialize a Node.js project using npm init, and then install Express using npm install express.

npm init -y

npm install express

Start the server:

node server.js

Q6. How do you create a basic Express server?

```
const express = require("express");


const app = express();


app.get("/", (req, res) => {
  res.send("Welcome to Express");
});


app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

मराठी स्पष्टीकरण:

- require("express") – Express import करतो.
- express() – Express application तयार करतो.
- app.get() – GET route तयार करतो.
- req – Client कडून आलेली request.
- res – Client ला पाठवायचा response.
- app.listen() – Server सुरू करतो.

2. Routing in Express.js

Q7. What is Routing in Express.js?

Interview Answer:

Routing is the process of defining how an application responds to different HTTP requests and URLs.

मराठी:

User कोणत्या URL वर request पाठवतो आणि कोणती HTTP method वापरतो त्यानुसार कोणता function execute करायचा हे Routing ठरवते.

```
app.get("/users", (req, res) => {
  res.send("Get all users");
});

app.post("/users", (req, res) => {
  res.send("Create user");
});

app.put("/users/:id", (req, res) => {
  res.send("Update user");
});

app.delete("/users/:id", (req, res) => {
  res.send("Delete user");
});
```

Q8. What are HTTP methods in Express.js?

Method	Purpose	Example
GET	Fetch data	Get all users
POST	Create data	Create user

PUT	Replace/update resource	Update full user
PATCH	Partially update data	Update user name
DELETE	Delete data	Delete user

Q9. What is the difference between PUT and PATCH?

Interview Answer:

PUT is generally used to replace the complete resource, while PATCH is used to update specific fields of a resource.

Example:

Existing user:

```
{
 "name": "Manoj",
 "email": "manoj@gmail.com",
 "age": 22
}
```

PUT request:

```
{
 "name": "Rahul",
 "email": "rahul@gmail.com",
 "age": 24
}
```

PATCH request:

```
{
 "name": "Rahul"
}
```

PATCH मध्ये फक्त name update होऊ शकते. PUT मध्ये resource पूर्ण replace करण्याचा अर्थ असतो.

Q10. What are Route Parameters?

Interview Answer:

Route parameters are dynamic values passed through the URL. We access them using req.params.

```
app.get("/users/:id", (req, res) => {
  console.log(req.params.id);

  res.send("User details");
});
```

URL:

http://localhost:5000/users/101

Output:

101

मराठी:

URL मधील :id हा dynamic parameter आहे. त्याची value req.params.id वापरून मिळते.

Q11. What are Query Parameters?

Interview Answer:

Query parameters are optional values passed in the URL after the question mark. We access them using req.query.

```
app.get("/products", (req, res) => {
  console.log(req.query);
  res.send("Products");
});
```

URL:

/products?category=mobile&price=10000

Output:

```
{
  category: "mobile",
  price: "10000"
}
```

मराठी:

Query parameters चा वापर filtering, searching, sorting आणि pagination साठी केला जातो.

Q12. Difference between req.params, req.query and req.body?

Property	Purpose	Example
req.params	URL मधील dynamic values	/users/10
req.query	URL मधील query values	?page=2
req.body	Request सोबत पाठवलेला data	JSON body

Example:

```
app.post("/users/:id", (req, res) => {
  console.log(req.params.id);
  console.log(req.query.role);
  console.log(req.body.name);

  res.send("Success");
});
```

Request:

POST /users/101?role=admin

Body:

```
{
  "name": "Manoj"
}
```

3. Middleware in Express.js

Very Important

Q13. What is Middleware in Express.js?

Interview Answer:

Middleware is a function that runs between the incoming request and the final response. It can access the request, response, and next function.

मराठी स्पष्टीकरण:

Client request पाठवतो. ती request थेट controller कडे जाण्याऐवजी मध्ये काही functions मधून जाऊ शकते. या functions ना Middleware म्हणतात.

Middleware वापर:

- Authentication
- Authorization
- Logging
- Request validation
- Error handling
- File upload

Example:

```
const logger = (req, res, next) => {
  console.log(req.method, req.url);

  next();
};

app.use(logger);

app.get("/", (req, res) => {
  res.send("Home Page");
});
```

next() पुढील middleware किंवा route handler कडे control पाठवतो.

Q14. What is next() in Express.js?

Interview Answer:

The next() function passes control to the next middleware function in the request-response cycle.

```
const checkUser = (req, res, next) => {
```

```
 console.log("Middleware executed");


 next();
};
```

महत्त्वाचे: next() call केला नाही आणि response देखील पाठवला नाही, तर request अडकू शकते.

Q15. What are the types of Middleware?

Express.js मध्ये मुख्यतः पाच प्रकारचे middleware आहेत.

Type	Explanation
Application-level	Application वर लागू होणारे
Router-level	Specific router वर लागू होणारे
Built-in	Express ने दिलेले middleware
Third-party	External package मधील middleware
Error-handling	Errors handle करणारे middleware

Examples:

Application-level:

```
app.use((req, res, next) => {
 console.log("Application Middleware");
 next();
});
```

Router-level:

```
const router = express.Router();


router.use((req, res, next) => {
 console.log("Router Middleware");
 next();
});
```

Built-in:

```
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
```

Third-party:

```
const cors = require("cors");

app.use(cors());
```

Error-handling:

```
app.use((err, req, res, next) => {
  res.status(500).json({
    message: "Internal Server Error"
  });
});
```

Q16. What is the difference between app.use() and app.get()?

app.use()	**app.get()**
Used to register middleware	Used to handle GET requests
Can work with multiple HTTP methods	Only handles GET
Can be applied to a path prefix	Matches a GET route
Commonly used for middleware	Commonly used for fetching data

```
app.use(express.json());

app.get("/users", (req, res) => {
  res.json([]);
});
```

Q17. What is express.json()?

Interview Answer:

express.json() is built-in middleware that parses incoming JSON request bodies and makes the data available in req.body.

```
app.use(express.json());


app.post("/users", (req, res) => {
  console.log(req.body);
  res.json(req.body);
});
```

मराठी:

Frontend कडून JSON data पाठवला जातो. express.json() तो parse करून req.body मध्ये उपलब्ध करून देतो.

Q18. What is express.urlencoded()?

It is built-in middleware used to parse incoming URL-encoded form data.

```
app.use(express.urlencoded({ extended: true }));
```

याचा वापर HTML form मधून आलेला data handle करण्यासाठी होतो.

Q19. What happens if we do not use express.json()?

Interview Answer:

If we do not use express.json(), Express will not automatically parse incoming JSON request bodies. So req.body may remain undefined for JSON requests.

4. Express Router and Project Structure

Q20. What is express.Router()?

Interview Answer:

express.Router() is used to create modular and separate route handlers. It helps organize routes in large applications.

Example:

routes/userRoutes.js

```
const express = require("express");


const router = express.Router();
```

```
router.get("/", (req, res) => {
  res.send("All Users");
});

router.post("/", (req, res) => {
  res.send("Create User");
});

module.exports = router;
```

server.js

```
const express = require("express");
const userRoutes = require("./routes/userRoutes");

const app = express();

app.use(express.json());

app.use("/api/users", userRoutes);

app.listen(5000);
```

आता routes:

GET /api/users

POST /api/users

Q21. Why do we use Router?

Interview Answer:

We use Express Router to separate routes into different files. It improves code organization, readability, reusability, and maintainability.

मराठी:

मोठ्या project मध्ये user, product, order, payment अशा वेगवेगळ्या routes असतात. सर्व routes एकाच server file मध्ये लिहिण्याऐवजी वेगळ्या files मध्ये ठेवतो.

Q22. Explain MVC Architecture.

Interview Answer:

MVC stands for Model, View, and Controller. It is a software design pattern that separates application logic into different parts.

Part	Responsibility
Model	Database structure and data operations
View	User interface
Controller	Business logic and request handling

MERN backend मध्ये साधारणपणे:

Client

|

Routes

|

Middleware

|

Controller

|

Model

|

MongoDB

उदाहरणार्थ, user registration:

1. Route registration API request receive करतो.
2. Middleware request validate करतो.
3. Controller registration logic execute करतो.
4. Model database मध्ये user create करतो.

5. Controller response पाठवतो.

5. Request and Response Handling

Q23. What is req in Express.js?

req represents the incoming HTTP request from the client.

It contains:

- req.params
- req.query
- req.body
- req.headers
- req.method
- req.url
- req.cookies (cookie-parser middleware वापरल्यास)

Q24. What is res in Express.js?

res represents the HTTP response that the server sends to the client.

Common methods:

```
res.send("Hello");


res.json({ name: "Manoj" });


res.status(200).json({ success: true });


res.redirect("/login");


res.sendFile("/path/to/file");
```

Q25. Difference between res.send() and res.json()?

res.send()	**res.json()**
Sends different types of responses	Sends JSON response

Can send strings, buffers, objects, etc.	Serializes data as JSON
Flexible response method	Specifically for JSON APIs

```
res.send("Hello World");


res.json({
  message: "Hello World"
});
```

Q26. What is res.status()?

It sets the HTTP status code of the response.

```
res.status(201).json({
  message: "User created successfully"
});
```

Common status codes:

Status	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized / Unauthenticated
403	Forbidden
404	Not Found
409	Conflict
422	Unprocessable Content
500	Internal Server Error

Q27. What is the difference between 401 and 403?

401 Unauthorized: User authentication is missing or invalid.

403 Forbidden: User is authenticated but does not have permission to access the resource.

उदाहरण:

- Login न केलेल्या user ने protected API access केली → 401.
- Normal user ने admin-only API access केली → 403.

6. Error Handling in Express.js

Q28. What is Error Handling?

Interview Answer:

Error handling is the process of detecting and managing errors in an application so that it does not crash unexpectedly and can send proper responses to clients.

मराठी:

API मध्ये database error, validation error, invalid ID किंवा server error येऊ शकतो. हे errors योग्य पद्धतीने handle करणे म्हणजे Error Handling.

Q29. What is Error-handling Middleware?

Interview Answer:

Error-handling middleware is a special middleware function that handles errors in an Express application. It has four parameters: err, req, res, and next.

```
app.use((err, req, res, next) => {
  console.error(err.message);

  res.status(err.statusCode || 500).json({
    success: false,
    message: err.message || "Internal Server Error"
  });
});
```

महत्त्वाचे: Error-handling middleware साधारणपणे सर्व routes आणि middleware नंतर register करतात.

Q30. How do you handle errors in async functions?

Express 5 मध्ये rejected promises आणि async route handlers मधून आलेले errors आपोआप error-handling middleware कडे forward होतात.

```
app.get("/users", async (req, res) => {
  const users = await User.find();


  res.json(users);
});
```

Express 4 मध्ये सामान्यतः try...catch आणि next(error) वापरतात किंवा async wrapper वापरतात.

```
app.get("/users", async (req, res, next) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (error) {
    next(error);
  }
});
```

Q31. What is the difference between throw and next(error)?

throw error	**next(error)**
Throws a JavaScript error	Passes an error to Express error middleware
Can be caught using try/catch	Triggers Express error handling
Used in normal JS logic	Used in Express middleware/routes

```
if (!user) {
  throw new Error("User not found");
}
```

Express 5 async route मध्ये हा error automatically forward होतो. Express 4 मध्ये catch करून next(error) करणे आवश्यक असते.

7. Authentication and Security

Important for MERN Interviews

Q32. What is Authentication?

Interview Answer:

Authentication is the process of verifying the identity of a user.

मराठी:

User खरोखर कोण आहे हे verify करणे म्हणजे Authentication.

Example: Login using email and password.

Q33. What is Authorization?

Interview Answer:

Authorization is the process of checking what resources or actions an authenticated user is allowed to access.

मराठी:

User ला कोणती permission आहे हे तपासणे म्हणजे Authorization.

Example:

- Admin – Add, update and delete products.
- Customer – View and order products.

Q34. What is JWT?

Interview Answer:

JWT stands for JSON Web Token. It is a compact token format commonly used to securely transfer claims between two parties. It is often used for authentication and authorization.

मराठी:

User login झाल्यावर server JWT generate करू शकतो. पुढील protected API requests मध्ये client हा token पाठवतो. Server token verify करून user ओळखतो.

JWT structure:

Header.Payload.Signature

Example:

```
const jwt = require("jsonwebtoken");
```

```
const token = jwt.sign(
 { userId: "123" },
 process.env.JWT_SECRET,
 { expiresIn: "1d" }
);
```

Token verify:

```
const decoded = jwt.verify(
 token,
 process.env.JWT_SECRET
);
```

Q35. What is bcrypt?

Interview Answer:

bcrypt is a password-hashing library used to securely hash passwords before storing them in a database.

```
const bcrypt = require("bcryptjs");


const hashedPassword = await bcrypt.hash(password, 10);
```

Compare password:

```
const isMatch = await bcrypt.compare(
 password,
 user.password
);
```

महत्त्वाचे: Password database मध्ये plain text स्वरूपात कधीही store करू नये.

Q36. What is CORS?

Interview Answer:

CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether a web page from one origin can access resources from another origin.

मराठी:

समजा React frontend चालतो:

http://localhost:5173

आणि Express backend चालतो:

http://localhost:5000

हे दोन वेगवेगळे origins आहेत. Browser मधील cross-origin requests साठी backend ला योग्य CORS policy configure करावी लागू शकते.

```
const cors = require("cors");


app.use(cors({
  origin: "http://localhost:5173",
  credentials: true
}));
```

Q37. What is Helmet in Express.js?

Helmet is a middleware package that helps secure Express applications by setting various HTTP security headers.

```
const helmet = require("helmet");


app.use(helmet());
```

Q38. What is Rate Limiting?

Interview Answer:

Rate limiting restricts the number of requests a client can make within a specific time period. It helps protect APIs from abuse and brute-force attacks.

```
npm install express-rate-limit

const rateLimit = require("express-rate-limit");


const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
```

```
  limit: 100
});

app.use("/api", limiter);
```

Q39. What is Cookie-parser?

Cookie-parser is middleware that parses cookies attached to incoming requests and makes them available in req.cookies.

```
npm install cookie-parser

const cookieParser = require("cookie-parser");

app.use(cookieParser());
```

Set cookie:

```
res.cookie("token", token, {
  httpOnly: true,
  secure: true,
  sameSite: "none"
});
```

Clear cookie:

```
res.clearCookie("token");
```

मराठी:

Cookies वापरून authentication token browser मध्ये ठेवता येतो. httpOnly cookie JavaScript मधून वाचता येत नाही. Production मध्ये secure आणि sameSite settings योग्य environment आणि deployment नुसार configure कराव्या लागतात.

8. Express.js Advanced Concepts

Q40. What is REST API?

Interview Answer:

REST stands for Representational State Transfer. It is an architectural style used to design web APIs around resources and standard HTTP methods.

Example:

GET /api/users

GET /api/users/:id

POST /api/users

PUT /api/users/:id

PATCH /api/users/:id

DELETE /api/users/:id

Q41. What is API Versioning?

API versioning means maintaining different versions of an API to support changes without breaking existing clients.

/api/v1/users

/api/v2/users

मराठी:

API मध्ये मोठे changes केले आणि जुन्या frontend ला breaking changes झाले, तर versioning मुळे जुनी आणि नवीन API वेगवेगळी ठेवता येते.

Q42. What is File Upload in Express.js?

Interview Answer:

File uploading in Express.js is commonly handled using middleware such as Multer. It processes multipart/form-data requests.

npm install multer

```
const multer = require("multer");


const upload = multer({
  dest: "uploads/"
});


app.post("/upload", upload.single("image"), (req, res) => {
  res.json({
    message: "File uploaded",
```

```
  file: req.file
 });
});
```

मराठी:

Frontend मधून image किंवा document पाठवण्यासाठी multipart/form-data वापरतो. Multer request मधील file process करतो.

Production मध्ये file validation, size limits, file type checks आणि सुरक्षित storage महत्त्वाचे आहेत. Cloudinary किंवा object storage वापरता येते.

Q43. What is Pagination?

Interview Answer:

Pagination is the process of dividing a large amount of data into smaller pages. It improves API performance and user experience.

Example:

```
app.get("/users", async (req, res) => {
 const page = Number(req.query.page) || 1;
 const limit = Number(req.query.limit) || 10;

 const skip = (page - 1) * limit;

 const users = await User.find()
  .skip(skip)
  .limit(limit);

 res.json(users);
});
```

Example API:

/api/users?page=1&limit=10

Formula:

\[skip = (page - 1) \times limit \]

Q44. What is API Validation?

Interview Answer:

API validation checks whether incoming request data follows the required rules before processing it.

Example:

```
if (!req.body.email) {
  return res.status(400).json({
    message: "Email is required"
  });
}
```

Production मध्ये express-validator किंवा Zod सारख्या validation libraries वापरू शकतो.

Q45. What is the difference between Authentication Middleware and Authorization Middleware?

Authentication	Authorization
Checks user identity	Checks user permissions
Verifies token/session	Checks roles or permissions
Usually runs first	Usually runs after authentication
Example: Is user logged in?	Example: Is user admin?

Example:

```
const verifyToken = (req, res, next) => {
  // Verify token
  next();
};


const isAdmin = (req, res, next) => {
  if (req.user.role !== "admin") {
    return res.status(403).json({
```

```
    message: "Access denied"
   });
 }

 next();
};

app.delete(
 "/api/products/:id",
 verifyToken,
 isAdmin,
 deleteProduct
);
```

Q46. What is the difference between app.listen() and http.createServer()?

app.listen() is a convenient method to start an Express application.

```
app.listen(5000);
```

http.createServer() creates a Node.js HTTP server explicitly.

```
const http = require("http");

const server = http.createServer(app);

server.listen(5000);
```

Explicit HTTP server creation is useful when integrating features such as Socket.IO.

Q47. How do you connect Express.js with MongoDB?

Interview Answer:

We commonly use Mongoose to connect an Express.js application with MongoDB. Mongoose provides schemas, models, validation, and database operations.

```
npm install mongoose
```

```
const mongoose = require("mongoose");


mongoose.connect(process.env.MONGO_URI)
 .then(() => {
  console.log("MongoDB Connected");
 })
 .catch((error) => {
  console.log(error);
 });
```

9. Industry-Level Express.js Project Structure

मोठ्या MERN project मध्ये code व्यवस्थित ठेवण्यासाठी खालील structure वापरू शकतो.

```
backend/
│
├── config/
│   └── db.js
│
├── controllers/
│   └── userController.js
│
├── middleware/
│   ├── authMiddleware.js
│   └── errorMiddleware.js
│
├── models/
│   └── userModel.js
│
├── routes/
```

```
│   └── userRoutes.js
│
├── utils/
│   └── generateToken.js
│
├── .env
├── server.js
└── package.json
```

प्रत्येक folder चे काम:

Folder	Responsibility
config	Database आणि app configuration
controllers	Business logic
middleware	Authentication, validation, error handling
models	Database schemas आणि models
routes	API endpoints
utils	Reusable helper functions

Q48. What is the difference between Controller and Middleware?

Interview Answer:

Middleware performs common operations during the request-response cycle, such as authentication and validation. A controller handles the main business logic and sends the response.

Example:

```
// Middleware
const checkAuth = (req, res, next) => {
  console.log("Checking authentication");
  next();
};
```

```
// Controller
const getUsers = async (req, res) => {
  const users = await User.find();


  res.status(200).json(users);
};
```

Q49. What is Environment Variable?

Interview Answer:

Environment variables store configuration values outside the source code. They are commonly used for database URLs, API keys, and secret keys.

```
npm install dotenv
```

.env

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/mydb
JWT_SECRET=mysecret
```

server.js

```
require("dotenv").config();


const PORT = process.env.PORT || 5000;
```

महत्त्वाचे: .env file GitHub वर push करू नये. ती .gitignore मध्ये add करावी.

Q50. How do you improve Express.js API performance?

Interview Answer:

We can improve API performance by using pagination, database indexes, caching, compression, efficient queries, and proper error handling.

Important practices:

1. Use pagination for large datasets.
2. Add proper MongoDB indexes.

3. Avoid unnecessary database queries.
4. Use caching where appropriate.
5. Use compression middleware when beneficial.
6. Avoid blocking operations.
7. Apply rate limiting.
8. Monitor API response times.

10. Output-Based Interview Questions

हे प्रश्न interview मध्ये JavaScript आणि Express concepts समजले आहेत का हे तपासण्यासाठी विचारले जाऊ शकतात.

Q51. What will be the output?

```
app.get("/test", (req, res) => {
 console.log("Route 1");
 res.send("Hello");
});

app.get("/test", (req, res) => {
 console.log("Route 2");
 res.send("World");
});
```

Answer:

Route 1

Browser response:

Hello

Why?

पहिला matching route response पाठवतो. त्यानंतर दुसरा route execute होत नाही.

Q52. What will happen here?

```
app.use((req, res, next) => {
 console.log("Middleware 1");
```

```
 next();
});

app.get("/", (req, res) => {
 console.log("Route");
 res.send("Done");
});
```

Output:

Middleware 1

Route

Browser response: Done

Q53. What happens if next() is missing?

```
app.use((req, res, next) => {
 console.log("Middleware");
});

app.get("/", (req, res) => {
 res.send("Home");
});
```

Answer:

Request अडकू शकते. कारण middleware ने next() call केले नाही आणि response देखील पाठवला नाही.

Q54. What is the output?

```
app.get("/user/:id", (req, res) => {
 res.json({
  id: req.params.id,
  page: req.query.page
 });
```

```
});
```

URL:

/user/101?page=2

Output:

```
{
  "id": "101",
  "page": "2"
}
```

11. Practical Coding Interview Questions

Q55. Create a simple CRUD API using Express.js.

```
const express = require("express");

const app = express();

app.use(express.json());

let users = [
 { id: 1, name: "Manoj" },
 { id: 2, name: "Rahul" }
];

// GET - All Users
app.get("/users", (req, res) => {
  res.status(200).json(users);
});

// GET - Single User
app.get("/users/:id", (req, res) => {
```

```
  const user = users.find(
    (u) => u.id === Number(req.params.id)
  );

  if (!user) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  res.json(user);
});

// POST - Create User
app.post("/users", (req, res) => {
  const newUser = {
    id: Date.now(),
    name: req.body.name
  };

  users.push(newUser);

  res.status(201).json(newUser);
});

// PUT - Update User
app.put("/users/:id", (req, res) => {
  const user = users.find(
```

```
  (u) => u.id === Number(req.params.id)
 );

 if (!user) {
  return res.status(404).json({
   message: "User not found"
  });
 }

 user.name = req.body.name;

 res.json(user);
});

// DELETE - Delete User
app.delete("/users/:id", (req, res) => {
 const userId = Number(req.params.id);

 const initialLength = users.length;

 users = users.filter((u) => u.id !== userId);

 if (users.length === initialLength) {
  return res.status(404).json({
   message: "User not found"
  });
 }
```

```
 res.json({
  message: "User deleted successfully"
 });
});

app.listen(5000, () => {
 console.log("Server running on port 5000");
});
```

टीप: हे फक्त Express CRUD समजण्यासाठी in-memory example आहे. Actual MERN project मध्ये MongoDB आणि Mongoose वापरून data persist करावा.

Q56. How do you create a protected route?

```
const verifyToken = (req, res, next) => {
 const token = req.headers.authorization?.split(" ")[1];

 if (!token) {
  return res.status(401).json({
   message: "Token not provided"
  });
 }

 try {
  const decoded = jwt.verify(
   token,
   process.env.JWT_SECRET
  );

  req.user = decoded;
```

```
    next();
  } catch (error) {
    return res.status(401).json({
      message: "Invalid or expired token"
    });
  }
};

app.get("/profile", verifyToken, (req, res) => {
  res.json({
    message: "Welcome",
    user: req.user
  });
});
```

12. Common Express.js Interview Questions

Question	Short Answer
What is Express?	Node.js web framework
What is middleware?	Function between request and response
What is next()?	Passes control to next middleware
What is Router?	Modular route handler
What is req.body?	Request body data
What is req.params?	URL parameter values
What is req.query?	Query string values
What is CORS?	Browser cross-origin access mechanism
What is JWT?	Token format for claims
What is MVC?	Model, View, Controller pattern

What is REST?	API architectural style
What is Multer?	File upload middleware
What is Helmet?	HTTP security headers middleware
What is Rate Limiting?	Restricts request frequency
What is dotenv?	Loads environment variables
What is express.json()?	JSON body parsing middleware
What is express.Router()?	Creates modular routes
What is Error Middleware?	Handles application errors

MongoDB + Mongoose Interview Questions & Answers – Beginner to Advanced

1. MongoDB Fundamentals

Q1. What is MongoDB?

Interview Answer:

MongoDB is a NoSQL, document-oriented database. It stores data in flexible, JSON-like documents called BSON. It is commonly used to build scalable and flexible applications.

मराठी स्पष्टीकरण:

MongoDB हा NoSQL database आहे. यामध्ये data tables आणि rows मध्ये न ठेवता Collections आणि Documents मध्ये store केला जातो.

उदाहरणार्थ, आपल्या User Management System मध्ये प्रत्येक user ची माहिती एका document मध्ये store होते.

```
{
 "_id": "101",
 "name": "Manoj",
 "email": "manoj@gmail.com",
 "age": 22,
 "role": "developer"
}
```

Q2. What is a NoSQL Database?

Interview Answer:

A NoSQL database is a database that uses data models other than the traditional relational table model. It supports flexible data structures and different ways of storing and accessing data.

मराठी:

Traditional SQL database मध्ये tables, rows आणि columns असतात. MongoDB सारख्या NoSQL database मध्ये documents आणि collections असतात.

उदाहरण:

SQL Database	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary Key	_id
JOIN	$lookup / references

Q3. What is the difference between SQL and NoSQL?

SQL	NoSQL
Relational database	Non-relational database
Tables and rows	Collections and documents
Usually fixed schema	Flexible document structure
SQL query language	Database-specific query methods
Relationships often use foreign keys	Embedding and references
Example: MySQL	Example: MongoDB

Interview Answer:

SQL databases store data in tables with predefined structures, while NoSQL databases like MongoDB store data in flexible documents. The choice depends on application requirements.

Q4. What is BSON?

Interview Answer:

BSON stands for Binary JSON. It is a binary-encoded document format used by MongoDB to store data.

मराठी:

JSON सारखाच BSON असतो, पण BSON binary format मध्ये data store करतो. BSON मध्ये Date, ObjectId आणि इतर data types सुद्धा support होतात.

Example:

```
{
 name: "Manoj",
 age: 22,
 createdAt: new Date(),
 isActive: true
}
```

Q5. What is a Document in MongoDB?

Interview Answer:

A document is a single record in MongoDB. It stores data in key-value pairs.

```
{
 "_id": 1,
 "name": "Manoj",
 "email": "manoj@gmail.com"
}
```

मराठी: Collection मधील प्रत्येक individual record म्हणजे Document.

Q6. What is a Collection?

Interview Answer:

A collection is a group of MongoDB documents, similar to a table in a relational database.

Example:

Database: ecommerce

Collections:

- users
- products
- orders
- payments

मराठी: एका collection मध्ये एकाच प्रकारच्या entities शी संबंधित अनेक documents असतात.

Q7. What is the difference between Collection and Document?

Collection	Document
Group of documents	Single record
Similar to SQL table	Similar to SQL row
Contains multiple records	Contains fields and values
Example: users	Example: Manoj's user record

Q8. What is _id in MongoDB?

Interview Answer:

_id is a unique identifier for every document in a MongoDB collection. MongoDB automatically creates an ObjectId if we do not provide one.

Example:

```
{
  "_id": "671a1234567890abcdef1234",
  "name": "Manoj"
}
```

मराठी:

प्रत्येक document ला unique identity देण्यासाठी _id वापरला जातो. एका collection मध्ये दोन documents चा _id समान असू शकत नाही.

Q9. What is ObjectId?

Interview Answer:

ObjectId is a BSON data type commonly used as MongoDB's default document identifier. It is a 12-byte value that is designed to be unique.

```
const mongoose = require("mongoose");


const id = new mongoose.Types.ObjectId();


console.log(id);
```

Q10. Is MongoDB schema-less?

Interview Answer:

MongoDB is schema-flexible, not completely schema-less. Documents in a collection can have different structures, but we can enforce rules using schema validation.

मराठी:

MongoDB मध्ये प्रत्येक document ची structure सारखीच असणे आवश्यक नाही. पण production applications मध्ये data consistency साठी validation rules वापरतो.

2. MongoDB Installation and Basic Commands

Q11. How do you install MongoDB?

MongoDB वापरण्यासाठी मुख्य पर्याय:

1. MongoDB Community Server – Local computer वर database.
2. MongoDB Atlas – Cloud database.
3. MongoDB Compass – GUI tool.
4. mongosh – Command-line shell.

Useful links:

- MongoDB Community Server
- MongoDB Compass
- MongoDB Atlas

Q12. What is MongoDB Shell?

Interview Answer:

MongoDB Shell, also called mongosh, is a command-line interface used to interact with MongoDB databases.

Example:

mongosh

Q13. How do you create a database?

use college

मराठी: use command database निवडण्यासाठी वापरतो. Database प्रत्यक्षात data insert झाल्यावर तयार होतो.

Q14. How do you show all databases?

show dbs

Q15. How do you create a collection?

db.createCollection("users")

Q16. How do you show all collections?

show collections

Q17. How do you delete a collection?

db.users.drop()

Q18. How do you delete a database?

db.dropDatabase()

3. CRUD Operations in MongoDB

Very Important

CRUD stands for:

- Create – Insert data
- Read – Fetch data
- Update – Modify data
- Delete – Remove data

Q19. What is insertOne()?

Interview Answer:

insertOne() is used to insert a single document into a MongoDB collection.

```
db.users.insertOne({
  name: "Manoj",
  email: "manoj@gmail.com",
  age: 22
});
```

Q20. What is insertMany()?

It inserts multiple documents into a collection.

```
db.users.insertMany([
 {
   name: "Manoj",
   age: 22
 },
 {
   name: "Rahul",
   age: 23
 },
 {
```

```
  name: "Amit",
  age: 24
 }
]);
```

Q21. What is find()?

Interview Answer:

find() is used to retrieve multiple documents from a collection that match the given condition.

```
db.users.find();
```

Find users with age 22:

```
db.users.find({
 age: 22
});
```

Q22. What is findOne()?

Interview Answer:

findOne() returns the first document that matches the given query condition.

```
db.users.findOne({
 email: "manoj@gmail.com"
});
```

Q23. Difference between find() and findOne()?

find()	**findOne()**
Returns a cursor	Returns one matching document or null
Can return multiple documents	Returns only one document
Used for multiple records	Used for a single record

Q24. What is updateOne()?

Interview Answer:

updateOne() updates the first document that matches the given condition.

```
db.users.updateOne(
```

```
 { name: "Manoj" },
 {
   $set: {
     age: 23
   }
 }
);
```

Q25. What is updateMany()?

It updates all documents that match the given condition.

```
db.users.updateMany(
 { role: "student" },
 {
   $set: {
     status: "active"
   }
 }
);
```

Q26. What is $set?

$set is an update operator used to set the value of specific fields.

```
db.users.updateOne(
 { name: "Manoj" },
 {
   $set: {
     city: "Kolhapur"
   }
 }
);
```

Q27. What is deleteOne()?

It deletes the first document that matches the condition.

```
db.users.deleteOne({
  name: "Manoj"
});
```

Q28. What is deleteMany()?

It deletes all documents that match the given condition.

```
db.users.deleteMany({
  status: "inactive"
});
```

Q29. What is the difference between updateOne() and replaceOne()?

Interview Answer:

updateOne() modifies specific fields using update operators, while replaceOne() replaces the complete document except for its _id.

Example:

```
db.users.updateOne(
  { name: "Manoj" },
  { $set: { age: 23 } }
);
```

Replace:

```
db.users.replaceOne(
  { name: "Manoj" },
  {
    name: "Rahul",
    age: 24
  }
);
```

4. MongoDB Query Operators

Q30. What are Query Operators?

Interview Answer:

Query operators are special operators used to filter documents based on specific conditions.

मराठी: Database मधून आपल्याला आवश्यक असलेला data शोधण्यासाठी operators वापरतो.

Comparison Operators

Operator	Meaning
$eq	Equal
$ne	Not equal
$gt	Greater than
$gte	Greater than or equal
$lt	Less than
$lte	Less than or equal
$in	Matches any value in an array
$nin	Matches none of the values in an array

Examples:

Greater than:

```
db.products.find({
  price: { $gt: 1000 }
});
```

Less than:

```
db.products.find({
  price: { $lt: 500 }
});
```

Greater than or equal:

```
db.users.find({
  age: { $gte: 18 }
});
```

Using $in:

```
db.users.find({
  role: {
    $in: ["admin", "manager"]
  }
});
```

Q31. What are Logical Operators?

Operator	Meaning
$and	All conditions must match
$or	At least one condition must match
$not	Negates a condition
$nor	None of the conditions should match

Example $and:

```
db.users.find({
  $and: [
    { age: { $gte: 18 } },
    { city: "Kolhapur" }
  ]
});
```

Example $or:

```
db.users.find({
  $or: [
    { city: "Kolhapur" },
    { city: "Pune" }
  ]
});
```

Q32. What is $exists?

It checks whether a particular field exists in a document.

```
db.users.find({
  phone: { $exists: true }
});
```

Q33. What is $regex?

Interview Answer:

$regex is used to perform pattern matching on string fields.

```
db.users.find({
  name: {
    $regex: "^man",
    $options: "i"
  }
});
```

मराठी:

Search functionality मध्ये नाव शोधण्यासाठी regex वापरू शकतो.

i म्हणजे case-insensitive search.

5. Projection, Sorting and Pagination

Q34. What is Projection?

Interview Answer:

Projection is used to include or exclude specific fields from the returned documents.

```
db.users.find(
  {},
  {
    name: 1,
    email: 1,
    _id: 0
  }
```

);

मराठी: आपल्याला सर्व fields नको असतील तर फक्त आवश्यक fields fetch करण्यासाठी projection वापरतो.

Q35. What is sort()?

It sorts documents in ascending or descending order.

// Ascending

db.users.find().sort({ age: 1 });

// Descending

db.users.find().sort({ age: -1 });

- 1 = Ascending
- -1 = Descending

Q36. What is limit()?

It restricts the number of documents returned.

db.users.find().limit(5);

Q37. What is skip()?

It skips a specified number of documents.

db.users.find().skip(10);

Q38. How does Pagination work in MongoDB?

Interview Answer:

Pagination divides a large result set into smaller pages using skip() and limit().

Formula:

\[skip = (page - 1) \times limit \]

Example:

const page = 2;

const limit = 10;

db.products.find()

```
  .skip((page - 1) * limit)
  .limit(limit);
```

मराठी:

समजा एका page वर 10 products दाखवायचे आहेत. दुसऱ्या page वर पहिल्या 10 products skip करून पुढील 10 products दाखवतो.

मोठ्या collections साठी cursor-based pagination देखील उपयोगी ठरू शकते.

6. Indexing in MongoDB

Important Advanced Topic

Q39. What is Indexing in MongoDB?

Interview Answer:

An index is a data structure that improves the speed of query operations by helping MongoDB find documents more efficiently.

मराठी स्पष्टीकरण:

समजा आपल्या database मध्ये 10 लाख users आहेत. आपल्याला एका specific email वर user शोधायचा आहे.

Index नसेल तर database ला अनेक documents तपासावे लागू शकतात. Index असल्यास search अधिक efficient होऊ शकतो.

Example:

```
db.users.createIndex({
  email: 1
});
```

Q40. What are the types of Indexes in MongoDB?

Index	**Purpose**
Single Field	Index on one field
Compound	Index on multiple fields
Multikey	Index on array values
Text	Text search

Geospatial	Location-based queries
Unique	Prevents duplicate indexed values
TTL	Automatically expires documents after a configured time

Q41. What is a Unique Index?

Interview Answer:

A unique index ensures that no two documents have the same value for the indexed field.

```
db.users.createIndex(
 { email: 1 },
 { unique: true }
);
```

मराठी: दोन users चा email same असू नये यासाठी unique index वापरतो.

Q42. What is a Compound Index?

A compound index is an index created on multiple fields.

```
db.users.createIndex({
  city: 1,
  age: -1
});
```

मराठी: दोन किंवा अधिक fields एकत्र search किंवा sort करण्यासाठी compound index उपयोगी ठरतो. Field order महत्त्वाचा असतो.

Q43. What is the disadvantage of Indexing?

Interview Answer:

Indexes improve read performance, but they require extra storage and can slow down insert, update, and delete operations because indexes also need to be maintained.

मराठी: प्रत्येक field वर index बनवणे योग्य नाही. गरजेनुसार index तयार करावा.

Q44. What is explain()?

It provides information about how MongoDB executes a query.

```
db.users.find({
```

email: "manoj@gmail.com"

}).explain("executionStats");

It helps us understand:

- Query execution plan
- Index usage
- Documents examined
- Execution time statistics

7. Aggregation Framework

Very Important for Experienced Interviews

Q45. What is Aggregation in MongoDB?

Interview Answer:

Aggregation is a framework in MongoDB used to process documents through multiple stages and perform operations such as filtering, grouping, sorting, and calculating values.

मराठी:

समजा Expense Tracker मध्ये आपल्याला प्रत्येक महिन्याचा total expense शोधायचा आहे. प्रत्येक document manually calculate करण्याऐवजी Aggregation वापरतो.

Example documents:

```
[
 {
  "category": "Food",
  "amount": 500
 },
 {
  "category": "Travel",
  "amount": 300
 },
 {
```

```
  "category": "Food",
  "amount": 200
 }
]
```

Aggregation:

```
db.expenses.aggregate([
 {
  $group: {
   _id: "$category",
   totalAmount: {
    $sum: "$amount"
   }
  }
 }
]);
```

Output:

```
[
 {
  "_id": "Food",
  "totalAmount": 700
 },
 {
  "_id": "Travel",
  "totalAmount": 300
 }
]
```

Q46. What is Aggregation Pipeline?

Interview Answer:

An aggregation pipeline is a sequence of stages where the output of one stage becomes the input of the next stage.

मराठी:

एकामागून एक data processing steps execute होतात.

Documents

|

$match

|

$group

|

$sort

|

$project

|

Final Result

Q47. What is $match?

It filters documents based on specified conditions.

```
db.products.aggregate([
 {
   $match: {
     price: { $gt: 1000 }
   }
 }
]);
```

Q48. What is $group?

It groups documents based on a specified field and performs calculations.

```
db.orders.aggregate([
 {
```

```
  $group: {
    _id: "$status",
    totalOrders: { $sum: 1 }
  }
 }
]);
```

Q49. What is $sum?

It calculates the sum of numeric values.

```
{
 $group: {
  _id: "$category",
  total: { $sum: "$amount" }
 }
}
```

Q50. What is $project?

It includes, excludes, or reshapes fields in aggregation results.

```
db.users.aggregate([
 {
  $project: {
    name: 1,
    email: 1,
    _id: 0
  }
 }
]);
```

Q51. What is $sort?

It sorts documents in an aggregation pipeline.

```
db.products.aggregate([
```

```
 {
  $sort: {
   price: -1
  }
 }
]);
```

Q52. What is $lookup?

Interview Answer:

$lookup is an aggregation stage used to perform a left outer join between documents from two collections.

मराठी:

SQL मध्ये JOIN असतो. MongoDB मध्ये वेगवेगळ्या collections मधील related data जोडण्यासाठी $lookup वापरता येतो.

Example:

```
db.orders.aggregate([
 {
  $lookup: {
   from: "users",
   localField: "userId",
   foreignField: "_id",
   as: "userDetails"
  }
 }
]);
```

यामध्ये:

- from – कोणत्या collection मधून data आणायचा.
- localField – Current collection मधील field.
- foreignField – दुसऱ्या collection मधील matching field.

- as – Result कोणत्या field मध्ये ठेवायचा.

Q53. What is the difference between $match and find()?

find()	$match
Used in normal queries	Used inside aggregation pipeline
Returns matching documents	Filters documents between pipeline stages
Supports query filtering	Supports aggregation processing

Q54. What is $unwind?

It deconstructs an array field and outputs a separate document for each array element.

Example:

```
{
  "name": "Manoj",
  "skills": ["React", "Node", "MongoDB"]
}
```

Query:

```
db.users.aggregate([
 {
   $unwind: "$skills"
 }
]);
```

Output: तीन documents तयार होतील, प्रत्येकामध्ये एक skill असेल.

8. Relationships in MongoDB

Q55. How do you create relationships in MongoDB?

Interview Answer:

MongoDB supports relationships using embedded documents and references. We choose the approach based on the data structure and application requirements.

मराठी:

दोन collections मधील related data manage करण्यासाठी दोन मुख्य approaches आहेत.

1. Embedding

एकाच document मध्ये related data ठेवणे.

```
{
 "name": "Manoj",
 "address": {
  "city": "Kolhapur",
  "pincode": "416001"
 }
}
```

2. Referencing

एका document मधून दुसऱ्या document चा reference ठेवणे.

```
{
 "name": "Manoj",
 "departmentId": "101"
}
```

कधी काय वापरायचे?

Embedding	**Referencing**
Related data एकत्र वाचायचा असेल	Related data स्वतंत्रपणे वापरला जात असेल
Data size मर्यादित असेल	Data मोठा किंवा स्वतंत्र असेल
Read performance महत्त्वाची असेल	अनेक documents मध्ये sharing असेल
One-to-few	One-to-many / many-to-many cases

Q56. What is the difference between Embedding and Referencing?

Embedding stores related data inside the same document, while referencing stores the related document separately and connects it using an identifier.

Q57. What is populate() in Mongoose?

Interview Answer:

populate() is a Mongoose method that replaces specified reference fields with documents from another collection.

Example:

```
const orderSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
});
```

Query:

```
const orders = await Order.find()
  .populate("userId");
```

मराठी:

Order मध्ये फक्त userId store केला आहे. पण .populate() केल्यावर त्या user ची संबंधित माहिती मिळवता येते.

9. Mongoose Fundamentals

MERN Developer Must Know

Q58. What is Mongoose?

Interview Answer:

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It provides schemas, models, validation, middleware, and query helpers.

मराठी:

Node.js आणि MongoDB यांच्यामध्ये काम सोपे करण्यासाठी Mongoose वापरतो.

Mongoose च्या मदतीने:

- Schema define करता येतो.
- Model create करता येतो.
- Validation करता येते.

- CRUD operations करता येतात.
- Relationships populate करता येतात.
- Middleware/hooks वापरता येतात.

Q59. What is the difference between MongoDB and Mongoose?

MongoDB	Mongoose
Database	ODM library
Stores documents	Helps interact with MongoDB
Supports flexible documents	Provides schema-based modeling
Database engine	Node.js library
Uses MongoDB query language	Provides models and methods

Interview Answer:

MongoDB is the database, while Mongoose is a library that helps Node.js applications work with MongoDB using schemas and models.

Q60. What is Schema in Mongoose?

Interview Answer:

A schema defines the structure, data types, default values, and validation rules for documents in a MongoDB collection.

Example:

```
const mongoose = require("mongoose");


const userSchema = new mongoose.Schema({
 name: {
  type: String,
  required: true
 },


 email: {
```

```
  type: String,
  required: true,
  unique: true
 },

 age: {
  type: Number,
  min: 18
 }
});
```

Q61. What is a Model in Mongoose?

Interview Answer:

A model is a constructor created from a Mongoose schema. It provides an interface to create, read, update, and delete documents in a collection.

```
const User = mongoose.model("User", userSchema);
```

मराठी:

Schema म्हणजे structure आणि rules. Model म्हणजे त्या schema च्या आधारे database सोबत operations करण्यासाठी तयार केलेला interface.

Q62. Difference between Schema and Model?

Schema	**Model**
Defines structure	Performs database operations
Defines validation	Creates and retrieves documents
Blueprint	Working interface
Created using new mongoose.Schema()	Created using mongoose.model()

Q63. How do you create a document using Mongoose?

```
const user = await User.create({
 name: "Manoj",
```

```
 email: "manoj@gmail.com",
 age: 22
});
```

Q64. What is save() in Mongoose?

save() saves a new document or persists changes made to an existing document.

```
const user = new User({
 name: "Manoj",
 email: "manoj@gmail.com"
});

await user.save();
```

Q65. Difference between create() and save()?

create()	**save()**
Model method	Document method
Creates and saves document	Saves a document instance
Shorter syntax	Useful when modifying document before saving

Q66. What is findById()?

It finds a document using its _id.

```
const user = await User.findById("671a1234567890abcdef1234");
```

Q67. What is findByIdAndUpdate()?

It finds a document by ID and updates it.

```
const user = await User.findByIdAndUpdate(
 req.params.id,
 {
  name: req.body.name
 },
 {
```

```
  new: true,
  runValidators: true
 }
);
```

Important:

- new: true – Returns the updated document.
- runValidators: true – Runs schema validators for the update.

Q68. What is findByIdAndDelete()?

It finds a document by ID and deletes it.

```
const user = await User.findByIdAndDelete(
 req.params.id
);
```

10. Mongoose Validation

Q69. What is Validation in Mongoose?

Interview Answer:

Validation is the process of checking whether document data follows the rules defined in a schema before saving it to the database.

Example:

```
const userSchema = new mongoose.Schema({
 name: {
  type: String,
  required: true,
  trim: true
 },

 email: {
  type: String,
  required: true,
```

```
  lowercase: true
 },

 age: {
  type: Number,
  min: 18,
  max: 60
 }
});
```

Q70. What is required in Mongoose?

It makes a field mandatory.

```
email: {
 type: String,
 required: true
}
```

Q71. What is enum in Mongoose?

It restricts a field to a predefined set of allowed values.

```
role: {
 type: String,
 enum: ["admin", "user", "manager"],
 default: "user"
}
```

Q72. What is default in Mongoose?

It defines a default value when no value is provided.

```
status: {
 type: String,
 default: "active"
}
```

Q73. What is the difference between unique and required?

- required: true – Field must be present.
- unique: true – Creates a unique index to prevent duplicate indexed values.

महत्त्वाचे: Mongoose मधील unique हे सामान्य validator नाही. Duplicate key error database कडून येऊ शकतो. required आणि unique या वेगळ्या गोष्टी आहेत.

11. Mongoose Middleware and Hooks

Q74. What is Mongoose Middleware?

Interview Answer:

Mongoose middleware is a function that runs before or after certain database operations. It is also called middleware or hooks.

मराठी:

Database operation होण्यापूर्वी किंवा झाल्यानंतर काही logic execute करायचे असल्यास hooks वापरतो.

Example:

```
userSchema.pre("save", function () {
  console.log("Before saving user");
});
```

Q75. What is pre() Middleware?

pre() runs before a specified operation.

```
userSchema.pre("save", function () {
  console.log("Saving user...");
});
```

Q76. What is post() Middleware?

post() runs after a specified operation.

```
userSchema.post("save", function (doc) {
  console.log("User saved:", doc.name);
});
```

Q77. How do you hash a password using Mongoose middleware?

```
const bcrypt = require("bcryptjs");


userSchema.pre("save", async function () {
 if (!this.isModified("password")) {
  return;
 }

 this.password = await bcrypt.hash(
  this.password,
  10
 );
});
```

स्पष्टीकरण:

- pre("save") – Document save होण्याआधी execute होते.
- this.isModified("password") – Password बदलला आहे का ते तपासते.
- bcrypt.hash() – Password hash करते.
- this.password – Hashed password assign करते.

हा hook findByIdAndUpdate() सारख्या query update methods वर आपोआप चालत नाही. अशा updates साठी स्वतंत्र सुरक्षित logic आवश्यक असते.

12. Advanced MongoDB Concepts

Q78. What is Replication in MongoDB?

Interview Answer:

Replication is the process of maintaining multiple copies of data across different MongoDB servers using a replica set. It helps provide high availability and data redundancy.

मराठी:

एका server मध्ये problem आली तर दुसऱ्या server कडे data उपलब्ध राहण्यासाठी replication वापरतो.

Replica Set मध्ये:

- Primary – Writes स्वीकारतो.
- Secondary – Primary कडील changes replicate करतो.
- Arbiters – काही configurations मध्ये voting साठी वापरला जाऊ शकतो; तो data store करत नाही.

Q79. What is Sharding in MongoDB?

Interview Answer:

Sharding is a method of distributing data across multiple machines to handle large datasets and high-throughput workloads.

मराठी:

Database मधील खूप मोठा data वेगवेगळ्या servers मध्ये distribute करणे म्हणजे Sharding.

Q80. Difference between Replication and Sharding?

Replication	**Sharding**
Maintains copies of data	Distributes data
Improves availability	Supports horizontal scaling
Replica set	Sharded cluster
Multiple copies	Different data partitions

Q81. What is a Replica Set?

A replica set is a group of MongoDB servers that maintain the same data set and provide redundancy and high availability.

Q82. What is a Transaction in MongoDB?

Interview Answer:

A transaction is a group of database operations that are executed as one logical unit. MongoDB supports multi-document transactions with ACID properties in supported deployments.

मराठी:

समजा bank transfer मध्ये:

1. Account A मधून ₹500 कमी करायचे.
2. Account B मध्ये ₹500 add करायचे.

पहिली operation success आणि दुसरी fail झाली, तर data inconsistent होऊ शकतो. Transaction वापरून दोन्ही operations एकत्र commit किंवा abort करू शकतो.

Q83. What are ACID Properties?

Property	Meaning
Atomicity	All operations succeed or fail together
Consistency	Data remains valid
Isolation	Concurrent transactions are isolated
Durability	Committed changes persist

Q84. What is TTL Index?

Interview Answer:

TTL stands for Time To Live. A TTL index automatically removes documents after a specified period of time.

```
db.sessions.createIndex(
 { createdAt: 1 },
 { expireAfterSeconds: 3600 }
);
```

मराठी: Temporary sessions किंवा expiry असलेला data automatically remove करण्यासाठी वापरता येतो. TTL deletion background process द्वारे होते; exact वेळेला deletion होईलच असे नाही.

Q85. What is Text Index?

It is an index that supports text search on string content.

```
db.products.createIndex({
  name: "text",
  description: "text"
});
```

Search:

```
db.products.find({
  $text: {
    $search: "mobile phone"
  }
});
```

Q86. What is MongoDB Compass?

Interview Answer:

MongoDB Compass is a graphical user interface that allows developers to view, query, analyze, and manage MongoDB databases without using only command-line commands.

Q87. What is MongoDB Atlas?

Interview Answer:

MongoDB Atlas is a fully managed cloud database service for MongoDB. It allows developers to deploy, manage, monitor, and scale MongoDB databases in the cloud.

13. Important Mongoose Methods – Quick Revision

Method	**Use**
create()	Create document
insertMany()	Insert multiple documents
find()	Find multiple documents
findOne()	Find one document
findById()	Find by ID
findByIdAndUpdate()	Update by ID
findByIdAndDelete()	Delete by ID
updateOne()	Update first matching document
updateMany()	Update multiple documents
deleteOne()	Delete one document
deleteMany()	Delete matching documents

countDocuments()	Count matching documents
distinct()	Get unique field values
populate()	Resolve references
aggregate()	Run aggregation pipeline
lean()	Return plain JavaScript objects from queries

Q88. What is lean() in Mongoose?

Interview Answer:

lean() tells Mongoose to return plain JavaScript objects instead of full Mongoose documents. It can improve read performance when document methods and change tracking are not needed.

```
const users = await User.find().lean();
```

मराठी: फक्त data वाचायचा आहे आणि Mongoose document features आवश्यक नाहीत, तेव्हा lean() उपयोगी ठरतो.

14. Industry-Level Practical MongoDB Questions

Q89. How do you prevent duplicate email registration?

Answer:

We use a unique index on the email field and handle duplicate key errors in the backend.

```
const userSchema = new mongoose.Schema({
 email: {
  type: String,
  required: true,
  unique: true,
  lowercase: true
 }
});
```

Error handling:

```
try {
```

```
  const user = await User.create(req.body);


  res.status(201).json(user);
 } catch (error) {
  if (error.code === 11000) {
   return res.status(409).json({
    message: "Email already exists"
   });
  }


  res.status(500).json({
   message: "Server error"
  });
 }
```

Q90. How do you search products by name and price?

```
const products = await Product.find({
  name: {
    $regex: search,
    $options: "i"
  },
  price: {
    $lte: 5000
  }
});
```

Q91. How do you calculate total sales?

```
const result = await Order.aggregate([
  {
    $match: {
```

```
    status: "delivered"
   }
  },
  {
   $group: {
    _id: null,
    totalSales: {
     $sum: "$totalAmount"
    }
   }
  }
]);
```

Q92. How do you get monthly expenses?

```
const result = await Expense.aggregate([
 {
  $group: {
   _id: {
    month: {
     $month: "$date"
    },
    year: {
     $year: "$date"
    }
   },
   totalExpense: {
    $sum: "$amount"
   }
  }
```

```
 },
 {
  $sort: {
   "_id.year": 1,
   "_id.month": 1
  }
 }
]);
```

मराठी: प्रत्येक महिन्याचा खर्च वेगळा group करून त्याची बेरीज मिळवतो.

15. Output-Based MongoDB Questions

Q93. What is the output?

```
db.users.insertMany([
 { name: "A", age: 20 },
 { name: "B", age: 25 },
 { name: "C", age: 30 }
]);


db.users.find({
 age: { $gt: 20 }
});
```

Answer: B आणि C हे documents return होतील.

Q94. What is the output?

```
db.users.updateOne(
 { name: "Manoj" },
 {
  $inc: {
   age: 1
  }
```

```
 }
);
```

Answer: Manoj च्या age मध्ये 1 ने वाढ होईल.

Q95. What does this query do?

```
db.users.find({
 age: {
  $gte: 18,
  $lte: 30
 }
});
```

Answer: 18 ते 30 वयोगटातील documents शोधते.

Q96. What does this query return?

```
db.users.distinct("city");
```

Answer: Users collection मधील unique city values return करते.

16. Common MongoDB Interview Mistakes

1. MongoDB ला SQL database म्हणणे चुकीचे आहे.
2. Collection आणि Document यांचा फरक विसरू नये.
3. find() आणि findOne() एकसारखे नाहीत.
4. $set शिवाय update query लिहिताना replacement semantics समजून घ्याव्यात.
5. unique: true हा normal Mongoose validator नाही.
6. Index प्रत्येक field वर तयार करू नये.
7. populate() आणि $lookup हे समान APIs नाहीत; एक Mongoose feature आणि दुसरा MongoDB aggregation stage आहे.
8. MongoDB schema-flexible आहे; म्हणजे validation अजिबात नसते असे नाही.
9. Password plain text मध्ये store करू नये.
10. Production मध्ये database credentials code मध्ये hardcode करू नयेत.

MERN STACK – Complete Interview Questions & Answers

PART 1 – MERN Stack Fundamentals

1. What is MERN Stack?

Interview Answer:

MERN Stack is a full-stack JavaScript technology stack used to build modern web applications. It contains MongoDB, Express.js, React.js, and Node.js.

- MongoDB – Database
- Express.js – Backend framework
- React.js – Frontend library
- Node.js – JavaScript runtime

मराठी स्पष्टीकरण:

MERN Stack वापरून आपण पूर्ण website किंवा web application तयार करू शकतो. Frontend, Backend आणि Database या तिन्ही गोष्टी JavaScript ecosystem मध्ये develop करता येतात.

2. Why do we use MERN Stack?

Interview Answer:

We use MERN Stack because it allows developers to build complete applications using JavaScript. It is flexible, scalable, reusable, and has a large developer community.

मराठीत:

MERN वापरण्याचे फायदे:

1. Frontend आणि Backend साठी JavaScript.
2. React मुळे reusable UI components.
3. MongoDB मुळे flexible document database.
4. Express मुळे backend API development सोपे.
5. Large community आणि packages उपलब्ध.

3. Explain the architecture of MERN Stack.

Interview Answer:

In MERN Stack, React handles the user interface. It sends HTTP requests to the Express.js server running on Node.js. Express processes the request and communicates with MongoDB. Finally, the server sends a response back to React.

Understanding the MERN Stack: Building Interactive and Data-Driven Web Applications | by Anurag Joshi | Medium

medium.com

मराठी स्पष्टीकरण:

समजा User Login करतो:

1. User React मध्ये email आणि password टाकतो.
2. React API request पाठवतो.
3. Express request receive करतो.
4. Middleware request तपासतो.
5. Controller login logic execute करतो.
6. MongoDB मध्ये user शोधला जातो.
7. Password verify केला जातो.
8. Server JWT किंवा cookie द्वारे authentication result पाठवतो.
9. React response नुसार dashboard दाखवतो.

4. What is the difference between Frontend and Backend?

Frontend	**Backend**
User interface	Server-side logic
React	Node.js + Express
Browser मध्ये चालते	Server वर चालते
User interaction handle करते	Business logic handle करते
API request पाठवते	API response देते

5. What is an API?

Interview Answer:

API stands for Application Programming Interface. It allows two different software applications to communicate with each other.

मराठी: API म्हणजे दोन applications मधील communication चा मार्ग.

उदाहरण: React application Node.js backend ला /api/users request पाठवते आणि backend user data return करतो.

6. What is REST API?

Interview Answer:

REST stands for Representational State Transfer. REST API is an architectural style used to communicate between client and server using HTTP methods.

Common HTTP methods:

Method	Purpose	Example
GET	Data मिळवणे	Get all users
POST	नवीन data तयार करणे	Register user
PUT	पूर्ण resource update करणे	Update profile
PATCH	काही fields update करणे	Change username
DELETE	Data delete करणे	Delete user

7. What is the request-response cycle?

Interview Answer:

The request-response cycle is the process in which a client sends a request to the server, the server processes it, and then returns a response.

मराठी: Browser request पाठवतो → Backend request process करतो → Database मधून data घेतो → Response पाठवतो → Frontend UI update करतो.

8. What is JSON?

Interview Answer:

JSON stands for JavaScript Object Notation. It is a lightweight data format used to exchange data between frontend and backend.

```
{
  "name": "Manoj",
```

"email": "manoj@example.com",

"role": "developer"

}

JSON data format आहे; तो स्वतः database किंवा programming language नाही.

9. What is the difference between SQL and NoSQL?

SQL	NoSQL
Relational database	Non-relational database
Tables	Collections
Rows	Documents
Fixed schema structure	Flexible document structure
MySQL	MongoDB

10. Why is JavaScript used in the entire MERN Stack?

Interview Answer:

JavaScript is used in MERN Stack because it allows developers to use one programming language for both frontend and backend development. It also improves code reuse and developer productivity.

PART 2 – MERN Stack Project Architecture

11. Explain a typical MERN Stack folder structure.

एक professional project मध्ये frontend आणि backend वेगळे ठेवले जातात.

my-mern-project/

```
│
├── client/
│   ├── public/
│   └── src/
│       ├── assets/
│       ├── components/
│       ├── pages/
```

```
│      ├── layouts/
│      ├── hooks/
│      ├── context/
│      ├── services/
│      ├── store/
│      ├── utils/
│      ├── App.jsx
│      └── main.jsx
│
├── server/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── services/
│   ├── utils/
│   ├── app.js
│   └── server.js
│
├── .gitignore
└── README.md
```

महत्त्वाचे folders:

Folder	Purpose
components	Reusable React components
pages	Application pages

hooks	Custom React hooks
services	API communication
store	Redux state management
config	Database and environment configuration
controllers	Request handling logic
middleware	Authentication, validation, etc.
models	MongoDB schemas and models
routes	API endpoints
utils	Reusable helper functions

12. Explain MVC Architecture.

Interview Answer:

MVC stands for Model, View, and Controller. It is a software architectural pattern that separates application responsibilities.

- Model – Handles database structure and data.
- View – Displays user interface.
- Controller – Handles application logic and requests.

MERN मध्ये:

- Model → Mongoose Model
- View → React UI
- Controller → Express Controller

13. What is the difference between Routes and Controllers?

Interview Answer:

Routes define API endpoints, while controllers contain the logic that handles those endpoints.

// routes/userRoutes.js

router.get("/", getUsers);

// controllers/userController.js

```
const getUsers = async (req, res) => {
  const users = await User.find();
  res.status(200).json(users);
};
```

मराठी: Route म्हणजे कोणत्या URL वर कोणता controller execute करायचा हे ठरवणे. Controller म्हणजे actual काम करणे.

14. What is Middleware in MERN Stack?

Interview Answer:

Middleware is a function that executes during the request-response cycle. It can access the request object, response object, and the next middleware function.

```
const logger = (req, res, next) => {
  console.log(req.method, req.url);
  next();
};


app.use(logger);
```

Middleware चे उपयोग:

- Authentication
- Authorization
- Error handling
- Request logging
- Validation
- Rate limiting

15. What is the difference between Authentication and Authorization?

Authentication	Authorization
Who are you?	What can you access?
Identity verify करते	Permissions तपासते

Login	Admin/User access
JWT verification	Role checking

उदाहरण: User login करतो ते Authentication. Admin dashboard access करण्याची permission आहे का हे Authorization.

16. What is Environment Variable?

Interview Answer:

Environment variables store configuration values outside the source code. They help protect sensitive information and support different environments.

```
PORT=5000

MONGO_URI=mongodb://localhost:27017/myapp

JWT_SECRET=my_secret_key
```

Node.js मध्ये:

```
require("dotenv").config();


console.log(process.env.PORT);
```

महत्त्वाचे: .env file GitHub वर push करू नये.

17. What is CORS?

Interview Answer:

CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether a frontend application can access resources from another origin.

```
const cors = require("cors");


app.use(cors({
  origin: "http://localhost:5173",
  credentials: true
}));
```

मराठी: Frontend आणि Backend वेगवेगळ्या ports किंवा domains वर असतील, तर browser cross-origin request नियंत्रित करतो. योग्य CORS configuration आवश्यक असते.

PART 3 – React.js in MERN

18. What is React.js?

Interview Answer:

React is a JavaScript library used to build fast and interactive user interfaces. It uses reusable components and a declarative approach to UI development.

19. What is a Component?

Interview Answer:

A component is a reusable piece of UI in React. We can divide a large application into smaller components.

```
function Welcome() {
  return <h1>Welcome to MERN Stack</h1>;
}

export default Welcome;
```

उदाहरण: Navbar, Sidebar, Footer, Login Form हे वेगवेगळे components असू शकतात.

20. What are Props?

Interview Answer:

Props are used to pass data from a parent component to a child component. Props are read-only.

```
function User({ name }) {
  return <h2>Hello {name}</h2>;
}

function App() {
  return <User name="Manoj" />;
}
```

मराठी: Parent component कडून child component ला data पाठवण्यासाठी Props वापरतात.

21. What is State?

Interview Answer:

State is an object or value that stores data inside a component. When state changes, React re-renders the component when necessary.

```
import { useState } from "react";


function Counter() {
 const [count, setCount] = useState(0);


 return (
  <>
   <h2>{count}</h2>
   <button onClick={() => setCount(count + 1)}>
    Increase
   </button>
  </>
 );
}
```

22. Difference between Props and State.

Props	**State**
Parent कडून मिळतात	Component मध्ये manage होतात
Read-only	Setter द्वारे update होतात
Data passing	Dynamic data management
Parent controls values	Owning component controls values

23. What is Virtual DOM?

Interview Answer:

Virtual DOM is a lightweight JavaScript representation of the real DOM. React uses it to calculate UI changes and update the necessary parts of the real DOM.

मराठी: React आधी UI मधील बदलांची virtual representation तयार करतो. नंतर आवश्यक DOM updates करतो.

24. What is Reconciliation?

Interview Answer:

Reconciliation is the process React uses to compare the previous and new UI trees and determine the updates required in the real DOM.

25. What is the difference between useState and useEffect?

useState	**useEffect**
State manage करतो	Side effects handle करतो
State update करतो	External synchronization करतो
UI data ठेवतो	API calls, subscriptions इत्यादी

```
const [users, setUsers] = useState([]);


useEffect(() => {
 fetchUsers();
}, []);
```

26. Explain useEffect dependency array.

```
useEffect(() => {
 console.log("Effect executed");
}, []);
```

Dependency	**Behavior**
No dependency array	प्रत्येक committed render नंतर
[]	Component mount नंतर
[count]	count बदलल्यावर

टीप: React Strict Mode development मध्ये Effects च्या setup आणि cleanup चे अतिरिक्त cycle चालवू शकतो. त्यामुळे development logs दोनदा दिसू शकतात.

27. What is useRef?

Interview Answer:

useRef is a React Hook that stores a mutable value which does not cause a re-render when changed. It is also used to access DOM elements.

```
const inputRef = useRef(null);


const handleFocus = () => {
 inputRef.current.focus();
};


return (
 <>
  <input ref={inputRef} />
  <button onClick={handleFocus}>Focus</button>
 </>
);
```

28. What is useMemo?

Interview Answer:

useMemo is used to cache the result of an expensive calculation between renders.

```
const total = useMemo(() => {
 return products.reduce((sum, item) => sum + item.price, 0);
}, [products]);
```

29. What is useCallback?

Interview Answer:

useCallback is used to cache a function definition between renders.

```
const handleClick = useCallback(() => {
 console.log("Button clicked");
}, []);
```

useMemo vs useCallback:

- useMemo → Calculation result cache.

- useCallback → Function reference cache.

30. What is React Context API?

Interview Answer:

Context API allows us to share data between components without passing props manually through every level.

```
const AuthContext = createContext(null);


function App() {
 return (
  <AuthContext.Provider value={{ user: "Manoj" }}>
   <Dashboard />
  </AuthContext.Provider>
 );
}
```

31. What is Prop Drilling?

Interview Answer:

Prop drilling is the process of passing data through multiple intermediate components to reach a deeply nested child component.

Solution: Context API, Redux Toolkit किंवा योग्य state management approach.

32. What is React Router?

Interview Answer:

React Router is a library used for client-side navigation in React applications without reloading the entire page.

```
<Routes>
 <Route path="/" element={<Home />} />
 <Route path="/login" element={<Login />} />
 <Route path="/dashboard" element={<Dashboard />} />
</Routes>
```

33. What are Controlled and Uncontrolled Components?

Controlled Component: Form input React state द्वारे manage होतो.

```
const [name, setName] = useState("");


<input
 value={name}
 onChange={(e) => setName(e.target.value)}
/>
```

Uncontrolled Component: Input value DOM स्वतः manage करतो; React मध्ये value मिळवण्यासाठी useRef वापरू शकतो.

34. What is Conditional Rendering?

Interview Answer:

Conditional rendering means displaying different UI elements based on a condition.

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

35. Why are keys important in React?

Interview Answer:

Keys help React identify which items have changed, been added, or removed in a list.

```
{users.map((user) => (
 <div key={user._id}>
  {user.name}
 </div>
))}
```

शक्यतो stable unique ID वापरा. Array index हा प्रत्येक परिस्थितीत योग्य key नसतो.

36. What is Lazy Loading?

Interview Answer:

Lazy loading means loading components only when they are needed. It helps reduce the initial JavaScript bundle size.

```
const Dashboard = lazy(() => import("./Dashboard"));
```

यासोबत Suspense वापरतात.

37. What is Redux Toolkit?

Interview Answer:

Redux Toolkit is the recommended way to write Redux logic. It simplifies global state management using slices, reducers, and actions.

Important concepts:

- Store – Global state ठेवतो.
- Slice – State आणि reducers define करतो.
- Reducer – State update करण्याचे logic.
- Action – कोणता update करायचा हे दर्शवतो.
- Dispatch – Action पाठवतो.
- Selector – Store मधून data घेतो.

```
import { createSlice } from "@reduxjs/toolkit";


const cartSlice = createSlice({
 name: "cart",
 initialState: [],
 reducers: {
  addToCart: (state, action) => {
   state.push(action.payload);
  }
 }
});


export const { addToCart } = cartSlice.actions;
export default cartSlice.reducer;
```

Tricky Question: Why can we use push() in Redux Toolkit reducers?

कारण Redux Toolkit मध्ये Immer वापरले जाते. त्यामुळे आपण mutation सारखा दिसणारा code लिहू शकतो, पण Immer immutable state updates तयार करतो.

PART 4 – Node.js and Express.js Integration

38. What is Node.js?

Interview Answer:

Node.js is an open-source JavaScript runtime environment that allows us to execute JavaScript outside the browser. It is commonly used to build backend applications and APIs.

39. What is Express.js?

Interview Answer:

Express.js is a lightweight web framework built on Node.js. It helps us create APIs, handle HTTP requests, manage routes, and use middleware.

40. What is the Event Loop?

Interview Answer:

The Event Loop is a mechanism in Node.js that helps manage asynchronous operations and allows JavaScript to handle non-blocking tasks.

मराठी: Node.js मध्ये JavaScript execution आणि asynchronous कामांचे coordination Event Loop द्वारे होते. त्यामुळे I/O operations पूर्ण होण्याची वाट पाहत संपूर्ण application थांबत नाही.

41. What is Non-blocking I/O?

Interview Answer:

Non-blocking I/O allows Node.js to start an input/output operation without waiting for it to finish before handling other work.

उदाहरण: File read होत असताना इतर requests process होऊ शकतात.

42. What is npm?

Interview Answer:

npm stands for Node Package Manager. It is used to install, manage, and share JavaScript packages.

npm init -y

npm install express

npm install nodemon --save-dev

npm uninstall express

43. What is package.json?

It contains project metadata, dependencies, scripts, and other configuration.

```
{
  "name": "mern-project",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}
```

44. What is the difference between package.json and package-lock.json?

package.json	**package-lock.json**
Project dependencies define करतो	Exact dependency versions lock करतो
Scripts ठेवतो	Dependency tree details ठेवतो
Project configuration	Reproducible installation

45. What is express.json()?

Interview Answer:

express.json() is built-in Express middleware that parses incoming JSON request bodies and makes the parsed data available in req.body.

app.use(express.json());

46. Difference between req.params, req.query and req.body.

Property	**Example**	**Use**
req.params	/users/10	URL मधील dynamic value
req.query	/users?page=2	Filtering/pagination
req.body	POST JSON	Request data

```
app.get("/users/:id", (req, res) => {
  console.log(req.params.id);
```

```
 console.log(req.query.page);
});
```

47. What is Router in Express?

Interview Answer:

Express Router allows us to create modular and reusable route handlers. It helps organize large applications.

```
const router = express.Router();

router.get("/", getUsers);
router.post("/", createUser);

app.use("/api/users", router);
```

48. What is Error Handling Middleware?

Interview Answer:

Error handling middleware is used to handle application errors centrally in Express. It has four parameters: err, req, res, and next.

```
app.use((err, req, res, next) => {
 console.error(err);

 res.status(err.statusCode || 500).json({
  success: false,
  message: err.message || "Internal Server Error"
 });
});
```

49. What is the difference between PUT and PATCH?

- PUT – Resource पूर्णपणे replace करण्यासाठी वापरतात.
- PATCH – Resource मधील काही fields update करण्यासाठी वापरतात.

50. What are HTTP Status Codes?

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Unprocessable Content
500	Internal Server Error

PART 5 – MongoDB and Mongoose Integration

51. What is MongoDB?

Interview Answer:

MongoDB is a NoSQL document-oriented database. It stores data in BSON documents instead of traditional rows and tables.

52. What is a Collection?

A collection is a group of MongoDB documents. It is similar to a table in relational databases.

53. What is a Document?

A document is an individual record stored in a MongoDB collection.

```
{
  "_id": "123",
  "name": "Manoj",
  "email": "manoj@example.com",
  "age": 22
}
```

54. What is BSON?

Interview Answer:

BSON stands for Binary JSON. It is the binary-encoded document format used by MongoDB. It supports additional data types beyond standard JSON.

55. What is Mongoose?

Interview Answer:

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It provides schemas, models, validation, middleware, and query helpers.

56. What is Schema and Model?

Schema: Defines the structure and rules of documents.

Model: Provides an interface to interact with a MongoDB collection.

```
const mongoose = require("mongoose");


const userSchema = new mongoose.Schema({
 name: {
  type: String,
  required: true
 },
 email: {
  type: String,
  required: true,
  unique: true
 },
 age: Number
});


const User = mongoose.model("User", userSchema);
```

57. What is the difference between Schema and Model?

Schema	Model
Document structure define करतो	Database operations करतो
Rules आणि validation	Create, read, update, delete
Blueprint	Database interface

58. What is ObjectId?

ObjectId is a BSON data type commonly used as the unique identifier of a MongoDB document.

59. What is CRUD?

CRUD stands for Create, Read, Update, and Delete.

```
// Create
await User.create({ name: "Manoj" });


// Read
await User.find();


// Update
await User.findByIdAndUpdate(id, { name: "Rahul" });


// Delete
await User.findByIdAndDelete(id);
```

60. What is the difference between find() and findOne()?

- find() – Matching documents चा array return करतो.
- findOne() – पहिला matching document return करतो किंवा null.

61. What is populate() in Mongoose?

Interview Answer:

populate() is used to replace referenced document IDs with actual document data from another collection.

```
const orderSchema = new mongoose.Schema({
 user: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "User"
 }
});


const orders = await Order.find().populate("user");
```

मराठी: एका collection मध्ये दुसऱ्या collection चा ObjectId store केला असेल, तर संबंधित document ची माहिती मिळवण्यासाठी populate वापरतो.

62. What is the difference between embed and reference?

Embedding	**Referencing**
एका document मध्ये nested data	दुसऱ्या document चा ID
Related data एकत्र ठेवतो	Related data वेगळे ठेवतो
Read सोपे होऊ शकते	Data duplication कमी होऊ शकते
Document size वाढू शकतो	Populate किंवा lookup लागू शकतो

उदाहरण:

- User आणि त्याचे छोटे address details – Embedding योग्य ठरू शकते.
- User आणि हजारो orders – Referencing योग्य ठरू शकते.

63. What is Indexing in MongoDB?

Interview Answer:

Indexing improves query performance by allowing MongoDB to find documents more efficiently without scanning every document.

```
userSchema.index({ email: 1 });
```

1 म्हणजे ascending आणि -1 म्हणजे descending index order.

Disadvantage: Indexes अतिरिक्त storage घेतात आणि write operations चा खर्च वाढवतात.

64. What is Aggregation?

Interview Answer:

Aggregation is used to process documents and perform operations such as filtering, grouping, sorting, and calculating values.

```
const result = await Order.aggregate([
 {
  $group: {
   _id: "$status",
   totalOrders: { $sum: 1 }
  }
 }
]);
```

महत्त्वाचे stages:

- $match – Filter
- $group – Group data
- $sort – Sort
- $project – Select or reshape fields
- $limit – Limit results
- $lookup – Join-like operation
- $unwind – Array elements वेगळे करणे

65. What is the difference between deleteOne and deleteMany?

- deleteOne() – एक matching document delete करतो.
- deleteMany() – सर्व matching documents delete करतो.

66. What is Validation in Mongoose?

Validation checks whether the data satisfies defined rules before saving it.

```
const userSchema = new mongoose.Schema({
 age: {
  type: Number,
```

```
  min: 18,
  max: 60,
  required: true
 }
});
```

महत्त्वाचे: Mongoose validation आणि unique: true यांचा अर्थ सारखा नाही. unique unique index तयार करण्यासाठी मदत करतो; तो सामान्य validator नाही.

PART 6 – Authentication and Authorization

हा भाग MERN interview मध्ये खूप महत्त्वाचा आहे.

67. What is Authentication?

Authentication is the process of verifying a user's identity.

उदाहरण: Email आणि password वापरून Login.

68. What is Authorization?

Authorization determines what an authenticated user is allowed to access.

उदाहरण: Admin ला users delete करण्याची permission असू शकते, पण normal user ला नाही.

69. What is JWT?

Interview Answer:

JWT stands for JSON Web Token. It is a compact token format used to transfer claims between parties. In web applications, it is commonly used for authentication and authorization.

JWT चे तीन भाग असतात:

1. Header
2. Payload
3. Signature

Mastering API Security: JWT (JSON Web Tokens) Authentication in .NET Core | by Md Julakadar | Medium

medium.com

```
const jwt = require("jsonwebtoken");
```

```
const token = jwt.sign(
 { userId: user._id },
 process.env.JWT_SECRET,
 { expiresIn: "1d" }
);
```

70. What is bcrypt?

Interview Answer:

bcrypt is a password hashing library. It converts passwords into secure hashes so that plain-text passwords are not stored in the database.

```
const bcrypt = require("bcryptjs");


const hashedPassword = await bcrypt.hash(password, 10);


const isMatch = await bcrypt.compare(
 password,
 hashedPassword
);
```

महत्वाचे: Password encrypt करण्याऐवजी hash केला जातो. Hashing सामान्यतः one-way process असते.

71. What is the difference between Hashing and Encryption?

Hashing	**Encryption**
Generally one-way	Reversible with key
Password storage	Secure data transmission/storage
bcrypt, Argon2	AES

Original password recover करणे अपेक्षित नाही योग्य key ने data decrypt करता येतो

72. What is Cookie?

Interview Answer:

A cookie is a small piece of data stored by the browser. It can be sent automatically with matching HTTP requests based on its domain, path, and other settings.

```
res.cookie("token", token, {
 httpOnly: true,
 secure: true,
 sameSite: "none"
});
```

73. What is HttpOnly Cookie?

HttpOnly cookie cannot be accessed through JavaScript using document.cookie. It helps reduce the risk of token theft through certain XSS attacks.

74. What is the difference between LocalStorage and Cookies?

LocalStorage	**Cookies**
Browser storage	Browser-managed storage
JS access शक्य	HttpOnly असल्यास JS access नाही
Request सोबत automatically जात नाही	Matching requests सोबत जाऊ शकते
Expiry स्वतः manage करावी	Expiry सेट करता येते

75. What is Role-Based Access Control (RBAC)?

Interview Answer:

RBAC is an authorization method in which permissions are assigned based on user roles.

```
const authorize = (...roles) => {
 return (req, res, next) => {
  if (!roles.includes(req.user.role)) {
   return res.status(403).json({
    message: "Access denied"
   });
  }
```

```
  next();
 };
};
```

उदाहरण: admin, user, manager.

76. Explain Login Flow in MERN Stack.

[My Experience of Full-Stack Ecommerce Platform with MERN Stack with Framer Motion Animation | by Nalin sandaruwan | Medium]

[medium.com]

1. User login form submit करतो.
2. React backend ला POST request पाठवतो.
3. Express login route execute करतो.
4. User database मध्ये शोधला जातो.
5. bcrypt password compare करतो.
6. Password योग्य असल्यास JWT generate करतो.
7. JWT सुरक्षित cookie मध्ये set करतो.
8. Browser पुढील requests मध्ये cookie पाठवतो.
9. Authentication middleware token verify करतो.
10. Authorization middleware role तपासतो.

PART 7 – MERN Security

77. What is SQL Injection / NoSQL Injection?

Interview Answer:

Injection attacks happen when untrusted input is interpreted as part of a database query or command. Attackers may manipulate queries to access or modify data.

Protection:

- Input validation
- Safe query construction

- Avoid unsafe dynamic query objects
- Use appropriate database security controls

78. What is XSS?

XSS stands for Cross-Site Scripting. It is an attack in which malicious scripts are injected into web pages viewed by other users.

Protection:

- Escape untrusted output.
- Sanitize HTML where HTML input is required.
- Avoid unsafe dangerouslySetInnerHTML.
- Use a suitable Content Security Policy.

79. What is CSRF?

CSRF stands for Cross-Site Request Forgery. It tricks an authenticated browser into sending an unwanted request to a website.

Protection:

- CSRF tokens
- SameSite cookies
- Origin/Referer validation
- Proper request design

80. What is Rate Limiting?

Rate limiting restricts the number of requests a client can make within a specific time period.

उदाहरण: Login API वर एका IP कडून ठरावीक वेळेत जास्त attempts होऊ नयेत.

81. How do you secure a MERN application?

Interview Answer:

I secure a MERN application by validating user input, hashing passwords, using secure authentication, protecting API routes, configuring CORS correctly, adding rate limiting, and storing secrets in environment variables.

अतिरिक्त उपाय:

- HTTPS वापरणे.

- Secure, HttpOnly आणि योग्य SameSite cookies.
- Role-based authorization.
- Request validation.
- Dependency updates.
- Database access control.
- Error responses मध्ये sensitive details न दाखवणे.

PART 8 – Frontend and Backend Communication

82. How does React communicate with Node.js?

Interview Answer:

React communicates with the Node.js backend through HTTP requests using tools like Fetch API or Axios.

```
const getUsers = async () => {
 try {
  const response = await fetch(
   "http://localhost:5000/api/users"
  );

  if (!response.ok) {
   throw new Error("Failed to fetch users");
  }

  const data = await response.json();
  console.log(data);
 } catch (error) {
  console.error(error.message);
 }
};
```

83. What is Axios?

Interview Answer:

Axios is a promise-based HTTP client used to send requests from frontend or backend applications.

```
import axios from "axios";


const response = await axios.get("/api/users");
```

84. Difference between Fetch and Axios.

Fetch	**Axios**
Built-in browser API	External library
HTTP errors स्वतः check करावे लागतात	Non-2xx responses reject होतात
JSON parsing manually	JSON response data सहज मिळते
Basic HTTP client	Interceptors आणि additional features

85. What is Axios Interceptor?

Interview Answer:

Axios interceptors allow us to run logic before a request is sent or after a response is received.

```
axios.interceptors.response.use(
 (response) => response,
 (error) => {
  return Promise.reject(error);
 }
);
```

उपयोग:

- Common error handling
- Token handling
- Request logging
- Response processing

86. What is Pagination?

Interview Answer:

Pagination divides a large dataset into smaller pages. It improves performance and user experience by loading limited records at a time.

```
const page = 1;

const limit = 10;


const users = await User.find()

 .skip((page - 1) * limit)

 .limit(limit);
```

87. What is Search and Filtering?

Search वापरून specific records शोधतात, तर filtering वापरून condition नुसार records निवडतात.

```
const users = await User.find({

 name: { $regex: search, $options: "i" }

});
```

Production मध्ये user input validate करा आणि query complexity नियंत्रित ठेवा.

88. What is Debouncing?

Interview Answer:

Debouncing delays function execution until a certain amount of time has passed without another call.

उदाहरण: Search box मध्ये user typing थांबवल्यावर API call करणे.

89. What is Throttling?

Interview Answer:

Throttling ensures that a function executes at most once within a specified time interval.

उदाहरण: Scroll event handler वारंवार execute होऊ नये.

PART 9 – File Upload and Real-Time Communication

90. How do you upload files in MERN Stack?

Interview Answer:

In MERN Stack, we can upload files using Multer on the backend and FormData on the frontend. Files can then be stored locally or in cloud storage.

Frontend:

```
const formData = new FormData();


formData.append("image", file);


await axios.post("/api/upload", formData);
```

Backend:

```
const multer = require("multer");


const upload = multer({
 dest: "uploads/"
});


app.post("/upload", upload.single("image"), (req, res) => {
 res.json({
  message: "File uploaded",
  file: req.file
 });
});
```

Production मध्ये file type, file size, authorization आणि storage security तपासणे आवश्यक आहे.

91. What is Cloudinary?

Interview Answer:

Cloudinary is a cloud-based media management platform used to upload, store, optimize, and deliver images and videos.

92. What is Socket.IO?

Interview Answer:

Socket.IO is a library that enables real-time, bidirectional communication between clients and servers. It is commonly used for chat applications, notifications, and live updates.

Understanding socket.io , web sosckets , socket io in javascript

gowtham-uj.hashnode.dev

उदाहरण:

- Real-time chat
- Typing indicators
- Online/offline status
- Live notifications

93. Difference between HTTP and WebSocket.

HTTP	WebSocket
Request-response model	Persistent connection
प्रत्येक interaction मध्ये request	दोन्ही बाजूंनी messages
REST API	Real-time chat
Stateless protocol	Stateful connection

PART 10 – Deployment and Production

94. What is Deployment?

Interview Answer:

Deployment is the process of making an application available to users on a production server or hosting platform.

95. What is the difference between Development and Production?

Development	Production
Local environment	Live environment
Debugging enabled	Controlled logging

Test data	Real user data
Frequent changes	Stable release
Local URLs	Production domains

96. What is Build in React?

Interview Answer:

A production build prepares the React application for deployment by bundling and optimizing the application assets.

Vite:

npm run build

97. What is CI/CD?

Interview Answer:

CI/CD stands for Continuous Integration and Continuous Delivery or Deployment. It automates building, testing, and releasing software.

98. What is Reverse Proxy?

Interview Answer:

A reverse proxy receives client requests and forwards them to backend servers. It can also handle SSL termination, load balancing, and routing.

उदाहरण: Nginx.

99. What is Load Balancing?

Load balancing distributes incoming traffic across multiple servers to improve availability and performance.

100. How do you handle errors in a production MERN application?

Interview Answer:

I use centralized error handling, proper HTTP status codes, input validation, structured logging, and meaningful error messages. I avoid exposing sensitive internal details to users.

PART 11 – Practical MERN Coding Interview Questions

या प्रश्नांचा code स्वतः लिहून practice कर. Interview मध्ये logic explain करणेही महत्त्वाचे आहे.

101. Create a User Schema.

```
const mongoose = require("mongoose");


const userSchema = new mongoose.Schema(
 {
  name: {
   type: String,
   required: true,
   trim: true
  },
  email: {
   type: String,
   required: true,
   unique: true,
   lowercase: true
  },
  password: {
   type: String,
   required: true
  },
  role: {
   type: String,
   enum: ["user", "admin"],
   default: "user"
  }
 },
 {
  timestamps: true
 }
```

```
);

module.exports = mongoose.model("User", userSchema);
```

102. Create a Register API.

```
const bcrypt = require("bcryptjs");
const User = require("../models/User");

const registerUser = async (req, res, next) => {
 try {
  const { name, email, password } = req.body;

  if (!name || !email || !password) {
   return res.status(400).json({
    message: "All fields are required"
   });
  }

  const existingUser = await User.findOne({ email });

  if (existingUser) {
   return res.status(409).json({
    message: "User already exists"
   });
  }

  const hashedPassword = await bcrypt.hash(password, 12);

  const user = await User.create({
```

```
      name,
      email,
      password: hashedPassword
    });

    res.status(201).json({
      message: "Registration successful",
      user: {
        id: user._id,
        name: user.name,
        email: user.email
      }
    });
  } catch (error) {
    next(error);
  }
};
```

Interview मध्ये explain करायचे:

1. Request body मधून user data घेतो.
2. Required fields validate करतो.
3. Existing user तपासतो.
4. Password hash करतो.
5. MongoDB मध्ये user create करतो.
6. Success response पाठवतो.
7. Error middleware कडे errors पाठवतो.

टीप: Production API मध्ये request validation, normalized email, duplicate-key database errors आणि password policy देखील योग्य प्रकारे handle करा.

103. Create a Login API.

```
const loginUser = async (req, res, next) => {
 try {
  const { email, password } = req.body;

  const user = await User.findOne({ email });

  if (!user) {
   return res.status(401).json({
    message: "Invalid email or password"
   });
  }

  const isMatch = await bcrypt.compare(
   password,
   user.password
  );

  if (!isMatch) {
   return res.status(401).json({
    message: "Invalid email or password"
   });
  }

  res.status(200).json({
   message: "Login successful"
  });
 } catch (error) {
```

```
  next(error);
 }
};
```

हा example password verification दाखवतो. Production login मध्ये successful verification नंतर token किंवा secure session तयार करणे, सुरक्षित cookie set करणे आणि login rate limiting जोडणे आवश्यक आहे.

104. Create a GET API.

```
const getUsers = async (req, res, next) => {
 try {
  const users = await User.find().select("-password");

  res.status(200).json({
   success: true,
   count: users.length,
   users
  });
 } catch (error) {
  next(error);
 }
};
```

105. Create an UPDATE API.

```
const updateUser = async (req, res, next) => {
 try {
  const user = await User.findByIdAndUpdate(
   req.params.id,
   req.body,
   {
    new: true,
```

```
      runValidators: true
    }
  ).select("-password");

  if (!user) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  res.status(200).json({
    message: "User updated successfully",
    user
  });
 } catch (error) {
  next(error);
 }
};
```

Production मध्ये user ला कोणते fields update करण्याची permission आहे ते whitelist करावे. थेट पूर्ण req.body update करणे धोकादायक ठरू शकते.

106. Create a DELETE API.

```
const deleteUser = async (req, res, next) => {
 try {
  const user = await User.findByIdAndDelete(
    req.params.id
  );

  if (!user) {
```

```
    return res.status(404).json({
      message: "User not found"
    });
   }

   res.status(200).json({
    message: "User deleted successfully"
   });
  } catch (error) {
   next(error);
  }
};
```

107. Create Authentication Middleware.

```
const jwt = require("jsonwebtoken");

const protect = (req, res, next) => {
 try {
  const token = req.cookies.token;

  if (!token) {
   return res.status(401).json({
     message: "Not authenticated"
   });
  }

  const decoded = jwt.verify(
   token,
   process.env.JWT_SECRET
```

```
    );

    req.user = decoded;

    next();
  } catch (error) {
    return res.status(401).json({
      message: "Invalid or expired token"
    });
  }
};

module.exports = protect;
```

टीप: या example मध्ये cookie-parser middleware configure केलेले असणे आवश्यक आहे.

108. Create Protected Route.

```
router.get("/profile", protect, getProfile);
```

मराठी: आधी protect middleware token verify करतो. Token योग्य असल्यासच getProfile controller execute होतो.

PART 12 – Tricky MERN Interview Questions

हे प्रश्न interview मध्ये concept समजला आहे का ते तपासतात.

109. Can React directly connect to MongoDB?

Answer: No. React should not connect directly to MongoDB. It communicates with the backend API, and the backend securely accesses the database.

110. Can Node.js work without Express.js?

Answer: Yes. Node.js has a built-in HTTP module. Express makes backend development easier by providing routing and middleware features.

111. Is MongoDB schema-less?

Answer: MongoDB has a flexible document model. It does not require every document to follow a fixed relational schema, but applications can enforce structure using schema validation or Mongoose.

112. Is React a framework?

Answer: React is a JavaScript library, not a full framework.

113. Is Node.js single-threaded?

Answer: JavaScript execution in a typical Node.js event loop runs on a main thread, but Node.js also uses other threads and system-level mechanisms for certain operations.

114. Does useEffect run before rendering?

Answer: No. useEffect runs after React commits the component's changes to the DOM. useLayoutEffect runs earlier, after DOM changes but before the browser paints.

115. Why should we not directly modify React state?

Answer: Direct mutation can prevent React from detecting changes correctly and can lead to unexpected UI behavior. We should use state setters and immutable update patterns.

116. Why do we use async/await?

Answer: async/await makes asynchronous JavaScript code easier to read and maintain. It is built on Promises.

117. What happens when the server receives 1,000 requests?

Answer: Node.js can handle many concurrent I/O operations using its event-driven, non-blocking architecture. However, actual throughput depends on CPU work, database capacity, memory, network, and application design.

118. What is the difference between 401 and 403?

- 401 – Authentication is missing or invalid.
- 403 – The user is authenticated but does not have permission.

119. What is the difference between == and === in JavaScript?

- == – Compares values after type coercion.
- === – Compares values and types without type coercion.

120. Why do we use timestamps in MongoDB?

Answer: Timestamps help track when documents were created and updated. Mongoose can automatically manage createdAt and updatedAt.

PART 13 – Real-World Project-Based Questions

Interviewers अनेकदा विचारतात: "Explain your project and your contribution."

खालील प्रश्नांची उत्तरे तयार ठेव.

Interview Question	**काय explain करायचे?**
Explain your project.	Problem, purpose, features, tech stack
What was your role?	तुमचे actual contribution
Explain your database design.	Collections, schemas, relationships
How did you implement authentication?	bcrypt, JWT, cookies
How did you manage state?	useState, Context, Redux, Zustand
How did you handle API errors?	try/catch, error middleware
How did you upload images?	Multer, Cloudinary
How did you protect routes?	Authentication middleware
How did you improve performance?	Pagination, indexes, optimization
What challenges did you face?	Actual problems and solutions

3 important project scenarios

Scenario 1: E-commerce application

Interviewer: How do you manage a shopping cart?

उत्तरात explain कर:

- Product selection.
- Cart state management.
- Quantity updates.
- Price calculation.
- Backend validation.
- Order creation.
- Stock management.

Scenario 2: Real-time Chat Application

Interviewer: How do you implement real-time messaging?

उत्तरात explain कर:

- Socket.IO connection.
- User authentication.
- Chat room management.
- Send/receive message events.
- Message storage in MongoDB.
- Typing indicator.
- Online/offline status.

Scenario 3: Income and Expense Tracker

Interviewer: How do you calculate total income and expenses?

उत्तरात explain कर:

- Transaction model.
- Income/expense categories.
- Date filtering.
- Aggregation pipeline.
- Monthly reports.
- Charts and dashboard.